Numerical Optimization for Large Scale Problems
Report for Final Assignment

Alessia Serra (s359693), Enrico Signorini (s362974)

Winter Call - January 2026

Contents

1 Introduction **4**

2 Implemented methods **4**
2.1 Modified Newton Method with Backtracking Line Search 4
2.2 Truncated Newton Method with Conjugate Gradient 5

3 Test problems **6**
3.1 Broyden Tridiagonal 6
3.2 Chained Freudenstein Roth 6

4 Efficient finite differences implementation **7**
4.1 Broyden Tridiagonal 7
4.1.1 Approximation of the Hessian starting from the exact gradient - approximation mode 1 7
4.1.2 Approximation of the Gradient with first order FD - approximation mode 2 8
4.1.3 Approximation of the Hessian with second order FD - approximation mode 2 8
4.2 Chained Freudstein Roth 9
4.2.1 Approximation of the Hessian starting from the exact gradient - approximation mode 1 9
4.2.2 Approximation of the Gradient with first order FD - approximation mode 2 9
4.2.3 Approximation of the Hessian with second order FD - approximation mode 2 9

5 Results and comments **10**
5.1 Chained Freudestein-Roth 10
5.1.1 Exact Derivatives 10
5.1.2 Approximate derivatives 10
5.2 Broyden Tridiagonal 11
5.2.1 Exact Derivatives 11
5.2.2 Approximate derivatives 11
5.3 Theoretical vs. Observed Behavior 11

6 Plots **12**
6.1 Low Dimensional Analysis ($n = 2$) 12
6.1.1 Tridimensional view 12
6.1.2 Chained Freudenstein-Roth Optimization Trajectories ($n = 2$) 12
6.1.3 Chained Freudstein-roth, Rates of convergence ($n = 2$) 17
6.1.4 Broyden Tridiagonal, Optimization Trajectories ($n = 2$) 22
6.1.5 Broyden Tridiagonal, Rates of convergence ($n = 2$) 27
6.2 Medium Scale Analysis ($n = 10^3$) 32
6.2.1 Chained Freudstain-Roth, Rates of convergence ($n = 10^3$) 32
6.2.2 Broyden Tridiagonal, Rates of convergence ($n = 10^3$) 34
6.3 Large Scale Analysis ($n = 10^4$ and $n = 10^5$) 39
6.3.1 Chained Freudstain-Roth, Rates of convergence ($n = 10^4$) 39
6.3.2 Bryden Tridiagonal, Rates of convergence ($n = 10^4$) 42
6.3.3 Chained Freudstain-Roth, Rates of convergence ($n = 10^5$) 47
6.3.4 Broyden Tridiagonal, Rates of convergence ($n = 10^5$) 51

7 Code **56**
7.1 Newton methods 56
7.1.1 Modified Newton 56
7.1.2 Truncated Newton 57
7.2 Functions 59
7.2.1 Broyden Tridiagonal 59
7.2.2 Chained Freudenstein-Roth function 60
7.3 Efficient implementation of functions' derivatives using the exact gradient 60
7.3.1 Efficient implementation - Broyden Tridiagonal/ mode 1 60
7.3.2 Efficient implementation - Chained-Freudenstein-Roth/ mode 1 61

7.4 Efficient implementation of ALL functions' derivatives 62
7.4.1 Efficient implementation - Broyden Tridiagonal / mode 2 62
7.4.2 Efficient implementation - Chained Freudenstein Roth / mode 2 64
7.5 Main 67

8 Tables **75**

1 Introduction

In this report, we present the analysis and results for **Assignment 2.1**, which focuses on the implementation and testing of numerical methods designed for **unconstrained optimization**. This field of study involves finding the minimum value of a real-valued function where the variables are not subject to any restrictions, allowing the optimization algorithm to explore the entire multidimensional space freely to identify the optimal solution. In the field of large-scale unconstrained optimization, selecting an appropriate numerical strategy is essential because the efficiency of an algorithm often depends on its ability to handle high-dimensional spaces without consuming excessive computational resources. While second-order methods are highly valued for their fast convergence rates, they frequently face significant challenges when the cost of calculating or factoring the Hessian matrix becomes prohibitive for very large problems. This assignment investigates two primary second-order algorithms that offer distinct solutions to the problems of scalability and numerical stability: the **Modified Newton Method** and the **Truncated Newton Method**. To see how these methods perform in practice, we benchmark them against two mathematical functions that have specific internal structures: Broyden Tridiagonal, in which each term only depends on three variables at a time (x_{k-1}, x_k, x_{k+1}), and creates a pentadiagonal Hessian matrix, and Chained Freudenstein-Roth, with more complex pattern where variables are linked in overlapping pairs (x_i, x_{i+1}), resulting in a tridiagonal Hessian matrix. We analyzed the performance of these algorithms across various problem sizes, reaching up to $n = 10^5$ variables, to measure the specific trade-offs between efficiency and memory usage. While the direct solver approach is very efficient in terms of the number of steps it takes to reach a solution, it becomes extremely demanding on computer resources as the problem grows, whereas the iterative approach scales much better even though it is more sensitive to numerical errors. Furthermore, our analysis examines how "noise" or small inaccuracies in the derivative calculations can slow down the convergence speed, providing a clear overview of which methods remain stable and efficient in high-performance computing environments.

2 Implemented methods

The methods chosen to address these optimization tasks are **the Modified Newton Method with Backtracking Line Search** and **Truncated Newton Method** with the use of **Conjugate Gradient**. Both represent practical modifications of the classical Newton Method, a second-order optimization algorithm. While the standard Newton method offers quadratic convergence near the solution (Theorem 3.5 from [1]), it is affected by significant limitations: it requires the Hessian matrix to be positive definite at each iteration, involves expensive computations such as matrix factorizations, Hessian computation and solving linear systems. Moreover, it is very sensitive to the choice of the initial guess x_0. The two implemented variants address these lacks through different strategies.

2.1 Modified Newton Method with Backtracking Line Search

The **Modified Newton** method addresses the positive definiteness requirement of the Hessian in the classical Newton method by introducing a regularization term.
The modification procedure begins by examining the minimum eigenvalue of the Hessian H_k: when negative or null eigenvalues are found, a positive parameter τ is added to the Hessian, obtaining

$$B_k = H_k + \tau I,$$

where I denotes the identity matrix. The parameter τ is initialized with respect to the magnitude of the most negative eigenvalue and is iteratively increased until the Cholesky factorization of B_k succeeds. This procedure ensures that the resulting direction p_k is always a descent direction, preventing convergence to saddle points or local maximizers. Then, when a valid matrix B_k is obtained, its Cholesky factorization $B_k = LL^T$ is computed, and the descent direction is efficiently determined using forward and backward substitution

$$B_k p_k = -\nabla f(x_k),$$

so, to compute p_k, we exploit the structure of the Cholesky factorization computed before. This allows us to reduce the computational cost for solving the linear system from $O(n^3)$ (cost associated to general matrix inversion) to $O(n^2)$.

The step size along the descent direction is selected using an **Armijo backtracking line search**. Starting from $\alpha = 1$, to guarantee a quadratic rate of convergence near the solution x^*, the step size is successively reduced by a factor ρ until the Armijo condition is satisfied,

$$f(x_k + \alpha p_k) \leq f(x_k) + c_1 \alpha \nabla f(x_k)^T p_k$$

where c_1 is a small positive constant. This strategy guarantees sufficient decrease of the objective function at each iteration and ensures global convergence from arbitrary starting points, as established by **Zoutendijk's Theorem**.

The algorithm ends when the gradient norm has a value smaller than the treshold `tolgrad` or when the maximum number of iterations `kmax` is reached. The complete implementation is provided in 7.

2.2 Truncated Newton Method with Conjugate Gradient

The **Truncated Newton method** reduces the computational cost of the classical Newton approach by approximately solving the Newton system $H_k p_k = -\nabla f(x_k)$ using the **Conjugate Gradient** algorithm. This strategy is particularly convenient for large-scale problems where Hessian factorization is very expensive, because it requires only Hessian-vector products instead of matrix factorizations.

At each outer iteration k, the Conjugate Gradient (CG) method starts with $z_0 = 0$ and builds a sequence of search directions to progressively improve the solution. At each CG iteration j, the residual

$$r_j = -\nabla f(x_k) - H_k z_j$$

and the search direction d_j are computed. The step size along d_j is given by

$$\alpha_j = \frac{r_j^T r_j}{d_j^T H_k d_j},$$

which minimizes the local quadratic approximation. The next search direction is defined as

$$d_{j+1} = r_{j+1} + \beta_j d_j, \qquad \beta_j = \frac{r_{j+1}^T r_{j+1}}{r_j^T r_j},$$

which ensures H_k-conjugacy of the directions.

The CG iterations can terminate early according to two criteria. First, if

$$d_j^T H_k d_j \leq 0,$$

the Hessian is not positive definite along the direction d_j. In this case, the CG loop is stopped: if this occurs at $j = 1$, the steepest descent direction $-\nabla f(x_k)$ is used; otherwise, the current iterate z_j is selected.

Second, an adaptive stopping rule is used: the CG loop terminates when

$$\|r_j\| \leq \eta_k \|\nabla f(x_k)\|, \qquad \eta_k = \min\left(0.5, \sqrt{\|\nabla f(x_k)\|}\right).$$

This strategy ensures that the Newton system is solved more accurately near the optimum, reducing computational effort when far from the solution. Once an approximate Newton direction p_k is obtained from CG, an Armijo backtracking line search is used to determine the step size α_k, as in the Modified Newton method, to guarantee sufficient decrease of the objective function at each iteration. The algorithm terminates when the gradient norm is sufficiently small or when the maximum number of iterations is reached.

The main advantage of this approach is its computational efficiency. Each CG iteration requires only one Hessian-vector product, with a cost of $O(n)$ or $O(n \log n)$ depending on the problem structure. This is significantly cheaper than the $O(n^3)$ cost of Cholesky factorization required by the Modified Newton method. Moreover, the early stopping strategy avoids unnecessary iterations while solving the Newton system when an approximate solution is sufficient.

3 Test problems

Test functions represent different types of challenges: **Broyden Tridiagonal** is more uniform and sparse, while **Chained Freudenstein-Roth** is asymmetric and has a more complex interaction pattern. Testing the Truncated Newton and Modified Newton methods on these two problems shows how well they handle the different properties owned by the functions and how their performance scales with problem dimension.

3.1 Broyden Tridiagonal

Broyden Tridiagonal function (Problem no.31 in [2]) is defined as:

$$F(x) = \frac{1}{2}\sum_{k=1}^{m} f_k^2(x), \quad f_k(x) = (3-2x_k)x_k - x_{k-1} - 2x_{k+1} + 1$$

$$m = n, \quad x_0 = x_{n+1} = 0, \quad \bar{x}_l = -1$$

The suggested starting point $\bar{x}_l$ leads the algorithm to a complicated path, as the search space near it presents many stationary points. This function generates a sparse Hessian matrix with a **penta-diagonal structure**. Since each term f_k only depends on three consecutive terms (x_{k-1}, x_k, x_{k+1}), the interactions between variables are local. However, these local relations create a "chain effect" where a change in one variable affects the neighbors, which implies a change the next ones, and so on. The result of these interactions is that the structure becomes more difficult to solve as the dimension n increases, because the indirect link between distant variables becomes stronger and the optimization landscape develops more complex features that can slow down convergence. From the perspective of our assignment, this function is very useful for testing the Truncated Newton method: since the Hessian is sparse with only three non-zero elements per row, the linear system for the Newton step can be solved much faster than in dense problems using an iterative solver. Furthermore, the presence of quadratic terms in the function let us to see how the accuracy of the finite differences changes with different values of h. In fact, if the approximation of the gradient or the Hessian is not precise enough, the algorithm might take a much longer path to find the minimum or could not behave with typical fast convergence of Newton-type methods, showing how repeated numerical cancellation can affect the overall performace.
To compute the exact derivatives of the Broyden Tridiagonal function, we first derive the Jacobian matrix J of the vector-valued function $f(x)$, where each component f_k exhibits a tridiagonal dependence resulting in the partial derivatives $\frac{\partial f_k}{\partial x_{k-1}} = -1$, $\frac{\partial f_k}{\partial x_k} = 3 - 4x_k$, and $\frac{\partial f_k}{\partial x_{k+1}} = -2$. The gradient $\nabla F(x)$ is then efficiently evaluated using the identity $g = J^T f$; in the implementation, this is achieved through vectorized operations that account for the shifted indices of the off-diagonal elements, specifically by combining the term $(3 - 4x_k)f_k$ with the contributions from the adjacent residuals f_{k-1} and f_{k+1}. For the second-order information, the exact Hessian $\nabla^2 F(x)$ is constructed by summing the Gauss-Newton approximation $J^T J$ and the second-order term $\sum_{k=1}^{n} f_k \nabla^2 f_k$. Since each residual f_k is a quadratic form, its only non-zero second partial derivative is $\frac{\partial^2 f_k}{\partial x_k^2} = -4$, which simplifies the second-order summation into a purely diagonal "curvature" matrix. This approach ensures that the resulting Hessian preserves a sparse, banded structure while capturing the full curvature of the optimization landscape, allowing for the high-precision steps required by the Truncated Newton method.

3.2 Chained Freudenstein Roth

The Chained Freudstein Roth function (Problem no.34 in [2]) is defined as:

$$F(x) = \frac{1}{2}\sum_{k=1}^{m} f_k^2(x), \quad f_k(x) = \begin{cases} x_i + x_{i+1}((5 - x_{i+1})x_{i+1} - 2) - 13, & \mathrm{mod}(k,2) = 1 \\ x_i + x_{i+1}((1 + x_{i+1})x_{i+1} - 14) - 29, & \mathrm{mod}(k,2) = 0 \end{cases}$$

$$m = 2(n-1), \quad i = \lfloor (k+1)/2 \rfloor, \quad \bar{x}_l = \begin{cases} 0.5, & l < n \\ -2, & l = n \end{cases}$$

The particular feature of this function is that each residual f_k, for $k = 1, \ldots, m$, depends only on two consecutive components (x_i, x_{i+1}), where the index i is determined by the relation

$i = \lfloor (k+1)/2 \rfloor$. Consequently, each pair is associated with **two residuals**: an odd-indexed term and an even-indexed term , referred to as f_{even} and f_{odd} in the code . In order to handle this logic efficiently, the MATLAB implementation employs vectorization by extracting the vectors x_i the first $n-1$ components (stored as x_i) and the components from the second to the n-th (stored as x_{next}). This allows for the simultaneous computation of all $n-1$ odd an $n-1$ even residuals (stored respectively as f_{odd} and f_{even}), which are squared element-wise, summed, and scaled by 0.5 to obtain the objective function.

As before, the gradient is computed by observing that for $F(x) = \frac{1}{2}\|f\|_2^2$ where $f : \mathbb{R}^n \to \mathbb{R}^m$ can be obtained by $g = J^T f$. The Jacobian matrix is built as a sparse matrix exploiting the chained structure of the problem, where each row r corresponds to a residual f_r and contains exactly two non-zero entries at columns $c_1 = \lceil r/2 \rceil$ (for x_i) and $c_2 = c_1 + 1$ (for x_{i+1}). The partial derivatives with respect to the first variable x_i are uniformly defined as $\frac{\partial f_k}{\partial x_i} = 1$ and stored in the vector v_1, while the derivatives with respect to x_{i+1} are stored in v_2 and alternate between two analytical expressions based on the residual index: for odd rows, the code evaluates $d_{odd} = 10x_{i+1} - 3x_{i+1}^2 - 2$, whereas for even rows, it computes $d_{even} = 3x_{i+1}^2 + 2x_{i+1} - 14$. In order to construct the residual vector f, the same strategy used for the vector $v2$ is applied, with the difference that f_{odd} and f_{even} are used instead of d_{odd} and d_{even}.
Also for the Hessian we use the same formula used for Broyden function, i.e. $\nabla^2 F(x) = J_f(x)^T J_f(x) + \sum_{k=1}^m f_k(x)\nabla^2 f_k(x)$. For the first term calculation, **the Jacobian** has already been assembled previously, so it is only necessary to execute the matrix product with its transpose. Concerning the curvature term , since the component x_i appears in each residual f_k only as linear term, all second-order partial derivatives involving x_i and any other component that does not appear at all in the residual formulas are zero; therefore the only non-zero second order derivative of f_k is only with respect to the variable x_{i+1}. Then each Hessian is a sparse matrix with only one non-zero element at the diagonal position $(i+1, i+1)$. The individual Hessian results, where each component is scaled by its corresponding residual, are ultimately summed together to form a purely diagonal matrix. Within this specific mathematical structure, because the variable x_1 never performs the role of a "next" variable (x_{i+1}) in the sequence, the $(1,1)$ entry of the diagonal summation $\sum f_k \nabla^2 f_k$ consistently remains zero. To maintain high performance, the implementation avoids the computationally expensive task of summing m different sparse $n \times n$ matrices; instead, the program directly calculates the diagonal elements of the curvature term and stores them within a single, efficient vector. The **second-order derivatives** for all odd and even blocks are computed simultaneously by exploiting vectorization, utilizing the formulas $d2_{odd} = 10 - 6x_{i+1}$ and $d2_{even} = 6x_{i+1} + 2$ to process the data in a single pass. Once these values are determined, they are multiplied by their respective residuals and accumulated directly into the storage vector starting from the second index, which ensures the correct alignment of the physical variables. This vector is subsequently converted into a sparse diagonal matrix and assembled with the previous terms to obtain the final Hessian of the objective function. Because each variable x_i only interacts with itself and its immediate neighbors x_{i-1} and x_{i+1} within the summation of squared residuals, the matrix satisfies the condition $H_{ij} = 0$ for all $|i-j| > 1$. Consequently, the resulting Hessian is strictly tridiagonal, allowing the optimization algorithm to solve the linear systems involved with minimal memory usage and high numerical speed.

4 Efficient finite differences implementation

4.1 Broyden Tridiagonal

In Broyden Tridiagonal function the Hessian matrix the second-order partial derivatives $\frac{\partial^2 f}{\partial x_i \partial x_j}$ are equal to zero when the index difference $|i-j| > 2$. This happens because the interaction between variables is limited to at most two steps away (for instance, x_{i-1} and x_{i+1} both influence f_i), resulting in a sparse matrix where non-zero entries are placed strictly to the main diagonal and the two adjacent diagonals on each side.

4.1.1 Approximation of the Hessian starting from the exact gradient - approximation mode 1

The approximation of the Hessian for the Broyden Tridiagonal function exploits the specific sparsity pattern of the problem. Since the gradient component g_k depends on the variables spanning from

x_{k-2} to x_{k+2}, the resulting Hessian matrix is pentadiagonal, containing a main diagonal, two super-diagonals, and two sub-diagonals. To approximate this structure without computing the full dense matrix, the implementation employs a **graph coloring strategy** that defines five distinct groups of indices. The logic behind selecting five groups lies in the bandwidth of the matrix: because perturbing a variable x_j affects the gradient components in the range $[j-2, j+2]$, a step of 5 ensures that simultaneous perturbations do not result in mixing changes in the output vector. Consequently, the code iterates through these five groups (e.g., indices $group : 5 : n$), creating a sparse perturbation vector h_{vec} for each iteration. This allows the finite difference approximation $\frac{g(x+h)-g(x)}{h}$ to be computed for $n/5$ columns in parallel. From the resulting difference vector y, the algorithm extracts the specific bands: the element at index j represents the main diagonal contribution (H_{diag0}), while the elements at $j+1$ and $j+2$ correspond to the first (H_{diag1}) and second (H_{diag2}) super-diagonals respectively. Finally, these extracted diagonals are assembled into a sparse matrix using offsets ranging from -2 to 2, and the matrix is symmetrized to mitigate any minor numerical asymmetries introduced by the finite difference calculations.

4.1.2 Approximation of the Gradient with first order FD - approximation mode 2

In the file `fd_gradBroyden.m`, the gradient of the Broyden Tridiagonal function is calculated by employing a Forward Finite Difference scheme optimized for the problem's sparse tridiagonal structure. The differentiation step h is initialized using a base value 10^{-k}, where the code selects between a fixed step size or an adaptive one defined as $h = 10^{-k} \cdot \max(|x|, 10^{-8})$ to maintain numerical reliability across different magnitudes of the variable vector x. Mathematically, the forward finite difference formula approximates each gradient component g_i as follows: $g_i \approx \frac{F(x+he_i)-F(x)}{h}$. The code defines a local residual handle `calc_fk` that computes the value of the generic term f_k based on the triplet of variables $(x_{prev}, x_{curr}, x_{next})$, and applies it to augmented vectors to generate the base energy F_{base} for all indices simultaneously. Since each variable x_i contributes to the total objective function through three specific residuals—acting as the "next" variable in the $(i-1)$-th term, the "current" variable in the i-th term, and the "previous" variable in the $(i+1)$-th term—the gradient assembly is performed in three vectorized steps. First, the code perturbs the third argument (x_{next}) to capture the influence of x_i on the previous term f_{i-1} (stored in `diff_prev`); second, it perturbs the second argument (x_{curr}) to capture the influence on the current term f_i (stored in `diff_curr`); and third, it perturbs the first argument (x_{prev}) to capture the influence on the subsequent term f_{i+1} (stored in `diff_next`). The final gradient vector g_{fd} is the result of accumulating these three localized incremental ratios, appropriately shifted to align the indices.

4.1.3 Approximation of the Hessian with second order FD - approximation mode 2

The computation of the approximated Hessian is performed by the MATLAB function `fd_HessBroyden.m`, and in this case, the implementation takes advantage of the pentadiagonal sparsity structure of the Hessian matrix. To prevent the denominator h^2 from collapsing into numerical insignificance and maintain numerical precision for these second-order approximations, the code—after applying the same operations for the step size selection used in the gradient approximation—adjusts the step by taking its square root. In order to achieve an efficient implementation based on vectorization, the code defines a local function handle `calc_F` that computes the half-squared residual for a generic triplet of variables (x_{prev}, x, x_{next}). For the diagonal elements $H_{ii} = \frac{\partial^2 F}{\partial x_i^2}$, the implementation employs a **second-order central difference** scheme that requires considering how x_i affects the energy in every term where it appears. Since each component x_i acts as the "next" variable in block $i-1$, the "current" variable in block i, and the "previous" variable in block $i+1$, the code must evaluate the perturbations for all three roles. Consequently, the central differences are computed for three distinct cases: `D_prev` (perturbing the third argument), `D_curr` (perturbing the second argument), and `D_next` (perturbing the first argument). These resulting incremental ratios are then accumulated into the main diagonal vector `diag0` by shifting the indices to align the contributions correctly. The mixed partial derivatives are computed by evaluating cross-perturbations for the off-diagonal bands. For the first super-diagonal ($H_{i,i+1}$), the code sums two contributions: `f_a`, representing the interaction in the i-th term (current-next), and `f_b`, representing the interaction in the $(i+1)$-th term (previous-current). For the second super-diagonal ($H_{i,i+2}$), which arises from the interaction between the "previous" and "next" variables within a single residual, the code computes a single cross-term `f_c`. Finally,

these vectors are assembled into a sparse pentadiagonal matrix using offsets from -2 to 2, and the result is symmetrized to ensure mathematical validity.

4.2 Chained Freudstein Roth

4.2.1 Approximation of the Hessian starting from the exact gradient - approximation mode 1

The approach for the Chained Freudenstein-Roth function follows a similar logic, but it is even more efficient because the Hessian calculation uses the fact that the matrix is sparse and contains only three non-zero bands: the main diagonal, one super-diagonal, and one sub-diagonal. As every pair of components (x_i, x_{i+1}) appears in the function f_k when k is odd will also be present in f_{k+1}, specific function handle have been defined to evaluate the half sum of the squares of these two terms. For instance, the first term of the resulting vector depends on x_1 and x_2, while the subsequent term depends on x_2 and x_3, ensuring that the entire range of variables is covered through these overlapping pairs. Because of this mathematical arrangement, each individual variable influences two consecutive terms in the vector, with the notable exceptions of x_1 and x_n, which only affect the first and the last terms respectively. Consequently, when the program applies the finite difference formula to approximate the derivatives, it must evaluate both the case where the perturbed components are the current variables of the term and the case where they are the subsequent ones. This method ensures that the local interactions between neighboring variables are captured accurately, and by focusing only on these local connections, the code avoids checking the entire matrix and saves significant computational time. Just as in the other cases, the final tridiagonal result is stored as a sparse matrix and is symmetrized to eliminate any small numerical errors caused by the computer's rounding process.

4.2.2 Approximation of the Gradient with first order FD - approximation mode 2

In the file `fd_gradChained.m` the gradient of the Chained Freudenstein-Roth function is calculated by employing a **Forward Finite Difference** scheme optimized for the problem's sparse chained structure. The differentiation step h is initialized using a base value 10^{-k}, where the code selects between a fixed step size or an adaptive one defined as $h = 10^{-k} \cdot \max(|x|, 10^{-8})$ to maintain numerical reliability across different magnitudes of the variable vector x. Mathematically, the forward finite difference formula approximates each gradient component g_i as follows: $g_i \approx \frac{F(x+he_i)-F(x)}{h}$. The code defines a local energy handle `calc_F` that computes the sum of the squared residuals for a single pair of consecutive variables (u, v) and by passing the sliced vectors $u = x_{1:n-1}$ and $v = x_{2:n}$ as parameters to this handle generates a vector F_base containing the energy contribution of all $n-1$ blocks in the chain simultaneously. Since each variable x_k contributes to the total objective function only through two specific blocks, acting as the "current" variable u in the k-th block and as the "next" variable v in the $(k-1)$-th block, the gradient assembly is performed in two vectorized steps. First, the code perturbs the vector u by h to calculate the partial derivatives for indices 1 to $n-1$, and then it perturbs the vector v to capture the partial derivatives for indices 2 to n. The final gradient vector g_{fd} is the result of accumulating these localized incremental ratios.

4.2.3 Approximation of the Hessian with second order FD - approximation mode 2

The computation of the approximated Hessian is performed by the MATLAB function `fd_HessChained.m`, and even in this case the implementation takes advantage of tridiagonal sparsity structure of the Hessian matrix . To prevent the denominator h^2 from collapsing into numerical insignificance and maintain numerical precision for these second-order approximations, the code after applying the same operations for the step size selection with respect to the gradient approximation adjusts it by taking its square root. In order to achive an efficent implementation based on vectorization, even in this case is used the same local energy handle `calc_F`, already introduced for the gradient, that allows the computation of the second derivatives by focusing on local interactions rather than the global function. For the diagonal elements $H_{ii} = \frac{\partial^2 F}{\partial x_i^2}$, the implementation employs a second-order **central difference** scheme that requires considering how x_i affects the energy in every block where it appears. Since each component x_i (for $1 < i < n$) acts the "current" variable u in block i and as the "next" variable v in block $i-1$, the code must evaluate `calc_F` four times

to fully determine the i-th diagonal entry. These evaluations consist of `Fp_curr` and `Fm_curr`, calculated by perturbing u as $F(u+h, v)$ and $F(u-h, v)$, and `Fp_next` and `Fm_next`, calculated by perturbing v as $F(u, v+h)$ and $F(u, v-h)$. The resulting incremental ratios, $D2_{curr}$ and $D2_{next}$, are then accumulated into the main diagonal vector `diag0` . The mixed partial derivatives $H_{i,i+1}$ are computed by evaluating a cross-perturbation `F_cross`, defined as $F(u+h, v+h)$, for each pair of variables. This term is used in the mixed difference formula $\frac{F_{cross}-F(u+h,v)-F(u,v+h)+F(u,v)}{h_u h_v}$, effectively utilizing the previously calculated `Fp_curr` and `Fp_next` values. Finally, the matrix is assembled as a sparse tridiagonal structure, leveraging its symmetry property.

5 Results and comments

Convergence is affected by the tuning of parameters involved in the stopping criterions. Due to that, our choice was to scale `tolgrad` according to the dimensionality of the problem: for the low-dimensional case ($n = 2$), the convergence tolerance is set to 10^{-8}. For larger problem dimensions, this threshold is scaled by a factor of $\sqrt{n}$ (i.e., $10^{-8}\sqrt{n}$) to ensure that the average error per component remains of the same order of magnitude regardless of the problem size, thereby allowing for a consistent comparison across varying dimensions.

5.1 Chained Freudestein-Roth

5.1.1 Exact Derivatives

The availability of analytical derivatives allows both methods to exploit a more accurate curvature information, which is reflected in the strong Rate of Convergence (ROC) values. In low dimensions such as n = 2, the Modified Newton method demonstrates a stable **quadratic convergence** with an average of **7.33 iterations**. As the problem size increases to n = 1000, MN maintains its efficiency at **10.67 iterations**, whereas the Truncated Newton solver begins to struggle, succeeding in only 4 out of 6 points with an average of **17.5 iterations**. Theoretically, the ROC for MN remains close to 2.0 because the direct Cholesky factorization provides an exact descent direction. In contrast, the TN method uses a Conjugate Gradient solver that approximates the Newton step; if the truncation tolerance is reached too early, the ROC can degrade from quadratic to **superlinear**, which explains the higher iteration count in the curved valley of the Chained function.

When moving to high-dimensional scenarios like n = 100,000, we observe a clear trade-off between iteration count and computation time. MN still requires fewer iterations (**9.83**) to reach the minimum, but the computational cost of direct factorization becomes evident. TN requires more iterations (**16.67**) but completes the task in just **1.24s**, which is significantly faster than the **2.45s** required by MN. This confirms that for large-scale non-linear problems, the iterative nature of TN is superior, even if the individual steps are less precise than those of a direct solver.

5.1.2 Approximate derivatives

The introduction of finite difference approximations significantly alters the convergence behavior due to numerical noise; in fact, the number of success is about **14/36** for both methods and the iteration count for MN jumps to **38.77** for MN, representing a significant increase compared to the exact case. This occurs because the "noisy" Hessian prevents the algorithm from achieving a true quadratic ROC, forcing it into a slower superlinear regime. In high dimensions with k = 12, MN fails completely because the cumulative effect of cancellation errors makes the Hessian correction (τI) ineffective, preventing the method from identifying a valid descent direction. This stagnation is closely linked to our parameter tuning, specifically the choice of $\rho_{Cmn} = 0.5$ and $\rho_{Ctn} = 0.5$. These moderate backtracking factors are designed to navigate the curved valley of the Freudenstein-Roth function by taking cautious steps. However, when the gradient and Hessian are inconsistent due to noise, even a robust backtracking strategy cannot prevent the solver from hitting the jmax = 80 limit or the btmax = 50 safeguard, eventually leading to failure in the most restrictive dimensions.

5.2 Broyden Tridiagonal

5.2.1 Exact Derivatives

The Broyden Tridiagonal problem is generally well-conditioned, and both methods benefit from this structure. For n = 1000, the ROC values for both solvers are remarkably stable around 2.0, indicating consistent quadratic convergence. MN is slightly more efficient in terms of iterations (**8.83** vs **13.5**), yet TN is nearly 7 times faster in execution time (**0.0059s** vs **0.0392s**). This efficiency gap is maintained at n = 100,000, where TN finishes in **0.84s** while MN requires **3.46s**. Theoretically, this is because the tridiagonal structure allows the CG solver in TN to converge in very few internal iterations, whereas MN must still perform a complete (though sparse) factorization for every external step.

5.2.2 Approximate derivatives

The most interesting observation in the Broyden tests is the impact of our tuning parameters, particularly the very aggressive $\rho_{Bmn} = 0.1$. This value implies that when the Armijo condition (with c1 = 1e-4) is not met, the step size is reduced by 90% immediately. This was necessary because the standard Newton step often overshoots the minimum in this specific problem structure. By using such a small ρ, the MN method quickly finds a "safe" step that guarantees a sufficient decrease, maintaining the ROC near superlinear levels even when far from the solution. Conversely, the TN method uses a more standard $\rho_{Btn} = 0.5$. Because the TN solver computes a direction that is naturally "damped" by the Truncated Conjugate Gradient process, the resulting steps are less prone to the extreme overshooting seen in MN. This balance allowed TN to maintain a high success rate of **36**/**36** across all approximation levels (k = 4, 8, 12) in the high-dimensional Broyden tests.

5.3 Theoretical vs. Observed Behavior

Our results confirm the theoretical limit of direct solvers for large-scale optimization. While the Modified Newton method provides the most accurate steps and the best Rate of Convergence per iteration, the Truncated Newton approach is the only viable option for n = 100,000 due to its memory efficiency and iterative speed. The observed stagnation in the Chained function at k = 12 highlights the **"precision trap"**: higher-order finite differences theoretically reduce truncation error, but the resulting cancellation error in high dimensions becomes the dominant bottleneck. Finally, the jmax and btmax safeguards were rarely triggered for the Broyden function, suggesting that the well-conditioned nature of the problem makes it far less sensitive to the specific choice of line search parameters than the more difficult Chained Freudenstein-Roth function.

6 Plots

6.1 Low Dimensional Analysis ($n = 2$)

6.1.1 Tridimensional view

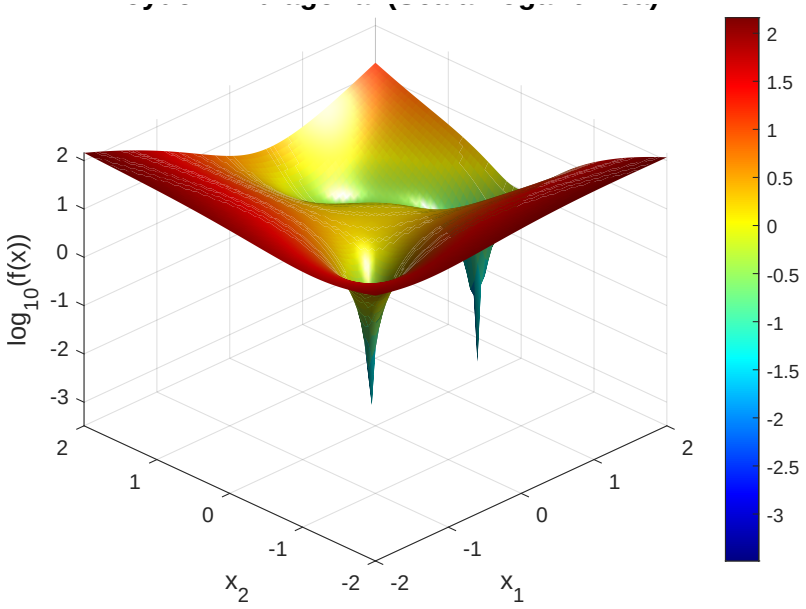


Figure 1: Broyden Tridiagonal function

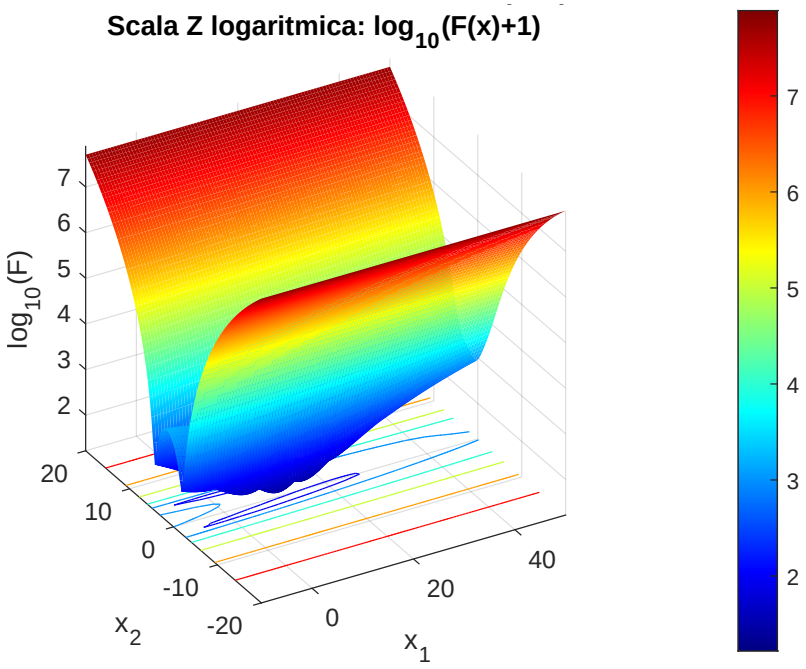
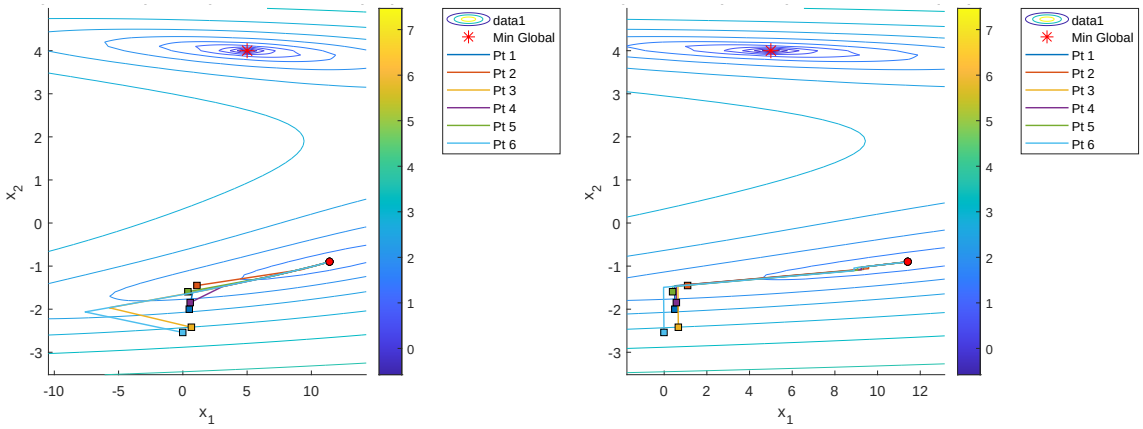


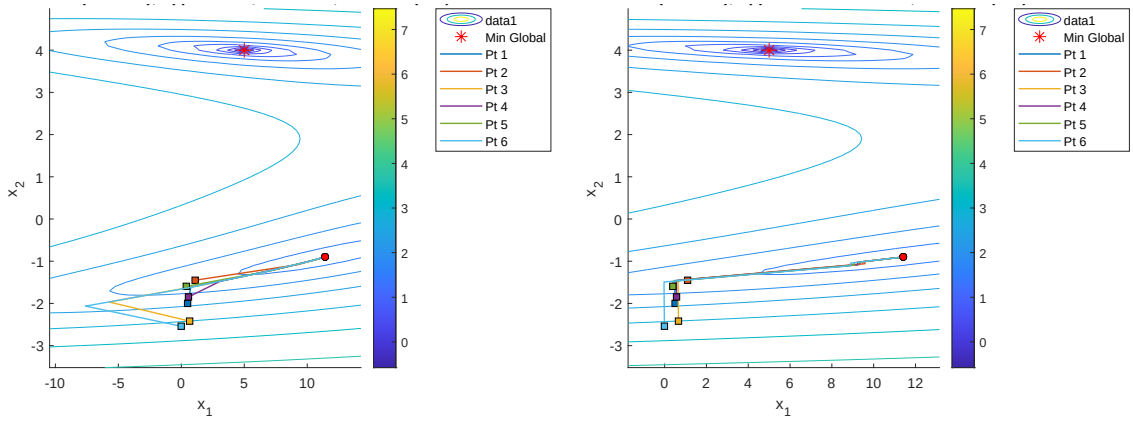
Figure 2: Chained FreudensteinRoth function

6.1.2 Chained Freudenstein-Roth Optimization Trajectories ($n = 2$)



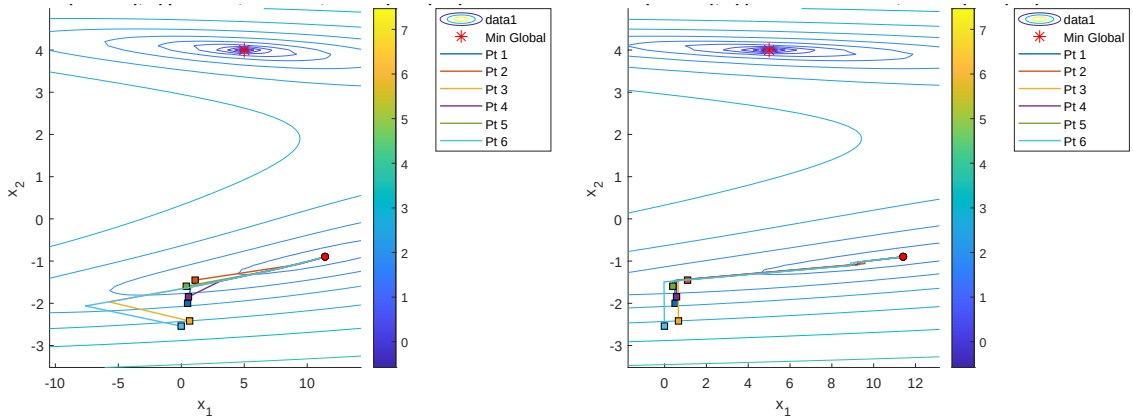
(a) Modified Newton (b) Truncated Newton

Figure 3: Trajectories Chained with exact derivatives



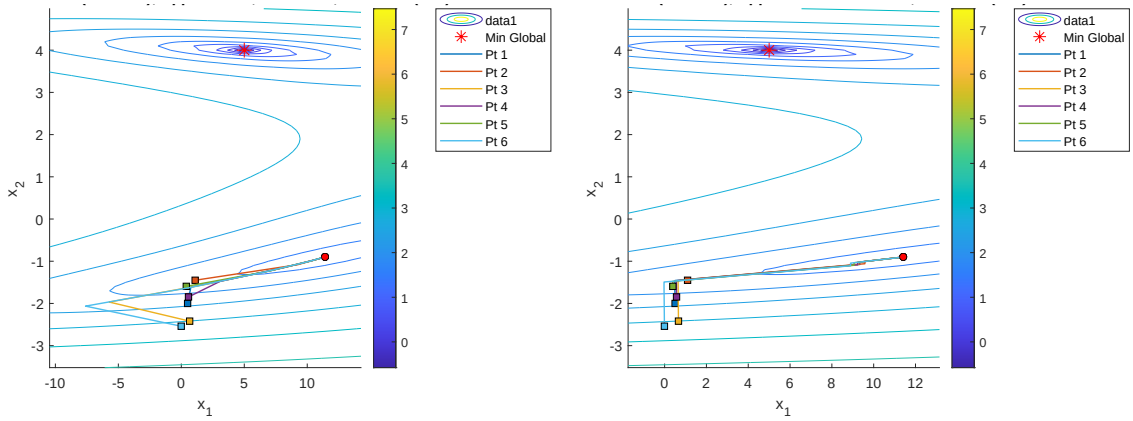
(a) Modified Newton (b) Truncated Newton

Figure 4: Trajectories Chained with approximated Hessian, k = 4, mode = fixed



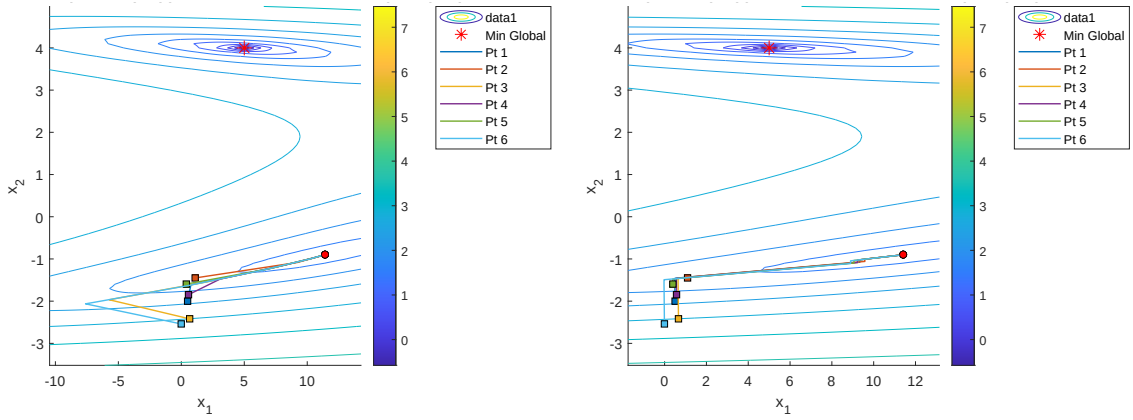
(a) Modified Newton (b) Truncated Newton

Figure 5: Trajectories Chained with approximated Hessian, k = 4, mode = adaptive



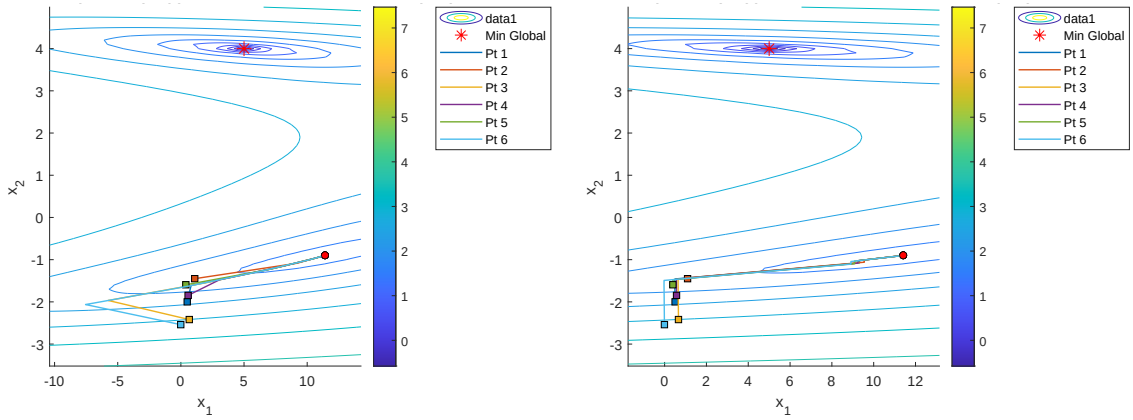
(a) Modified Newton (b) Truncated Newton

Figure 6: Trajectories Chained with approximated Hessian, k = 8, mode = fixed



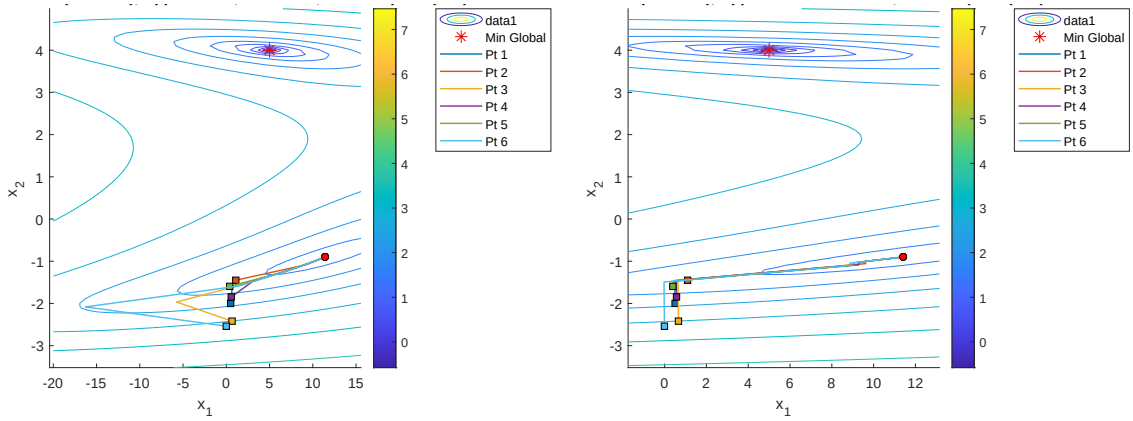
(a) Modified Newton (b) Truncated Newton

Figure 7: Trajectories Chained with approximated Hessian, k = 8, mode = adaptive



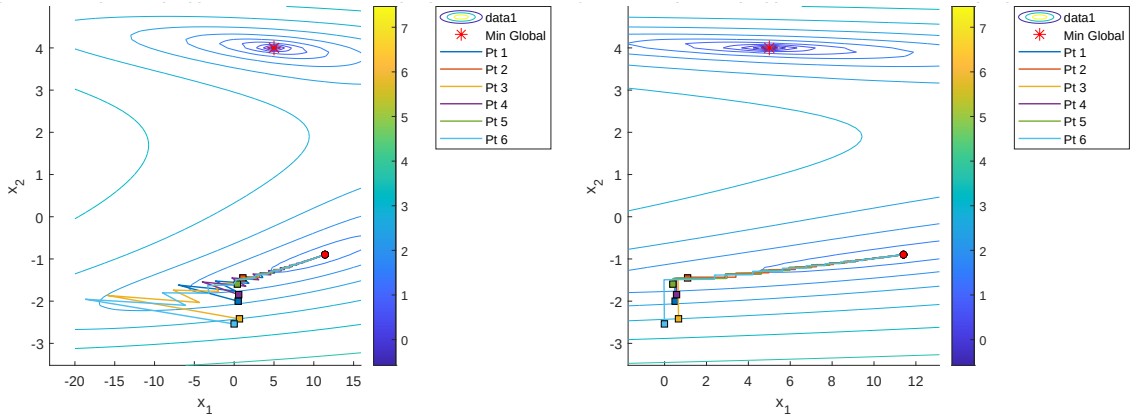
(a) Modified Newton (b) Truncated Newton

Figure 8: Trajectories Chained with approximated Hessian, k = 12, mode = fixed



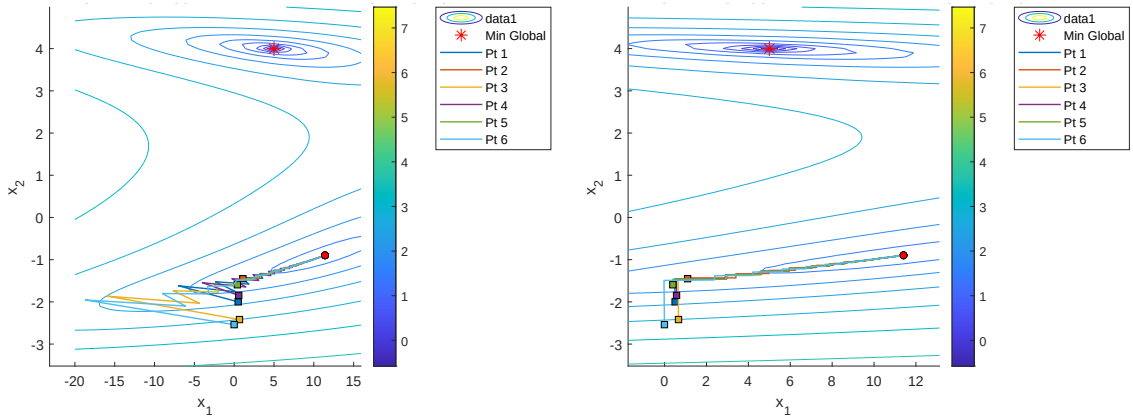
(a) Modified Newton (b) Truncated Newton

Figure 9: Trajectories Chained with approximated Hessian, k = 12, mode = adaptive



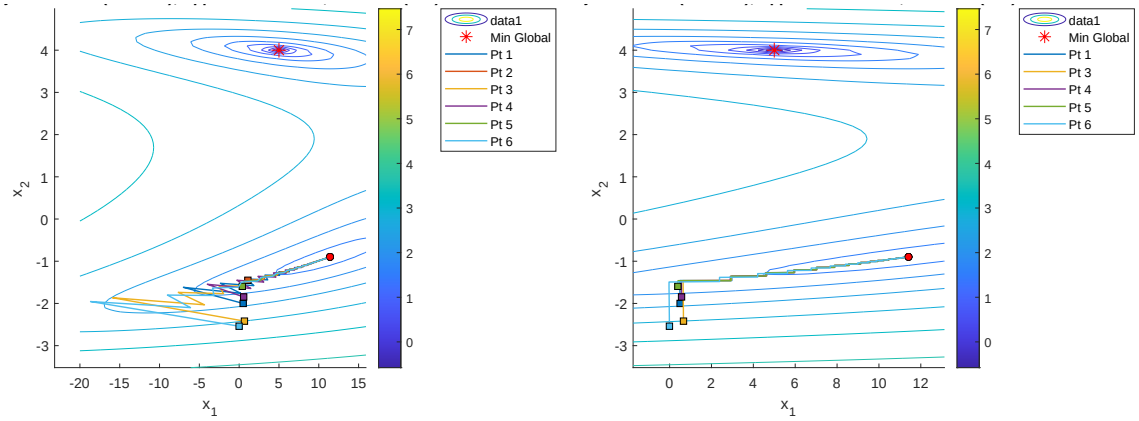
(a) Modified Newton (b) Truncated Newton

Figure 10: Trajectories Chained with all derivatives approximated, k = 4, mode = fixed



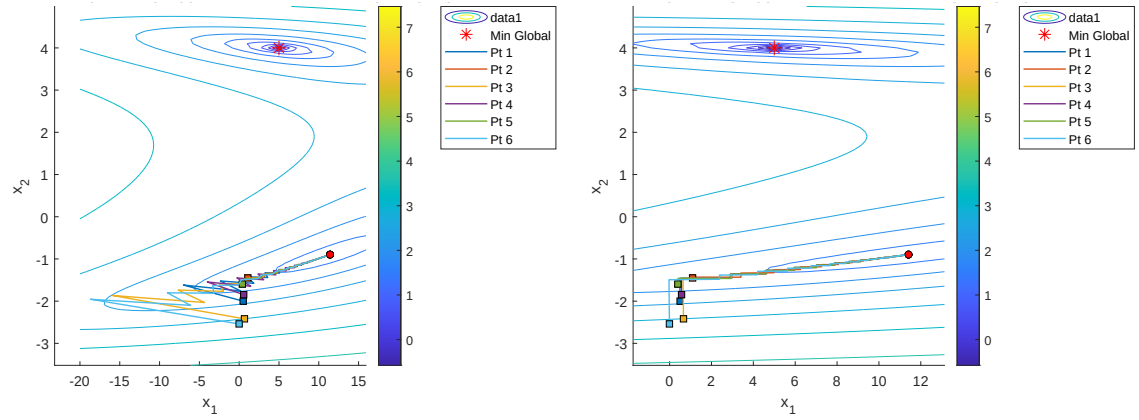
(a) Modified Newton (b) Truncated Newton

Figure 11: Trajectories Chained with all derivatives approximated, k = 4, mode = adaptive



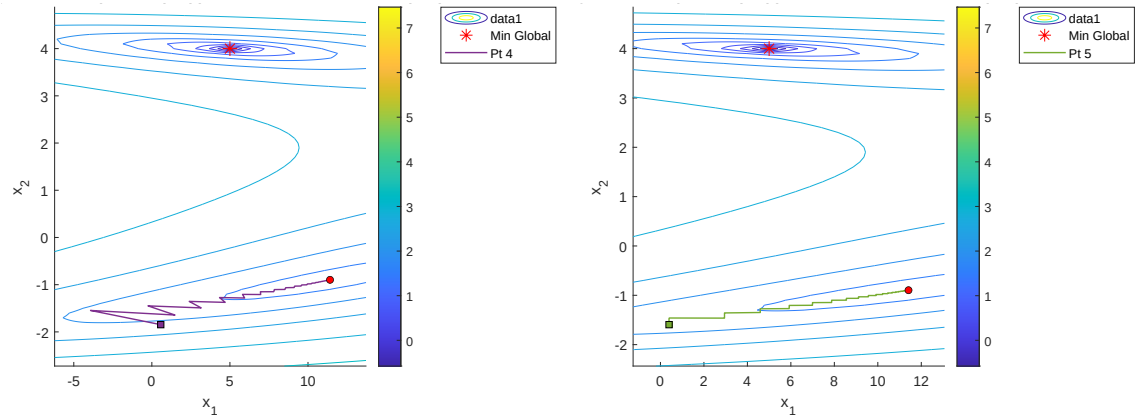
(a) Modified Newton (b) Truncated Newton

Figure 12: Trajectories Chained with all derivatives approximated, k = 8, mode = fixed



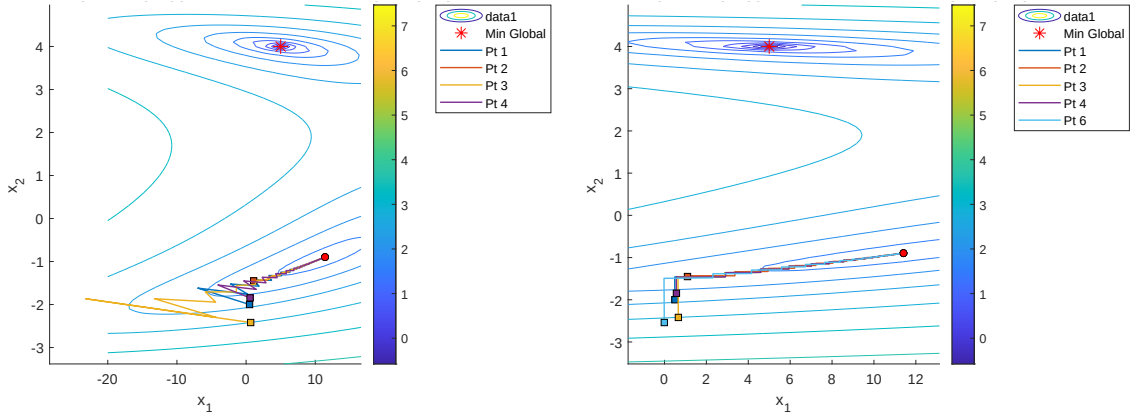
(a) Modified Newton (b) Truncated Newton

Figure 13: Trajectories Chained with all derivatives approximated, k = 8, mode = adaptive



(a) Modified Newton (b) Truncated Newton

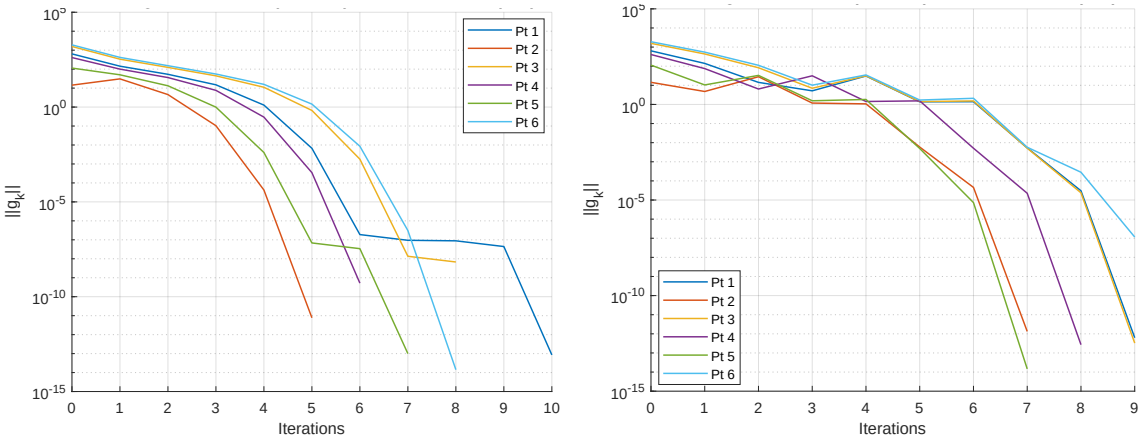
Figure 14: Trajectories Chained with all derivatives approximated, k = 12, mode = fixed



(a) Modified Newton (b) Truncated Newton

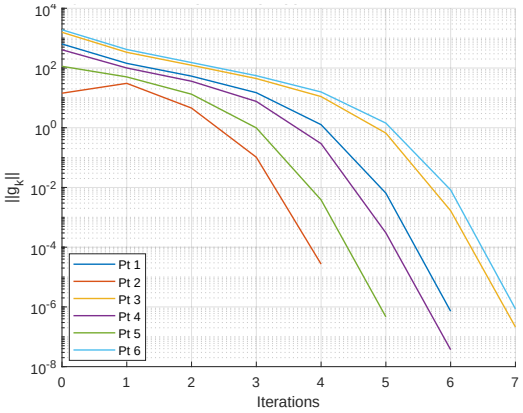
Figure 15: Trajectories Chained with all derivatives approximated, k = 12, mode = adaptive

6.1.3 Chained Freudstein-roth, Rates of convergence ($n = 2$)

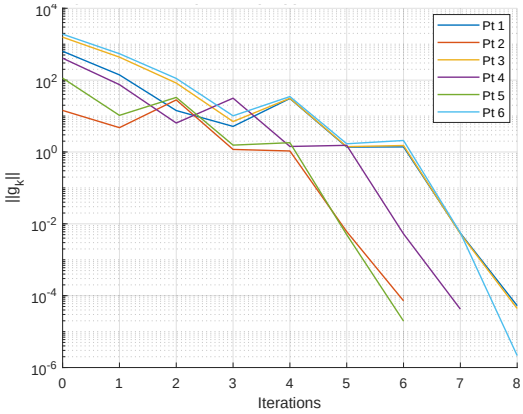


(a) Modified Newton (b) Truncated Newton

Figure 16: Rates of convergence Chained with exact derivatives

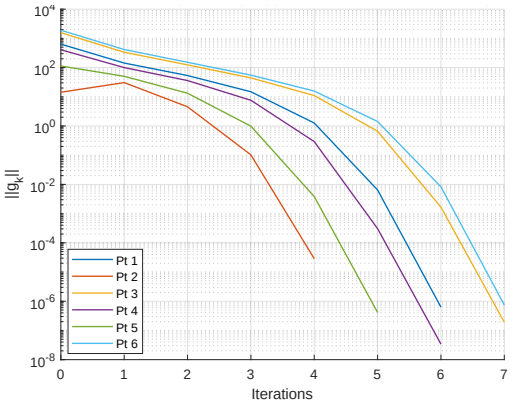


(a) Modified Newton

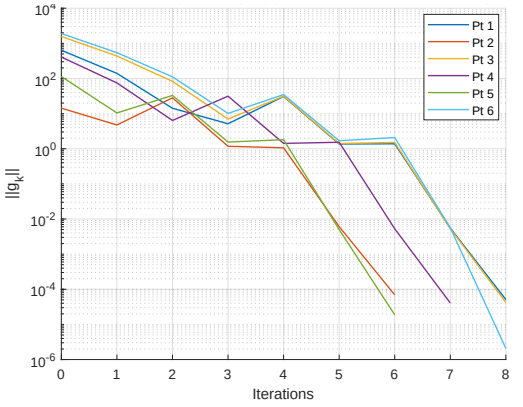


(b) Truncated Newton

Figure 17: Rates of convergence Chained with approximated Hessian, k = 4, mode = fixed

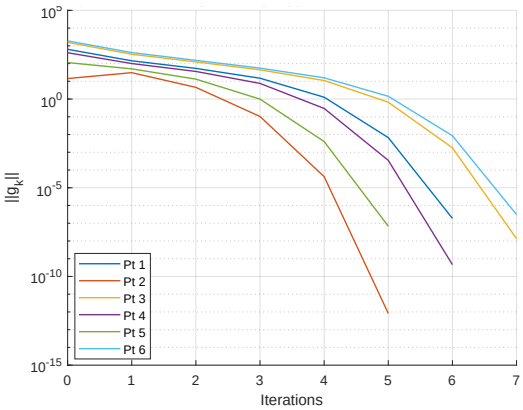


(a) Modified Newton

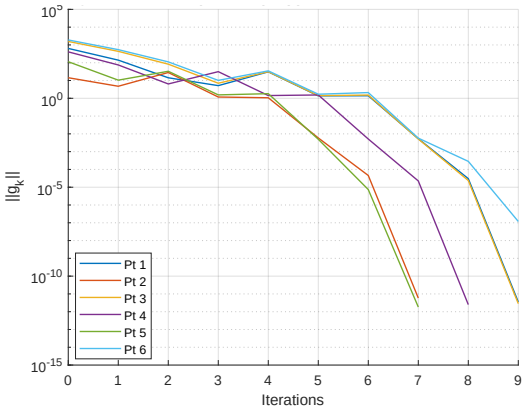


(b) Truncated Newton

Figure 18: Rates of convergence Chained with approximated Hessian, k = 4, mode = adaptive

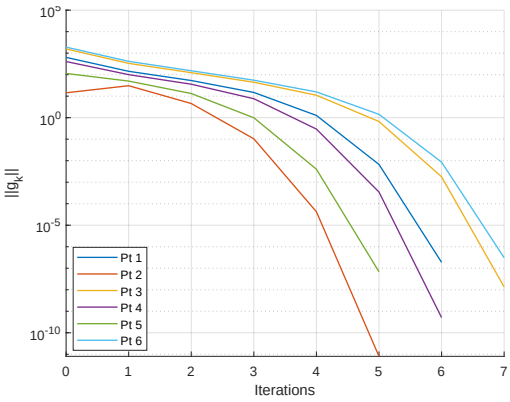


(a) Modified Newton

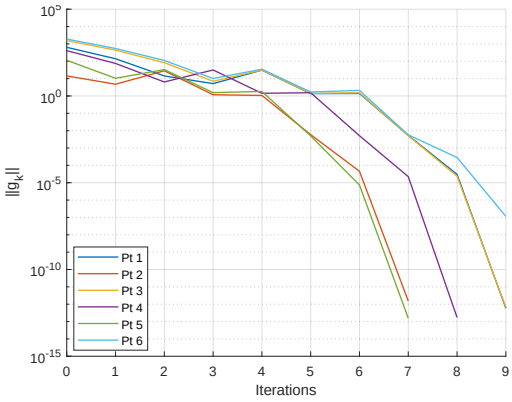


(b) Truncated Newton

Figure 19: Rates of convergence Chained with approximated Hessian, k = 8, mode = fixed

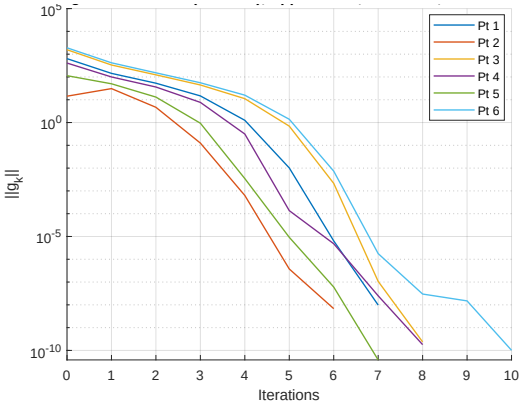


(a) Modified Newton

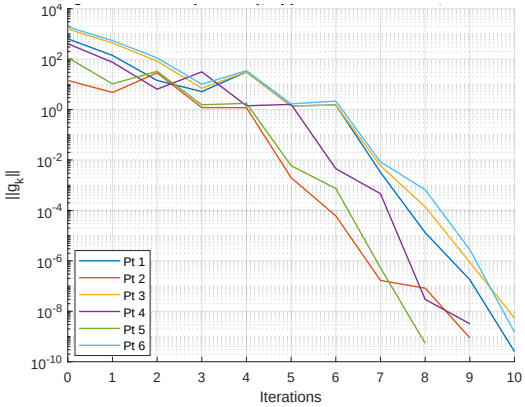


(b) Truncated Newton

Figure 20: Rates of convergence Chained with approximated Hessian, k = 8, mode = adaptive

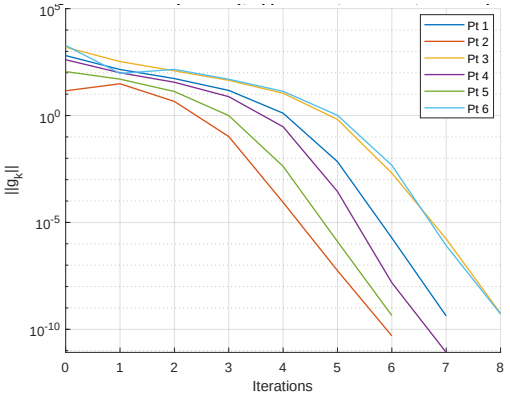


(a) Modified Newton

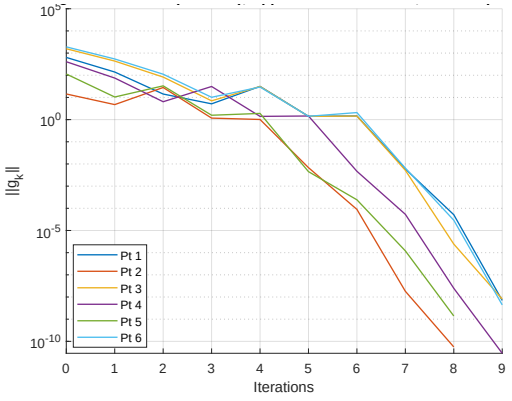


(b) Truncated Newton

Figure 21: Rates of convergence Chained with approximated Hessian, k = 12, mode = fixed

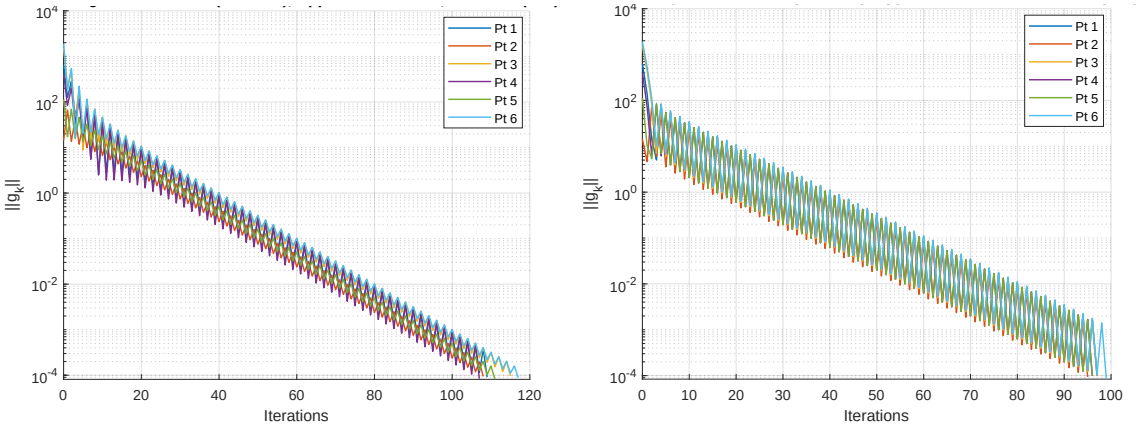


(a) Modified Newton



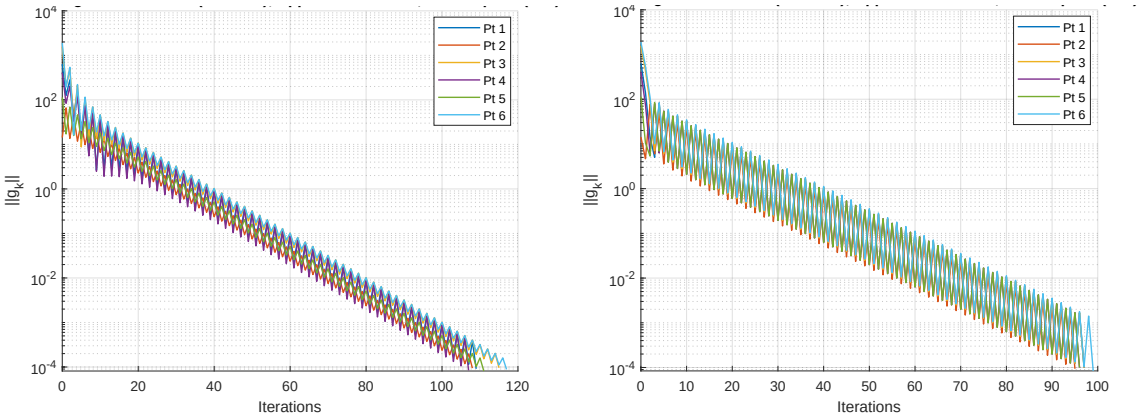
(b) Truncated Newton

Figure 22: Rates of convergence Chained with approximated Hessian, k = 12, mode = adaptive



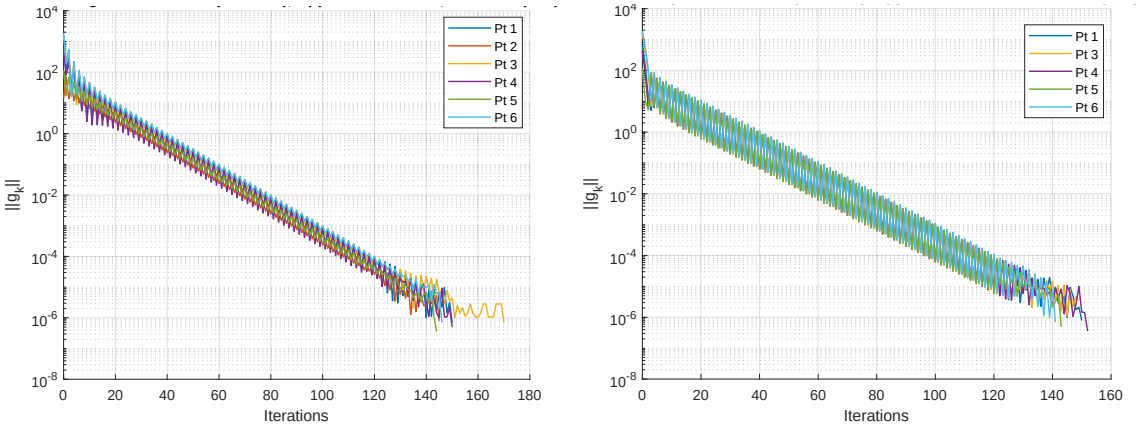
(a) Modified Newton (b) Truncated Newton

Figure 23: Rates of convergence Chained with all derivatives approximated, k = 4, mode = fixed



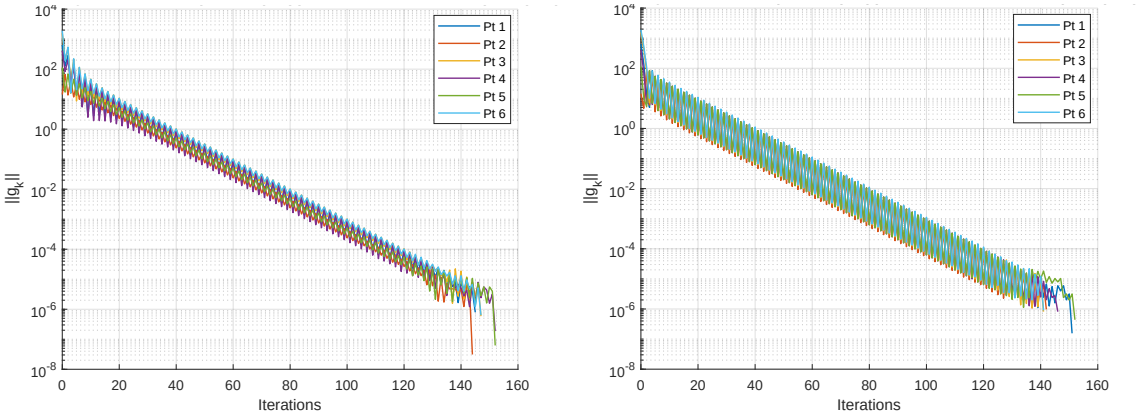
(a) Modified Newton (b) Truncated Newton

Figure 24: Rates of convergence Chained with all derivatives approximated, k = 4, mode = adaptive



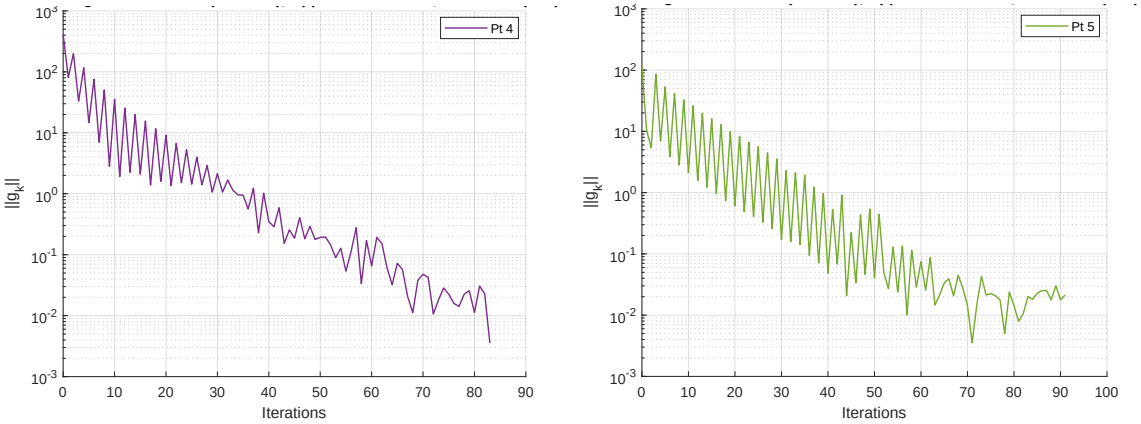
(a) Modified Newton (b) Truncated Newton

Figure 25: Rates of convergence Chained with all derivatives approximated, k = 8, mode = fixed



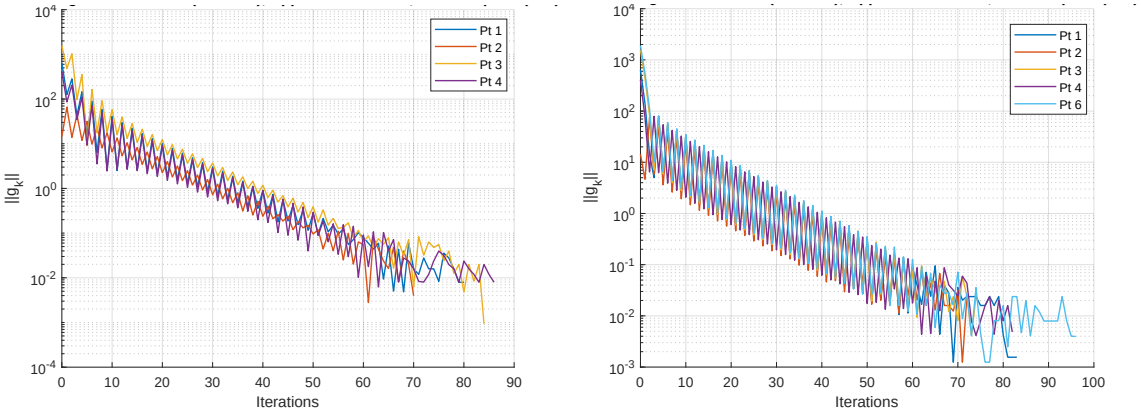
(a) Modified Newton (b) Truncated Newton

Figure 26: Rates of convergence Chained with all derivatives approximated, k = 8, mode = adaptive



(a) Modified Newton (b) Truncated Newton

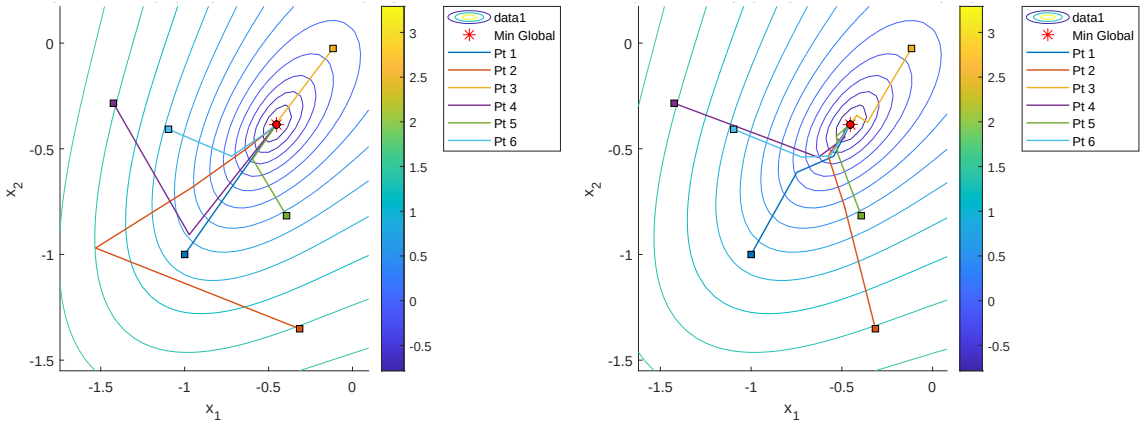
Figure 27: Rates of convergence Chained with all derivatives approximated, k = 12, mode = fixed



(a) Modified Newton (b) Truncated Newton

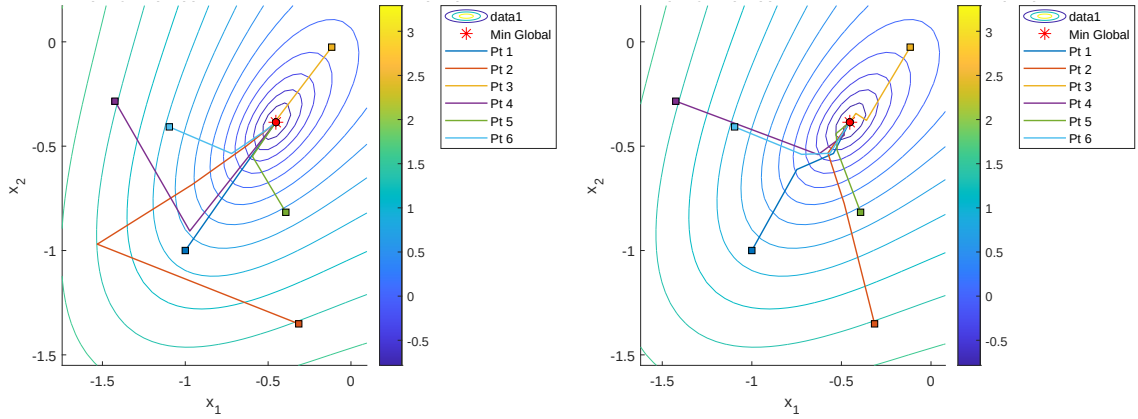
Figure 28: Rates of convergence Chained with all derivatives approximated, k = 12, mode = adaptive

6.1.4 Broyden Tridiagonal, Optimization Trajectories ($n = 2$)



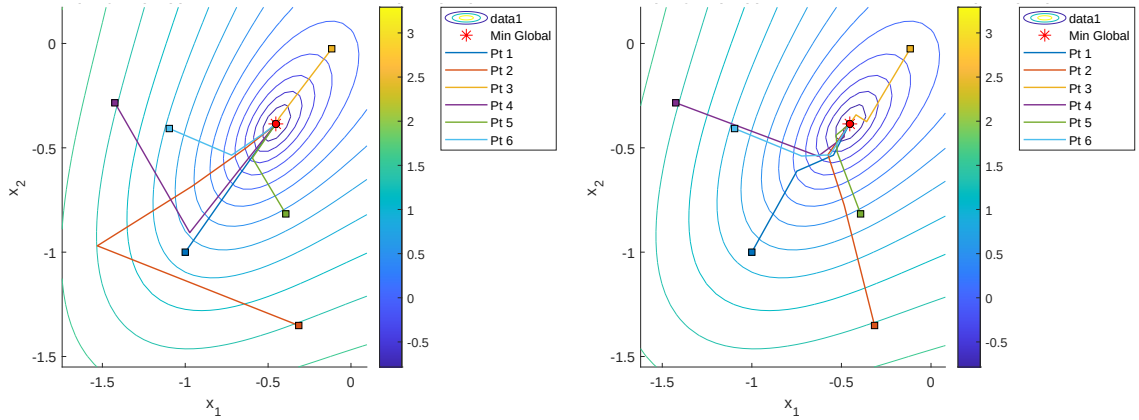
(a) Modified Newton (b) Truncated Newton

Figure 29: Trajectories Broyden with exact derivatives



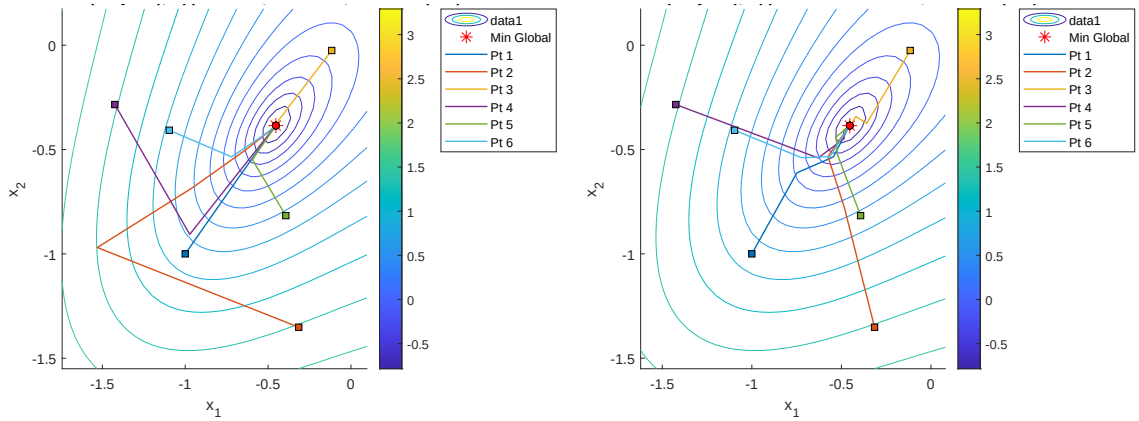
(a) Modified Newton (b) Truncated Newton

Figure 30: Trajectories Broyden with approximated Hessian, k = 4, mode = fixed



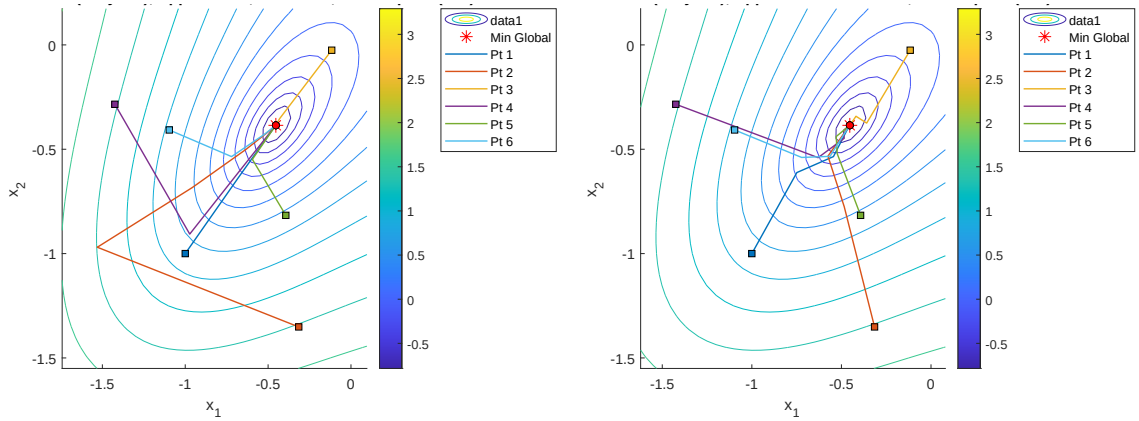
(a) Modified Newton (b) Truncated Newton

Figure 31: Trajectories Broyden with approximated Hessian, k = 4, mode = adaptive



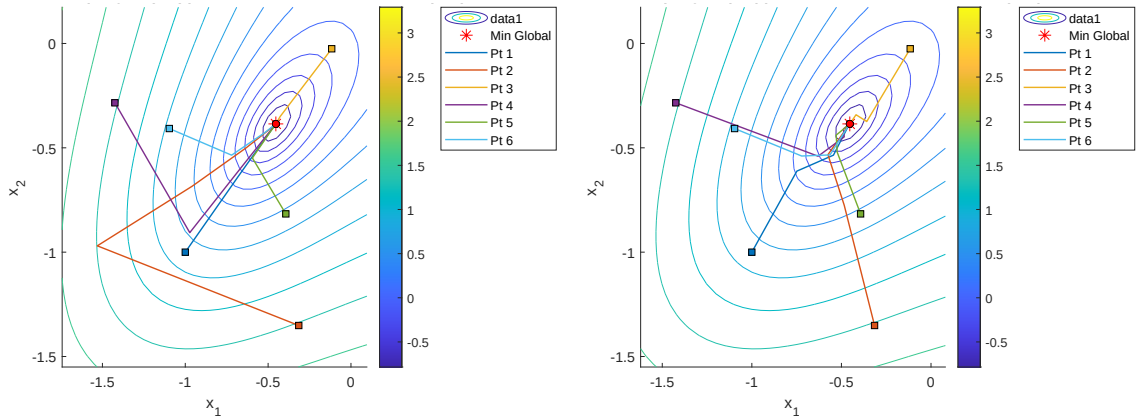
(a) Modified Newton (b) Truncated Newton

Figure 32: Trajectories Broyden with approximated Hessian, k = 8, mode = fixed



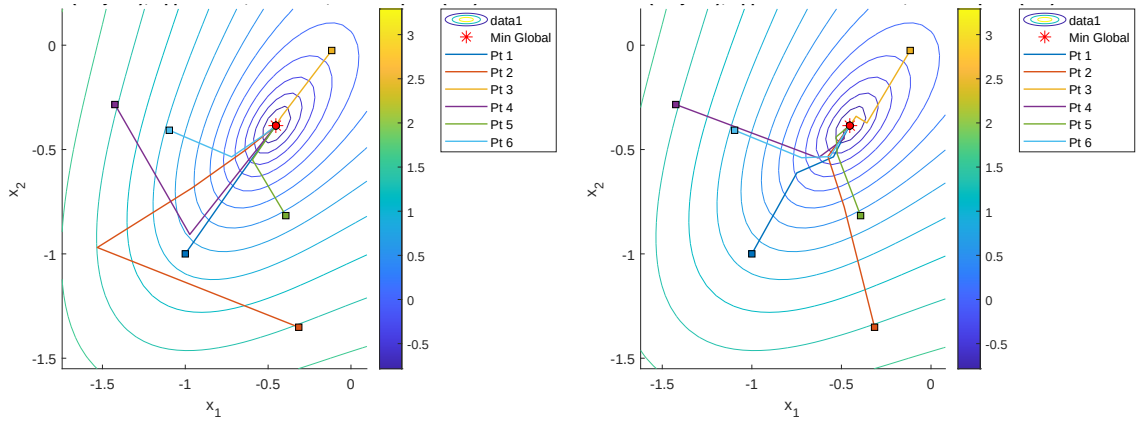
(a) Modified Newton (b) Truncated Newton

Figure 33: Trajectories Broyden with approximated Hessian, k = 8, mode = adaptive



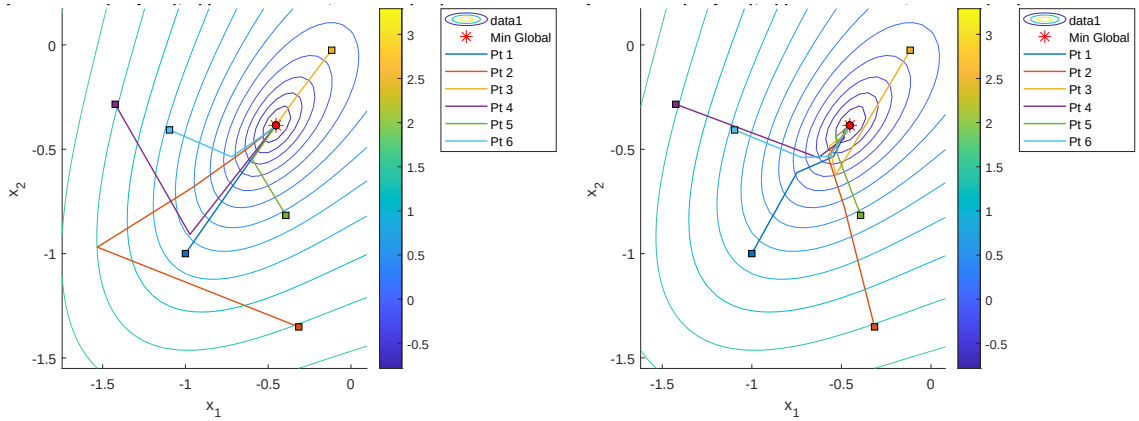
(a) Modified Newton (b) Truncated Newton

Figure 34: Trajectories Broyden with approximated Hessian, k = 12, mode = fixed



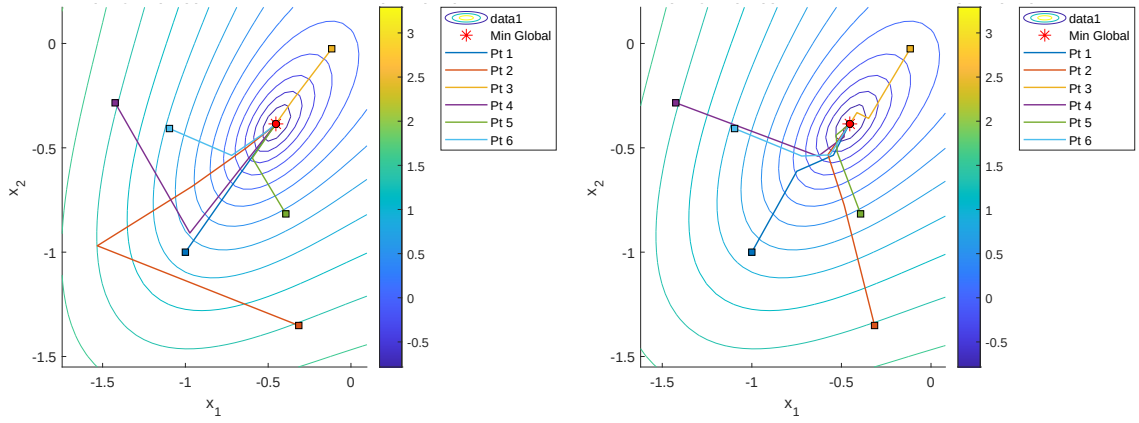
(a) Modified Newton (b) Truncated Newton

Figure 35: Trajectories Broyden with approximated Hessian, k = 12, mode = adaptive



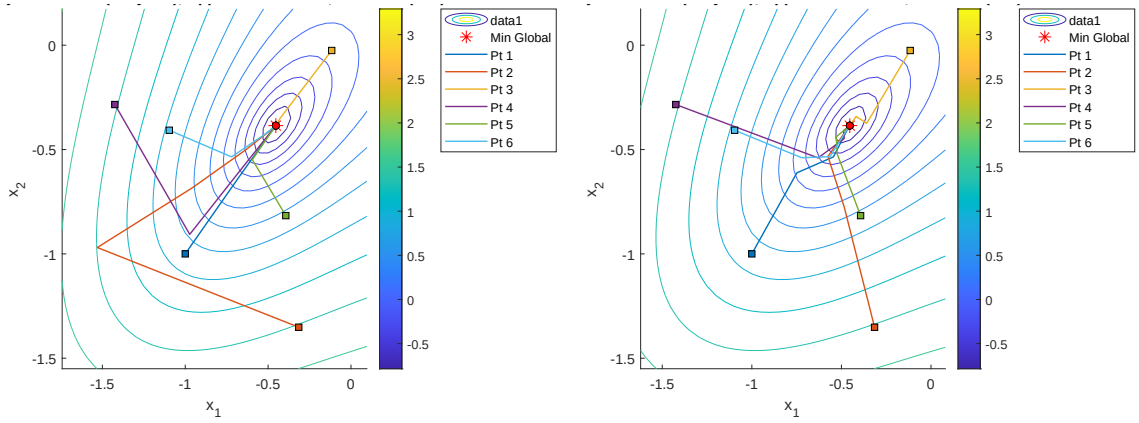
(a) Modified Newton (b) Truncated Newton

Figure 36: Trajectories Broyden with all derivatives approximated, k = 4, mode = fixed



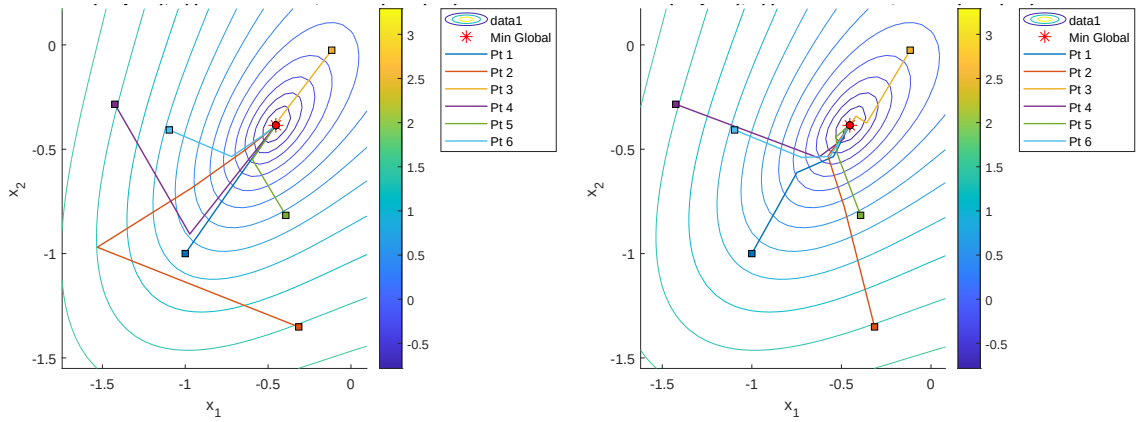
(a) Modified Newton (b) Truncated Newton

Figure 37: Trajectories Broyden with all derivatives approximated, k = 4, mode = adaptive



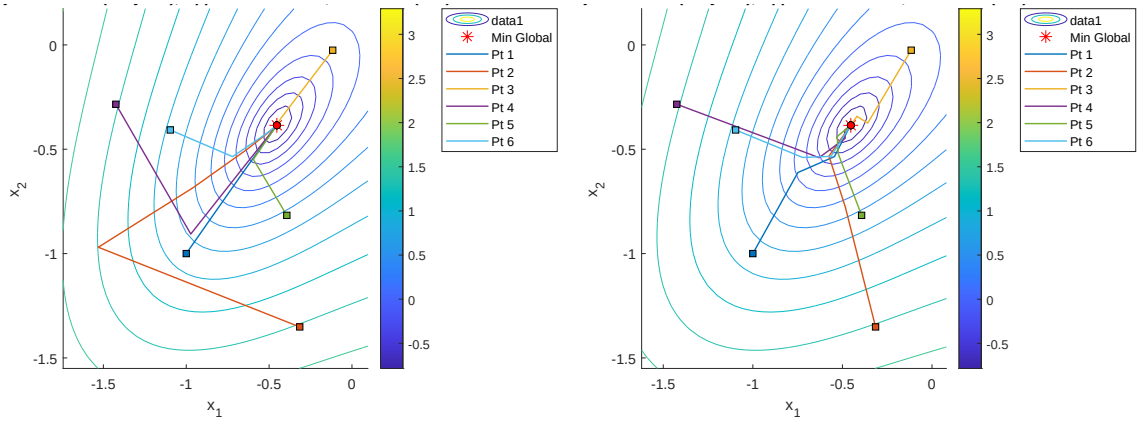
(a) Modified Newton (b) Truncated Newton

Figure 38: Trajectories Broyden with all derivatives approximated, k = 8, mode = fixed



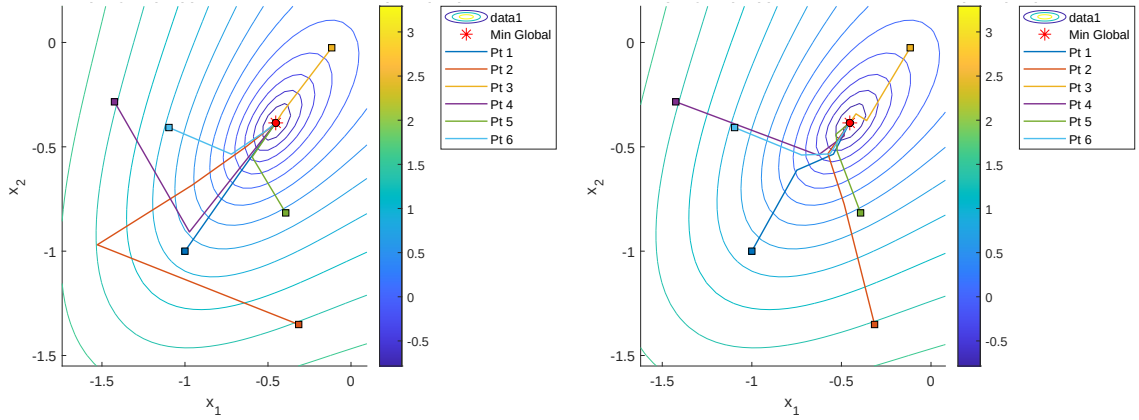
(a) Modified Newton (b) Truncated Newton

Figure 39: Trajectories Broyden with all derivatives approximated, k = 8, mode = adaptive



(a) Modified Newton (b) Truncated Newton

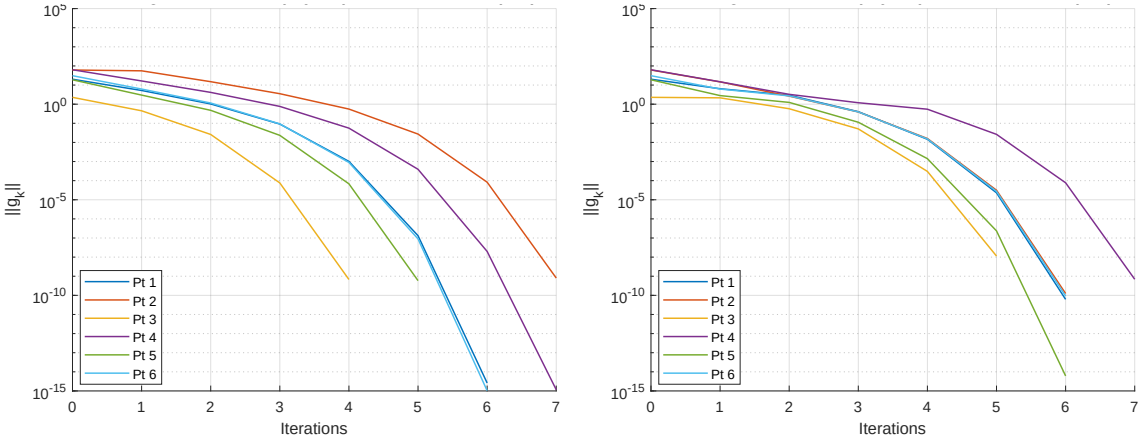
Figure 40: Trajectories Broyden with all derivatives approximated, k = 12, mode = fixed



(a) Modified Newton (b) Truncated Newton

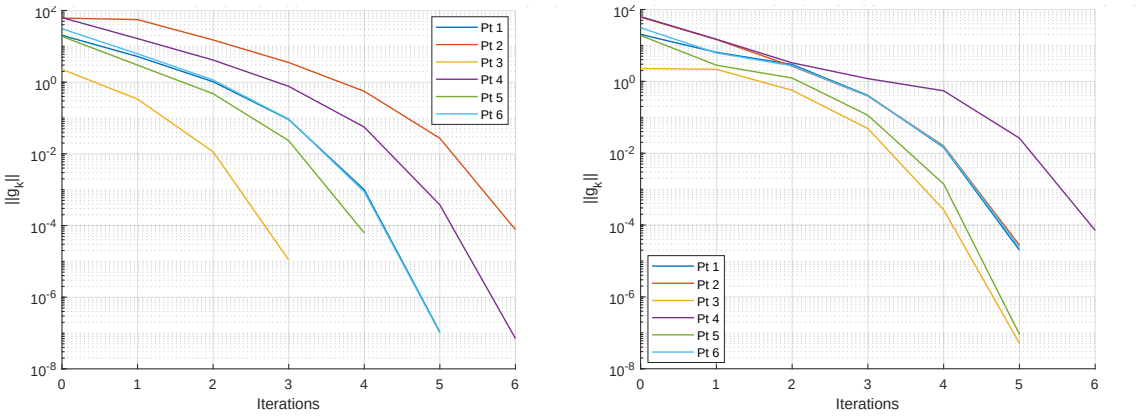
Figure 41: Trajectories Broyden with all derivatives approximated, k = 12, mode = adaptive

6.1.5 Broyden Tridiagonal, Rates of convergence ($n = 2$)



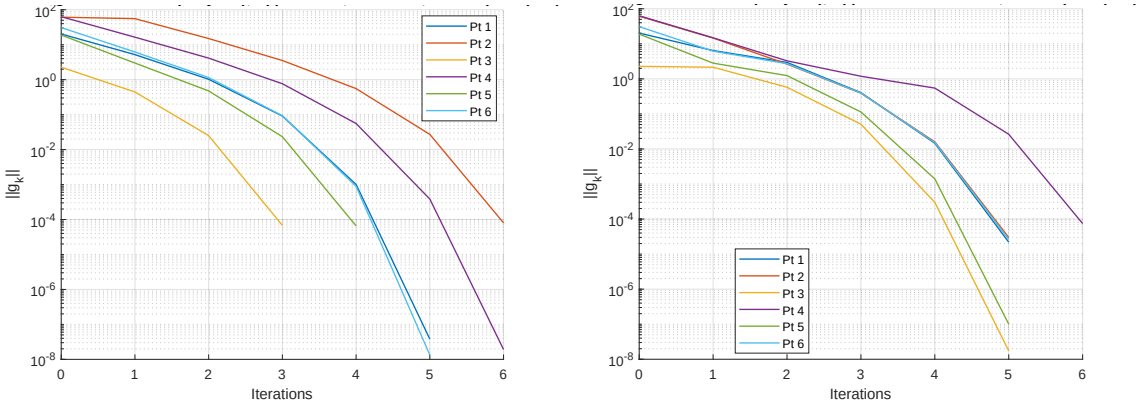
(a) Modified Newton (b) Truncated Newton

Figure 42: Rates of convergence Broyden with Exact derivatives



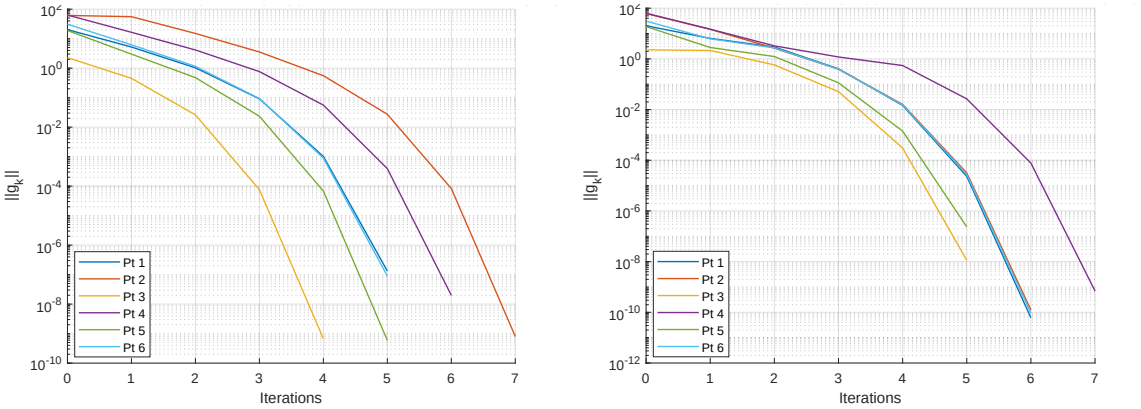
(a) Modified Newton (b) Truncated Newton

Figure 43: Rates of convergence Broyden with approximated Hessian, k = 4, mode = fixed



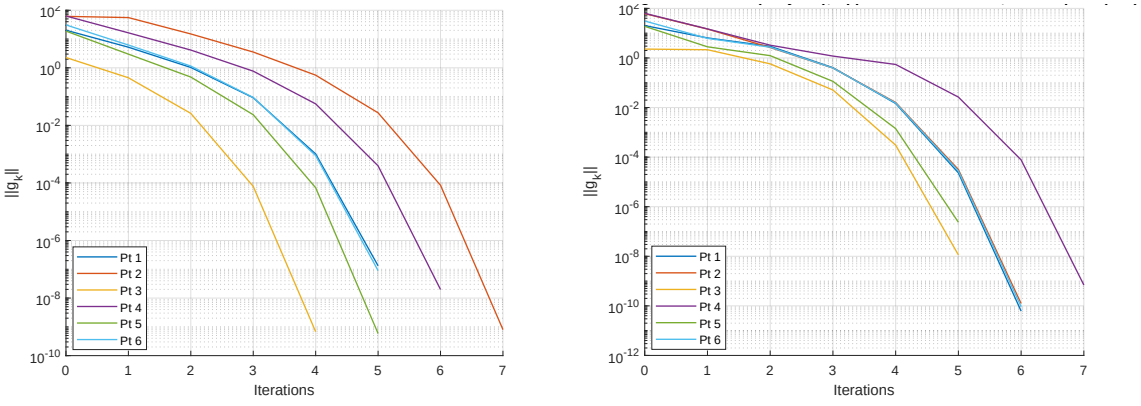
(a) Modified Newton (b) Truncated Newton

Figure 44: Rates of convergence Broyden with approximated Hessian, k = 4, mode = adaptive



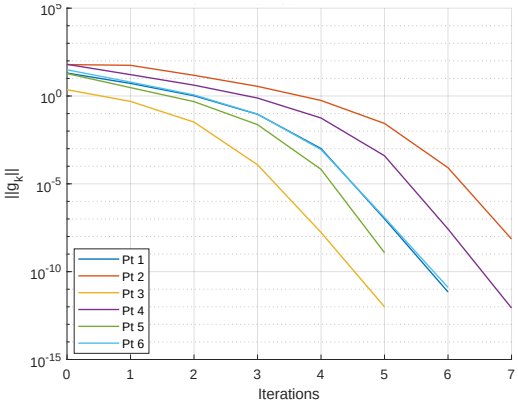
(a) Modified Newton (b) Truncated Newton

Figure 45: Rates of convergence Broyden with approximated Hessian, k = 8, mode = fixed

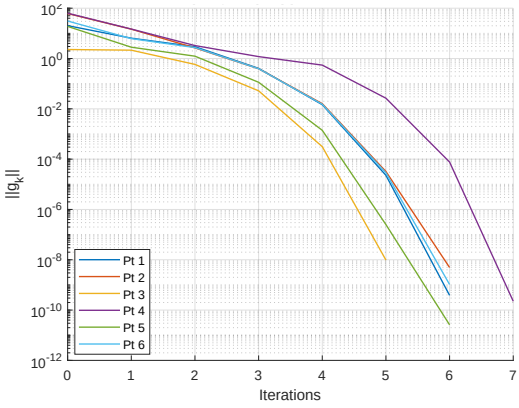


(a) Modified Newton (b) Truncated Newton

Figure 46: Rates of convergence Broyden with approximated Hessian, k = 8, mode = adaptive

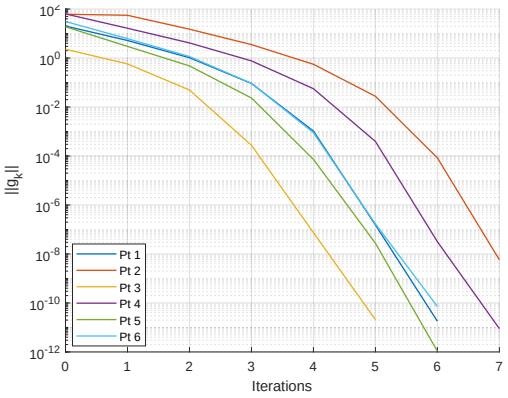


(a) Modified Newton

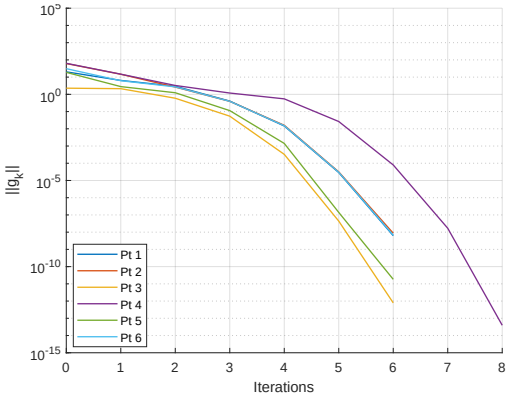


(b) Truncated Newton

Figure 47: Rates of convergence Broyden with approximated Hessian, k = 12, mode = fixed

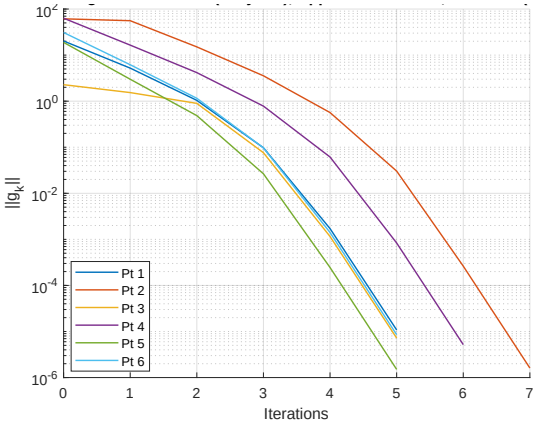


(a) Modified Newton

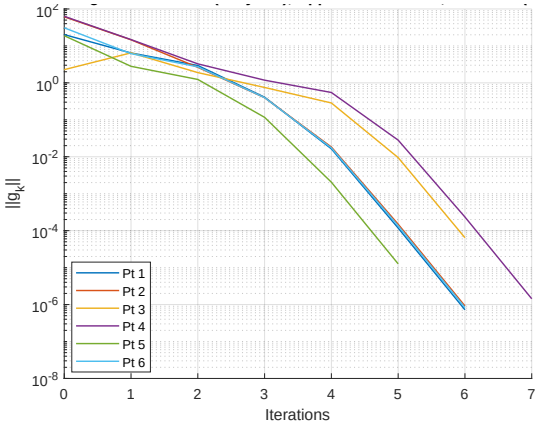


(b) Truncated Newton

Figure 48: Rates of convergence Broyden with approximated Hessian, k = 12, mode = adaptive

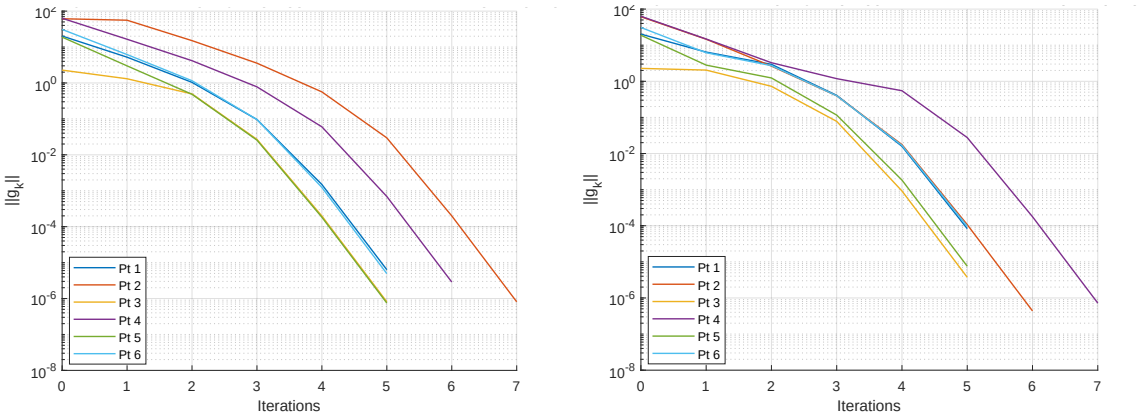


(a) Modified Newton



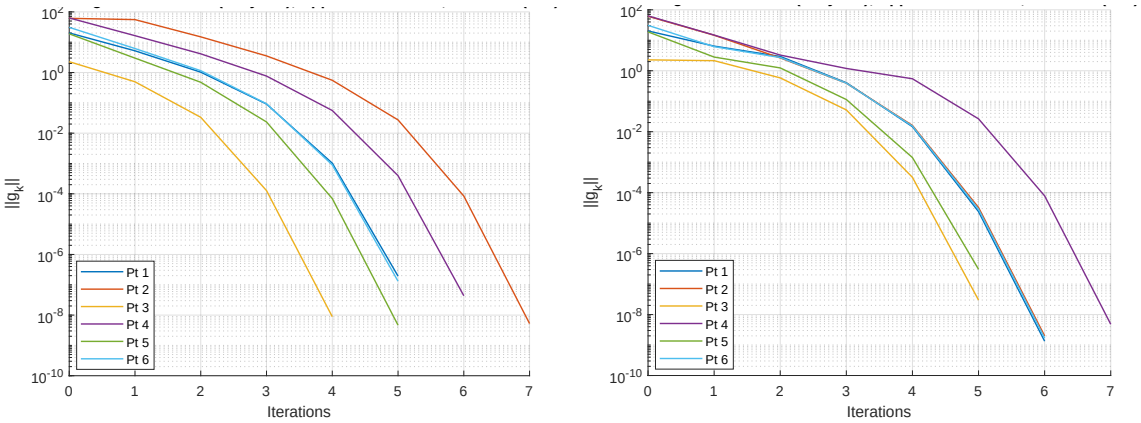
(b) Truncated Newton

Figure 49: Rates of convergence Broyden with all derivatives approximated, k = 4, mode = fixed



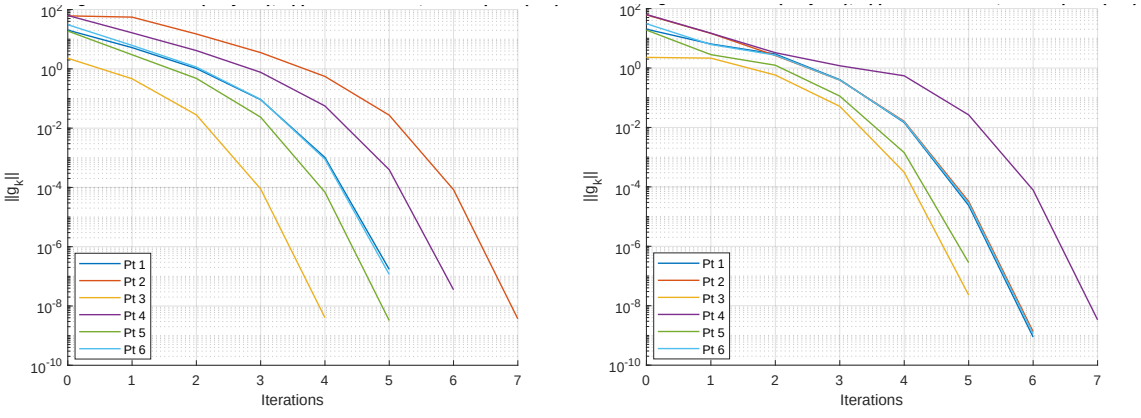
(a) Modified Newton (b) Truncated Newton

Figure 50: Rates of convergence Broyden with all derivatives approximated, k = 4, mode = adaptive



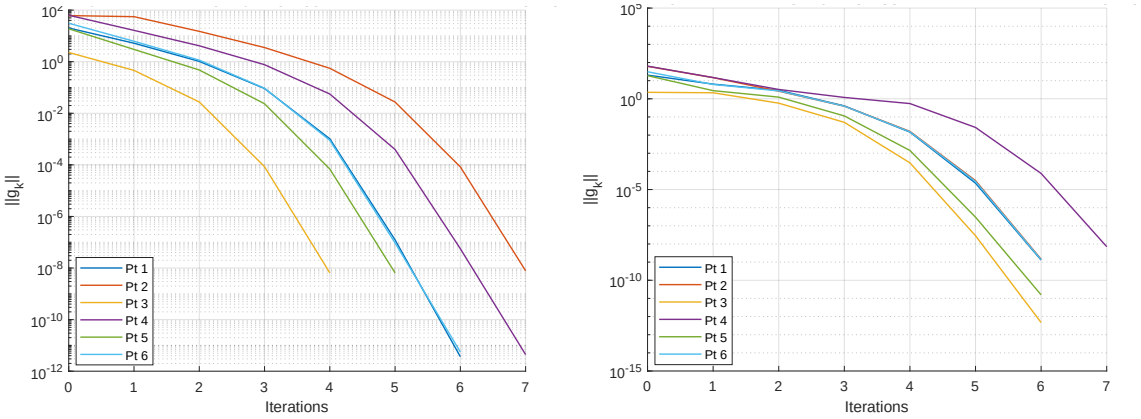
(a) Modified Newton (b) Truncated Newton

Figure 51: Rates of convergence Broyden with all derivatives approximated, k = 8, mode = fixed



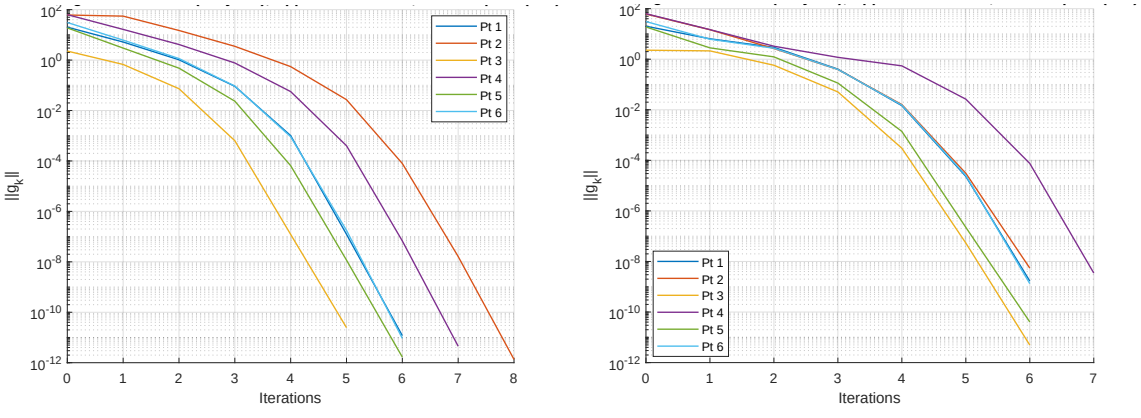
(a) Modified Newton (b) Truncated Newton

Figure 52: Rates of convergence Broyden with all derivatives approximated, k = 8, mode = adaptive



(a) Modified Newton (b) Truncated Newton

Figure 53: Rates of convergence Broyden with all derivatives approximated, k = 12, mode = fixed

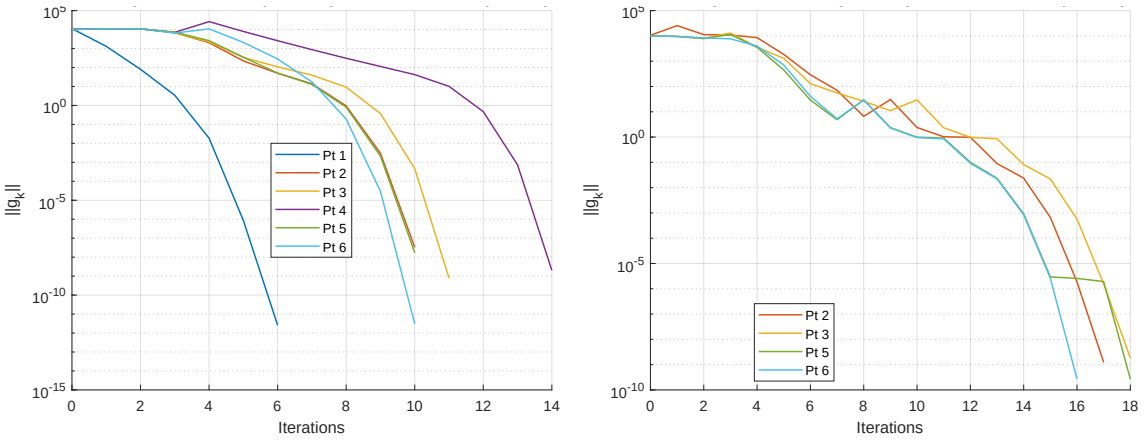


(a) Modified Newton (b) Truncated Newton

Figure 54: Rates of convergence Broyden with all derivatives approximated, k = 12, mode = adaptive

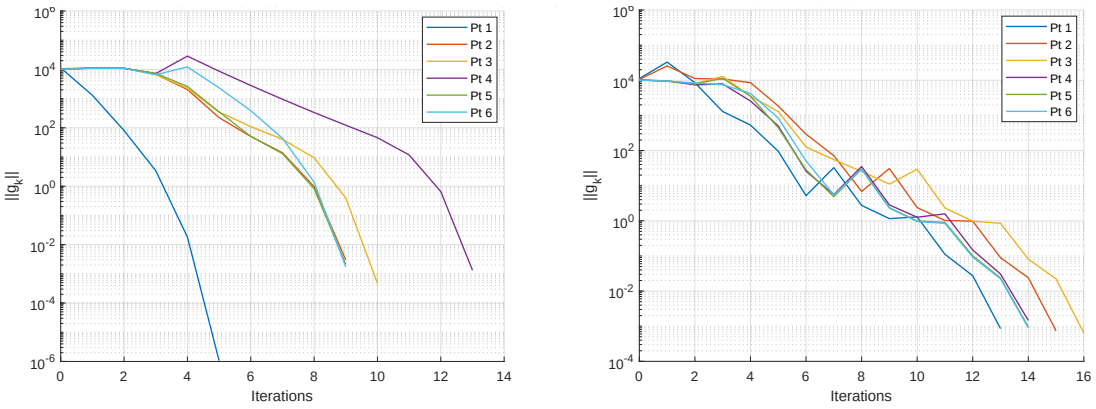
6.2 Medium Scale Analysis ($n = 10^3$)

6.2.1 Chained Freudstain-Roth, Rates of convergence ($n = 10^3$)



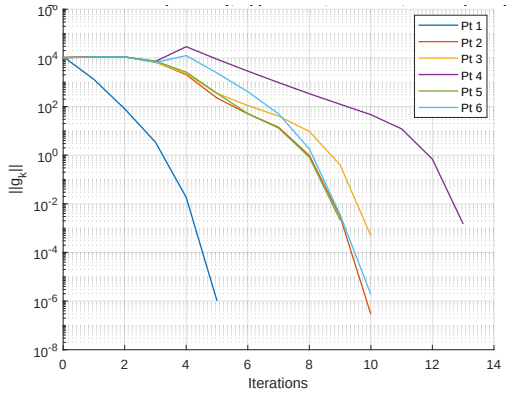
(a) Modified Newton (b) Truncated Newton

Figure 55: Rates of convergence Chained with exact derivatives

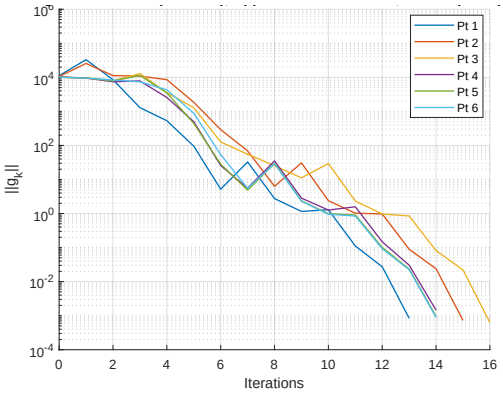


(a) Modified Newton (b) Truncated Newton

Figure 56: Rates of convergence Chained with approximated Hessian, k = 4, mode = fixed

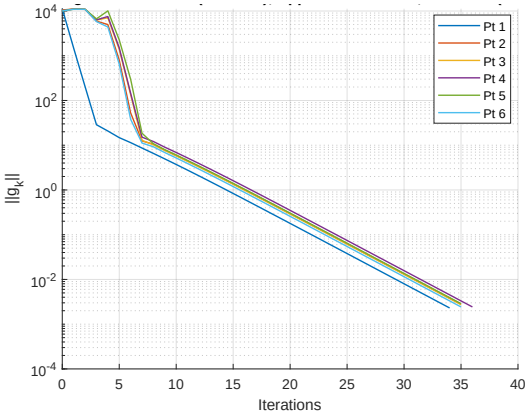


(a) Modified Newton

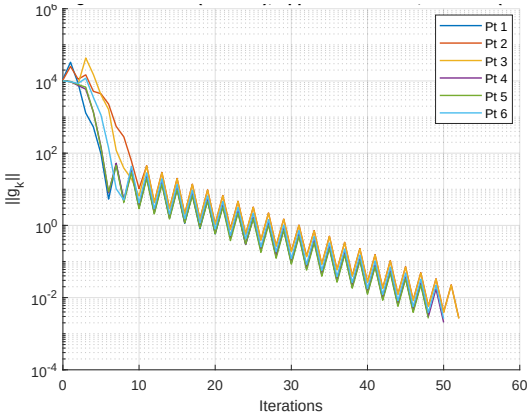


(b) Truncated Newton

Figure 57: Rates of convergence Chained with approximated Hessian, k = 4, mode = adaptive

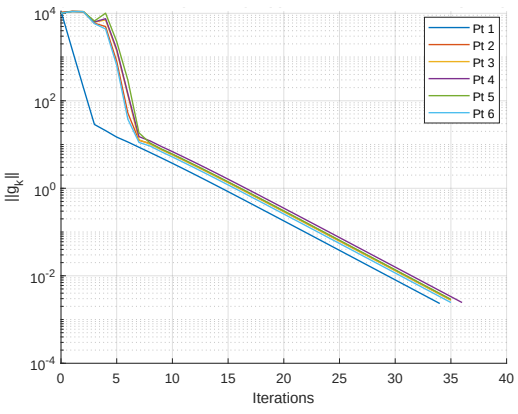


(a) Modified Newton

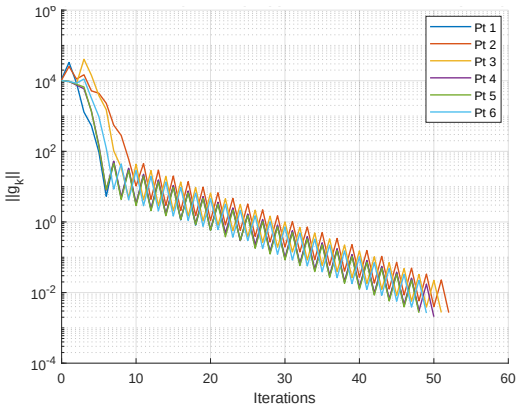


(b) Truncated Newton

Figure 58: Rates of convergence Chained with all derivatives approximated, k = 4, mode = fixed



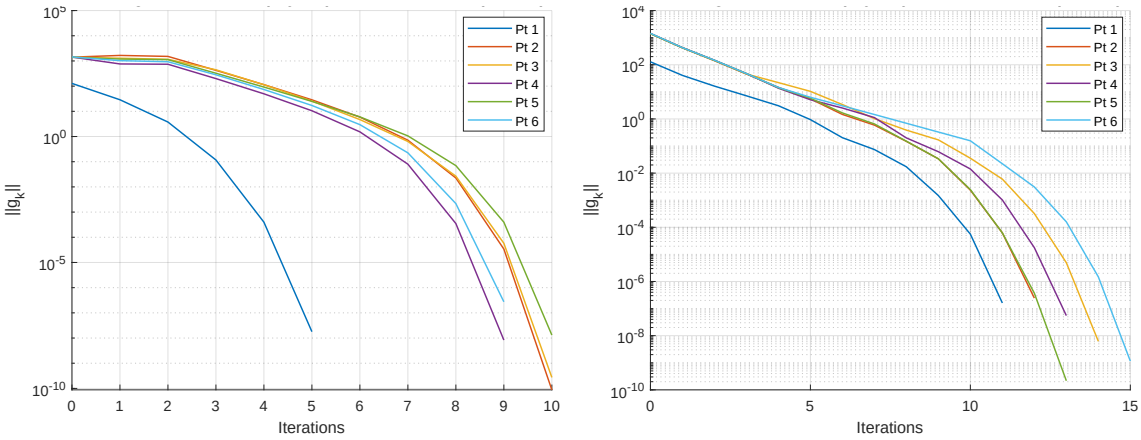
(a) Modified Newton



(b) Truncated Newton

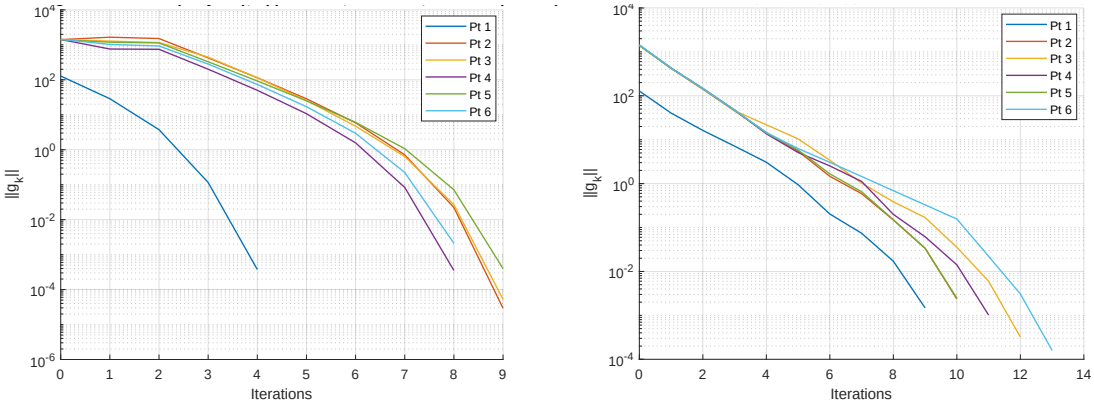
Figure 59: Rates of convergence Chained with all derivatives approximated, k = 4, mode = adaptive

6.2.2 Broyden Tridiagonal, Rates of convergence ($n = 10^3$)



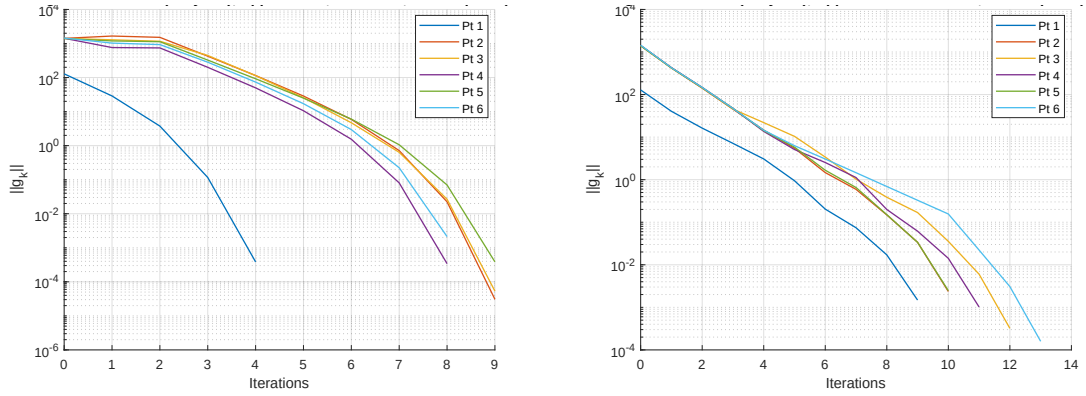
(a) Modified Newton (b) Truncated Newton

Figure 60: Rates of convergence Broyden with exact derivatives



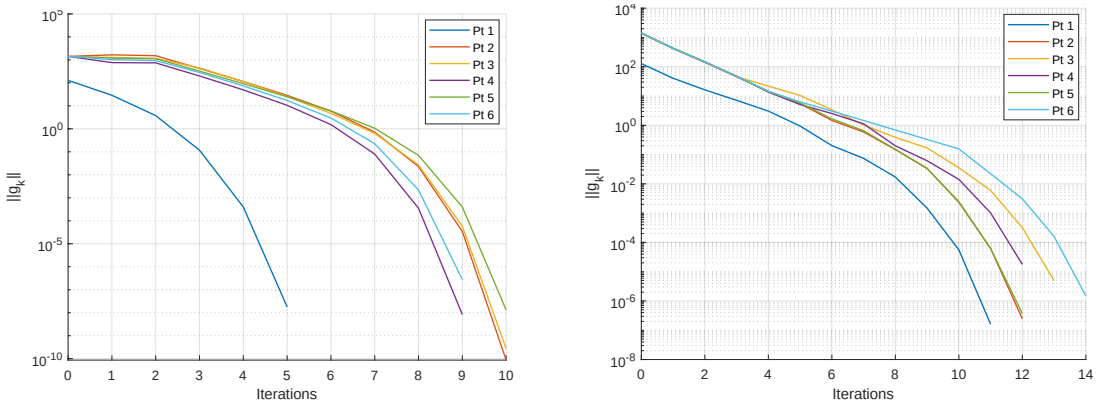
(a) Modified Newton (b) Truncated Newton

Figure 61: Rates of convergence Broyden with approximated Hessian, k = 4, mode = fixed



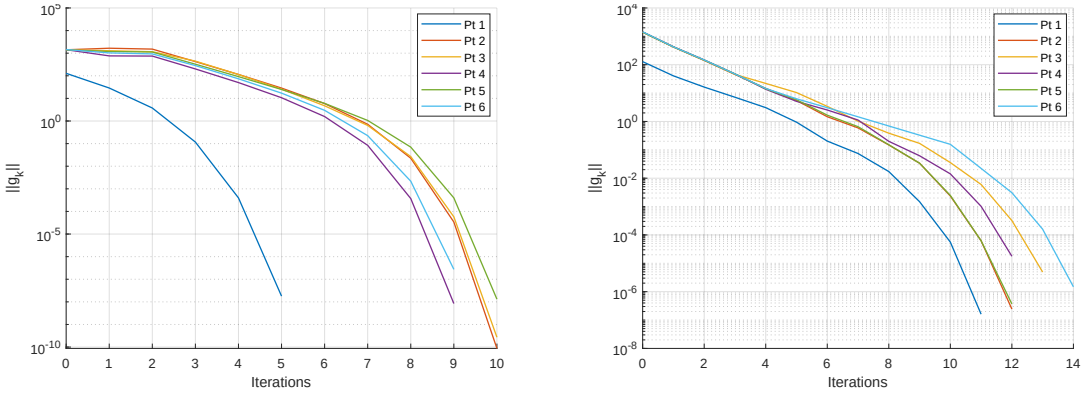
(a) Modified Newton (b) Truncated Newton

Figure 62: Rates of convergence Broyden with approximated Hessian, k = 4, mode = adaptive



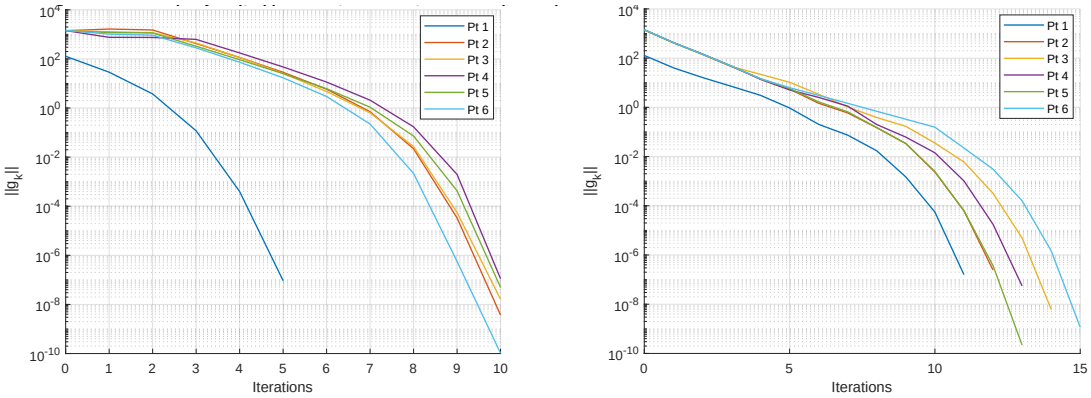
(a) Modified Newton (b) Truncated Newton

Figure 63: Rates of convergence Broyden with approximated Hessian, k = 8, mode = fixed



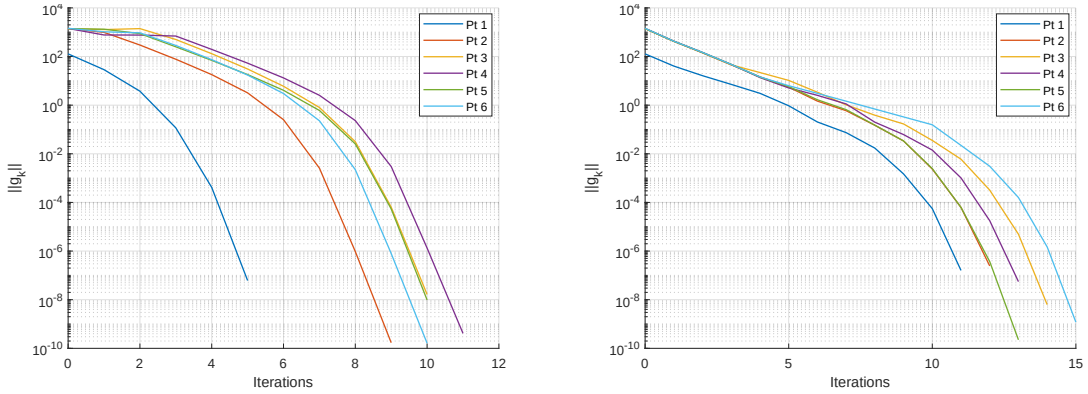
(a) Modified Newton (b) Truncated Newton

Figure 64: Rates of convergence Broyden with approximated Hessian, k = 8, mode = adaptive



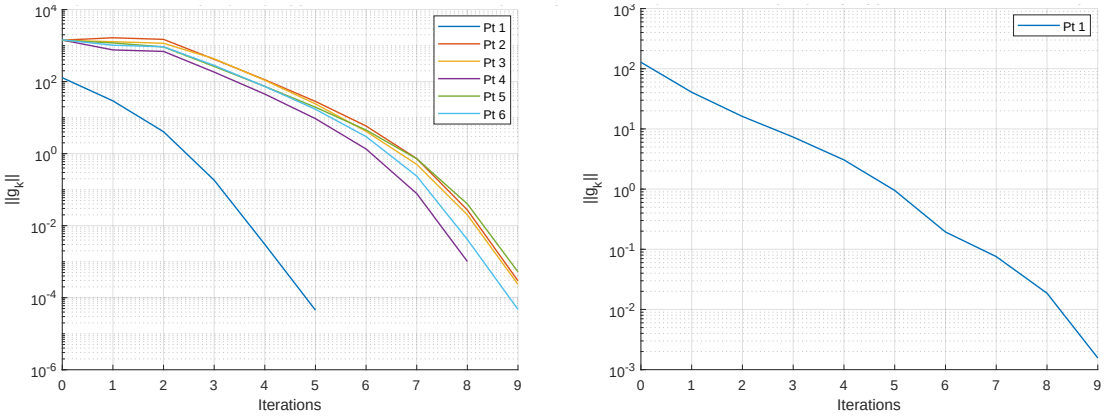
(a) Modified Newton (b) Truncated Newton

Figure 65: Rates of convergence Broyden with approximated Hessian, k = 12, mode = fixed



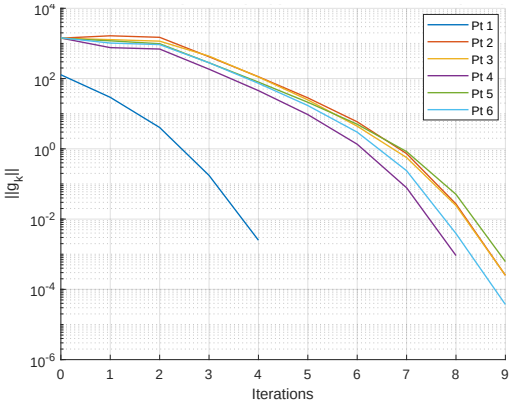
(a) Modified Newton (b) Truncated Newton

Figure 66: Rates of convergence Broyden with approximated Hessian, k = 12, mode = adaptive

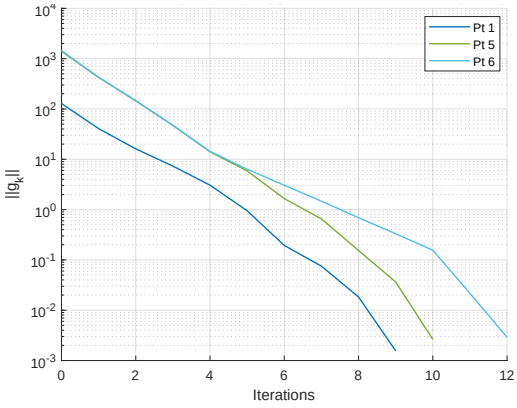


(a) Modified Newton (b) Truncated Newton

Figure 67: Rates of convergence Broyden with all derivatives approximated, k = 4, mode = fixed

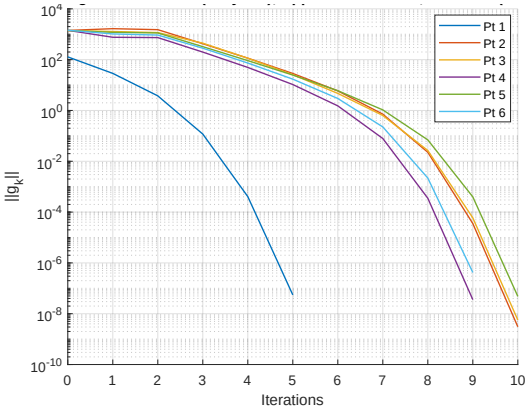


(a) Modified Newton

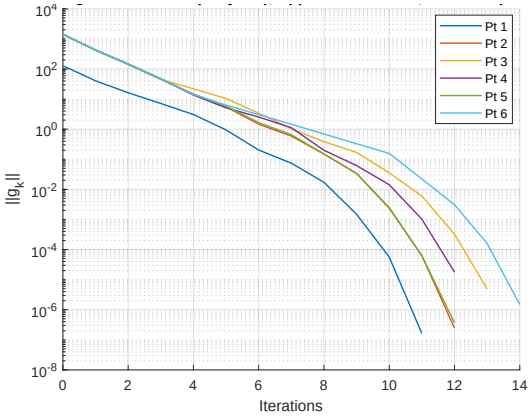


(b) Truncated Newton

Figure 68: Rates of convergence Broyden with all derivatives approximated, k = 4, mode = adaptive

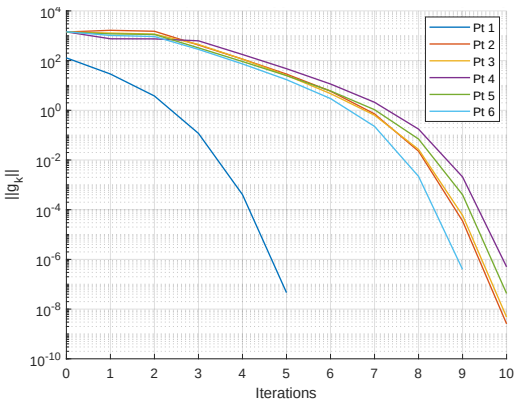


(a) Modified Newton

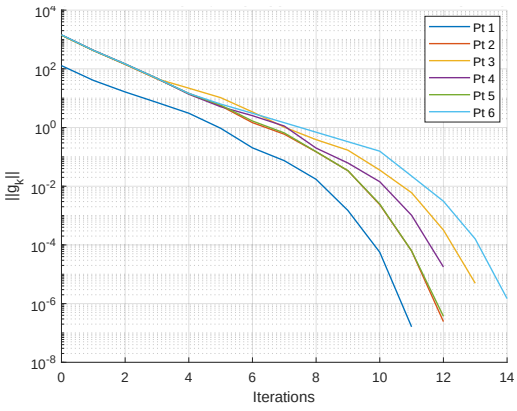


(b) Truncated Newton

Figure 69: Rates of convergence Broyden with all derivatives approximated, k = 8, mode = fixed

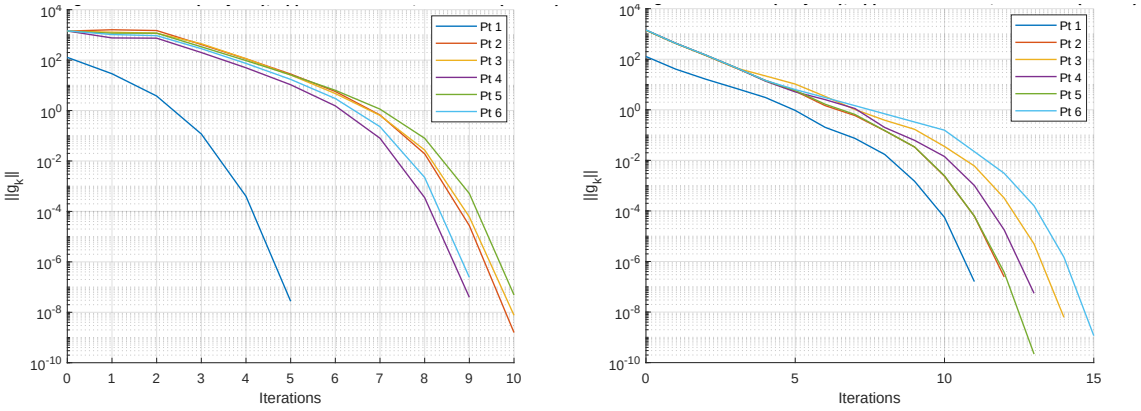


(a) Modified Newton



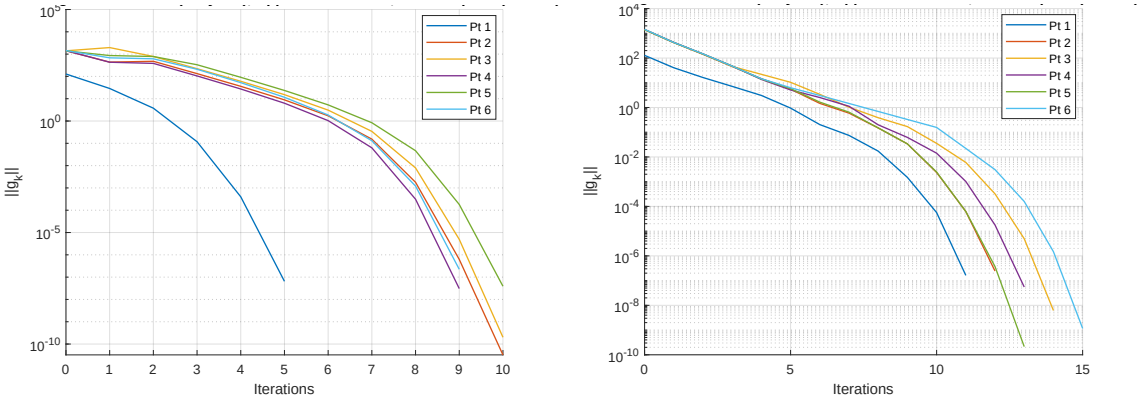
(b) Truncated Newton

Figure 70: Rates of convergence Broyden with all derivatives approximated, k = 8, mode = adaptive



(a) Modified Newton (b) Truncated Newton

Figure 71: Rates of convergence Broyden with all derivatives approximated, k = 12, mode = fixed

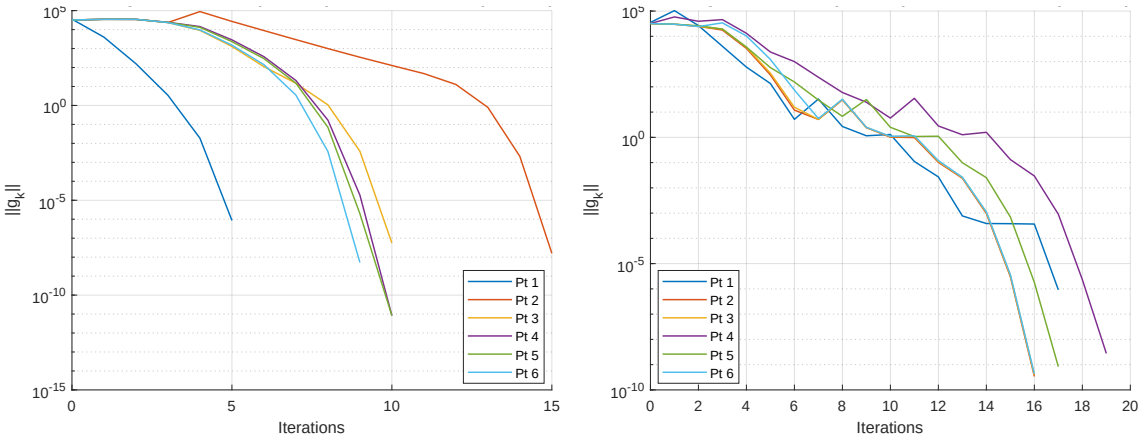


(a) Modified Newton (b) Truncated Newton

Figure 72: Rates of convergence Broyden with all derivatives approximated, k = 12, mode = adaptive

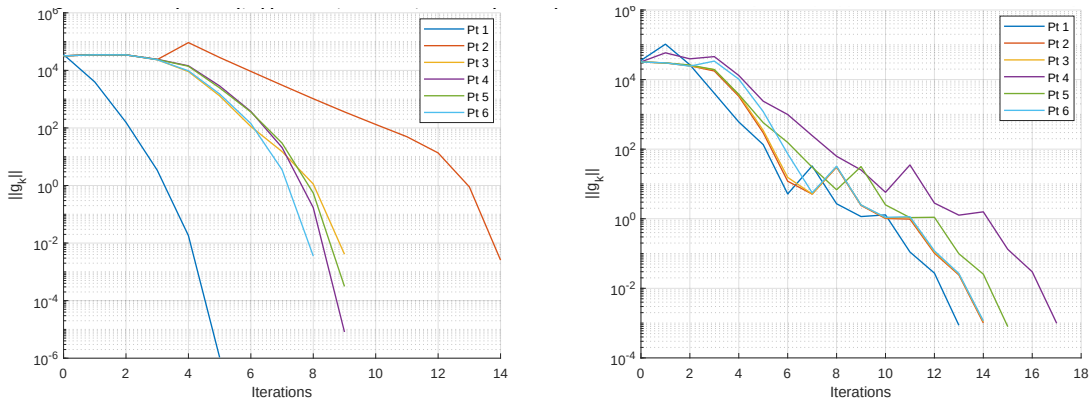
6.3 Large Scale Analysis ($n = 10^4$ and $n = 10^5$)

6.3.1 Chained Freudstain-Roth, Rates of convergence ($n = 10^4$)



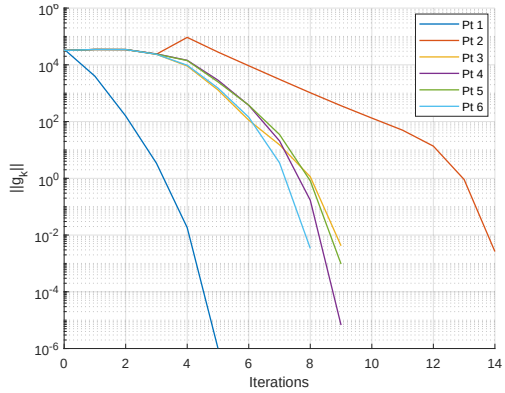
(a) Modified Newton (b) Truncated Newton

Figure 73: Rates of convergence Chained with exact derivatives

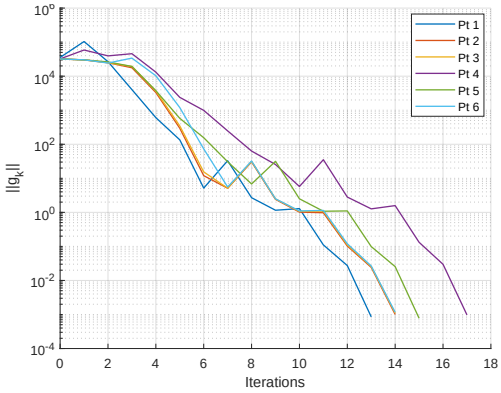


(a) Modified Newton (b) Truncated Newton

Figure 74: Rates of convergence Chained with approximated Hessian, k = 4, mode = fixed

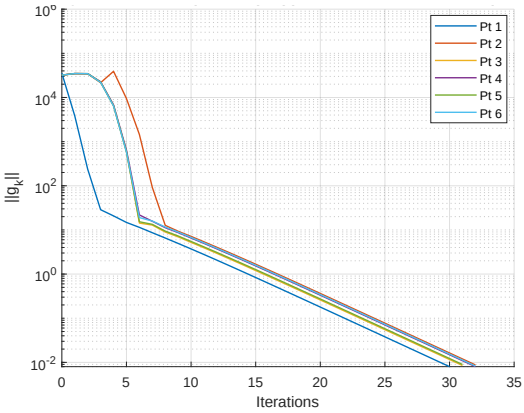


(a) Modified Newton

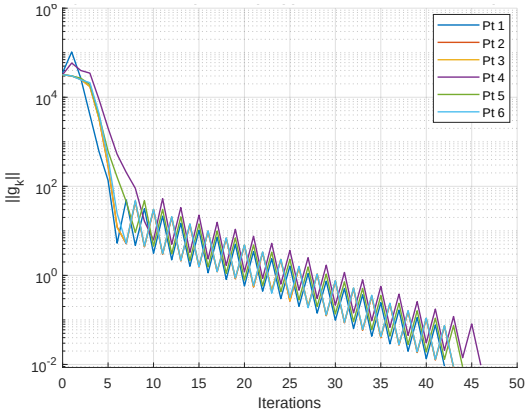


(b) Truncated Newton

Figure 75: Rates of convergence Chained with approximated Hessian, k = 4, mode = adaptive

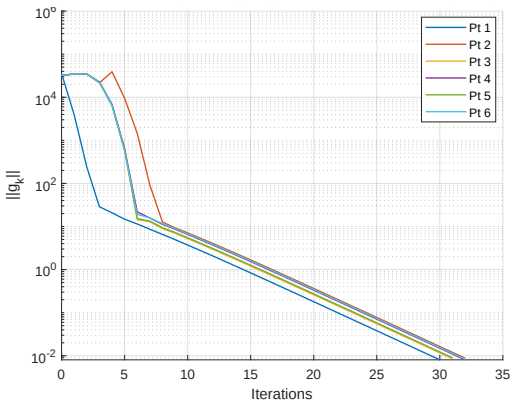


(a) Modified Newton

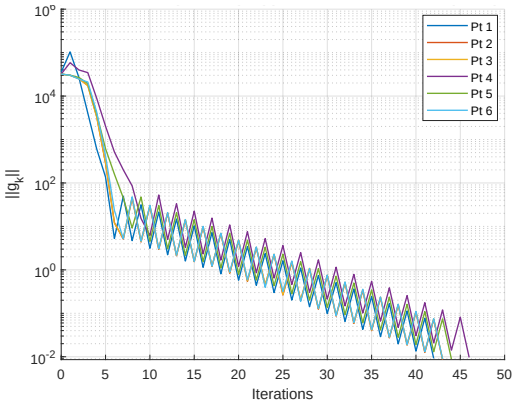


(b) Truncated Newton

Figure 76: Rates of convergence Chained with all derivatives approximated, k = 4, mode = fixed

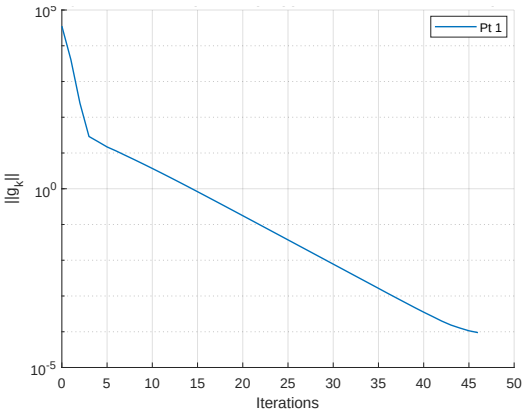


(a) Modified Newton

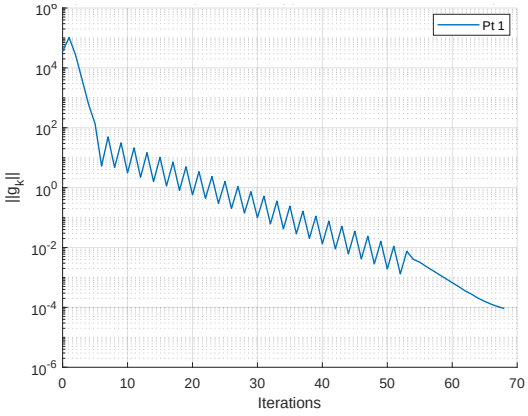


(b) Truncated Newton

Figure 77: Rates of convergence Chained with all derivatives approximated, k = 4, mode = adaptive

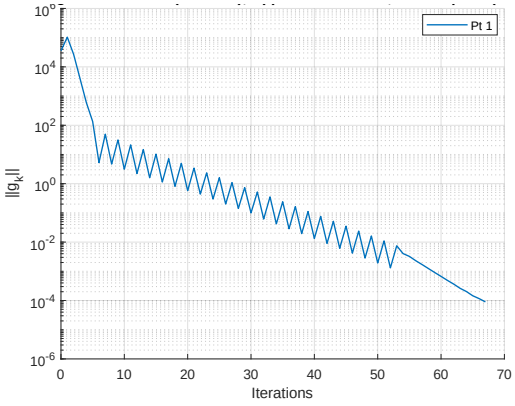


(a) Modified Newton



(b) Truncated Newton

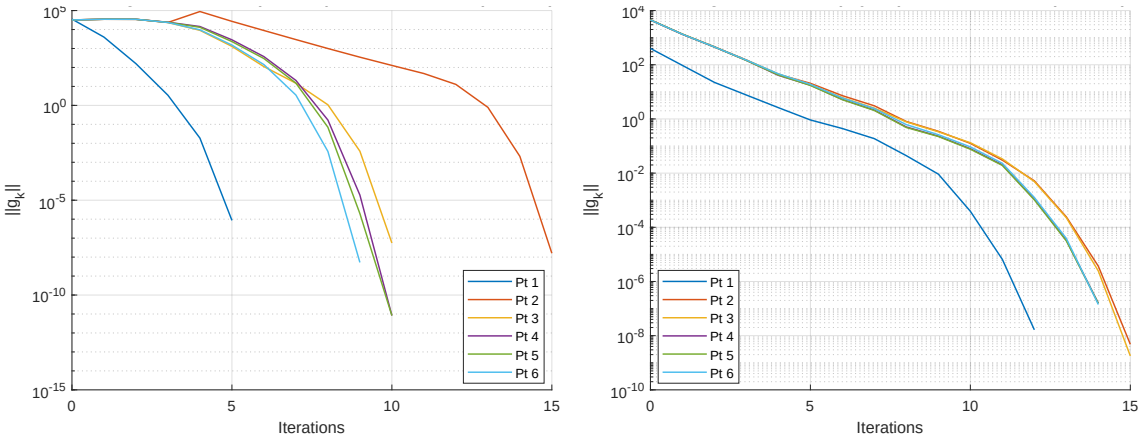
Figure 78: Rates of convergence Chained with all derivatives approximated, k = 8, mode = fixed



(a) Truncated Newton

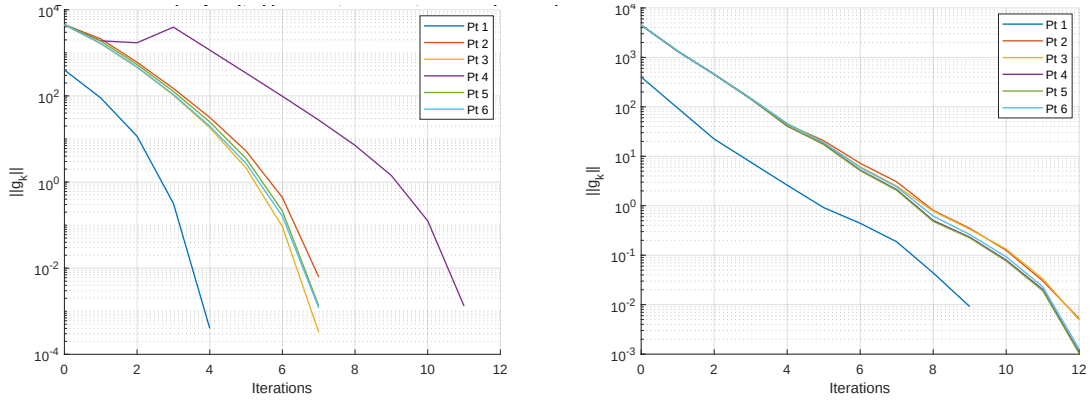
Figure 79: Rates of convergence Chained with all derivatives approximated, k = 8, mode = fixed

6.3.2 Bryden Tridiagonal, Rates of convergence ($n = 10^4$)



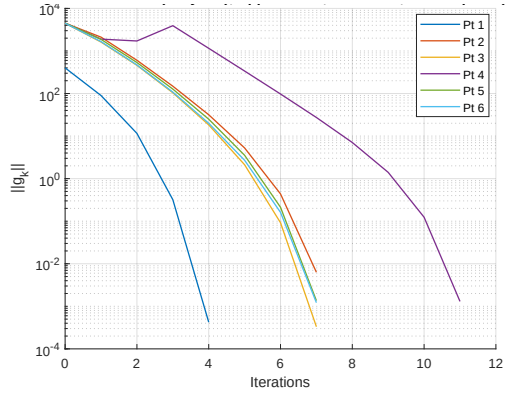
(a) Modified Newton (b) Truncated Newton

Figure 80: Rates of convergence Broyden with exact derivatives

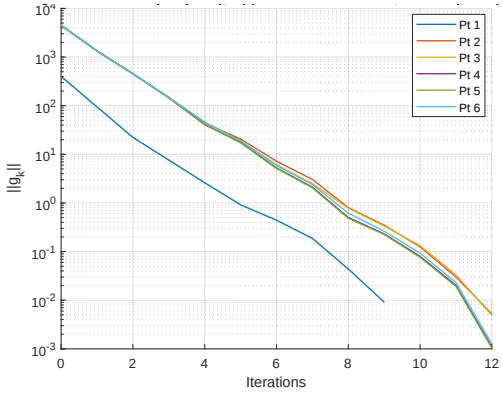


(a) Modified Newton (b) Truncated Newton

Figure 81: Rates of convergence Broyden with approximated Hessian, k = 4, mode = fixed

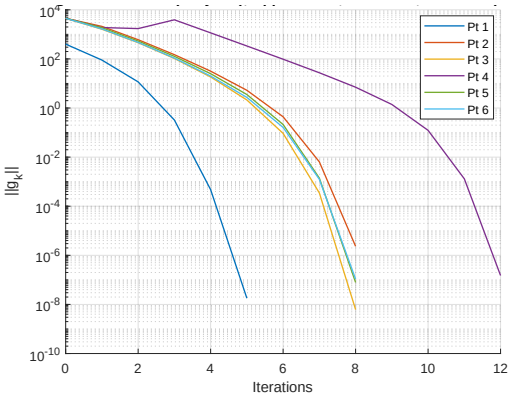


(a) Modified Newton

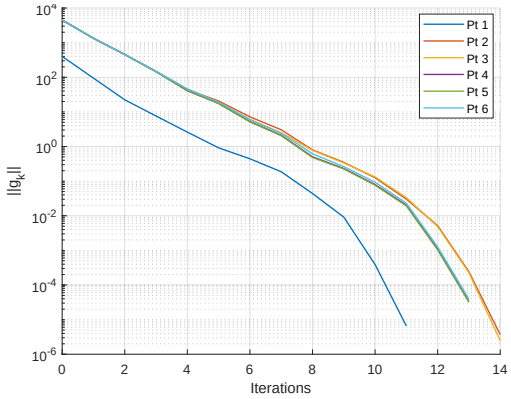


(b) Truncated Newton

Figure 82: Rates of convergence Broyden with approximated Hessian, k = 4, mode = adaptive

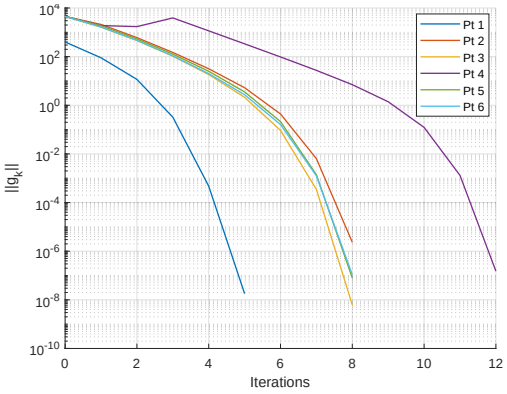


(a) Modified Newton

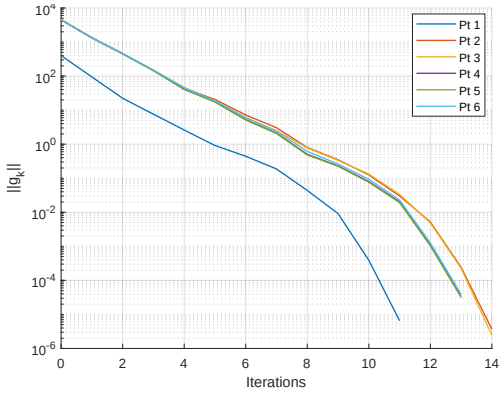


(b) Truncated Newton

Figure 83: Rates of convergence Broyden with approximated Hessian, k = 8, mode = fixed

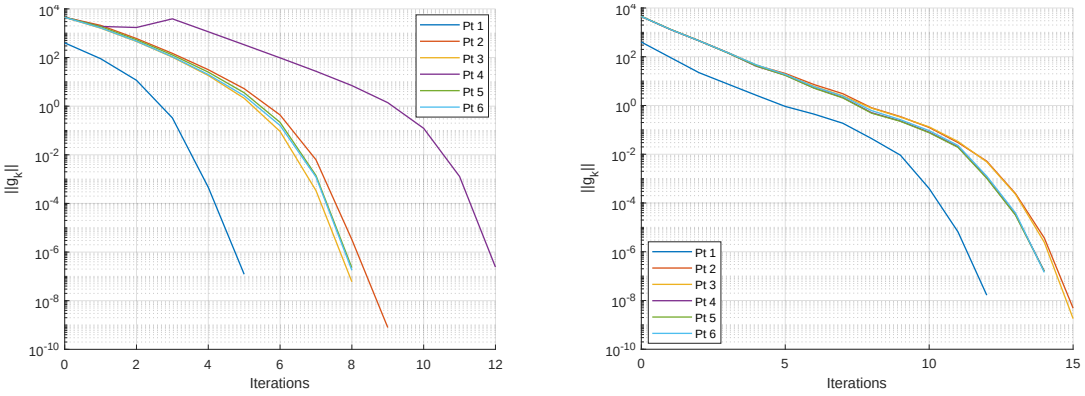


(a) Modified Newton



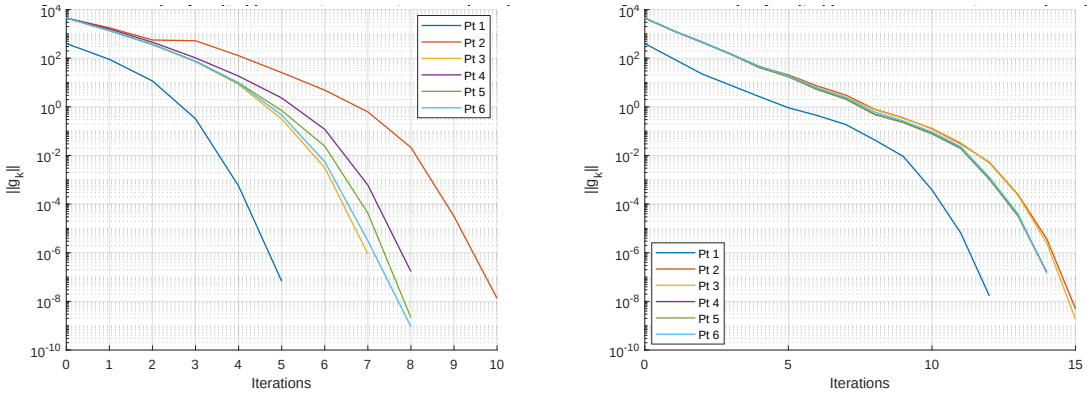
(b) Truncated Newton

Figure 84: Rates of convergence Broyden with approximated Hessian, k = 8, mode = adaptive



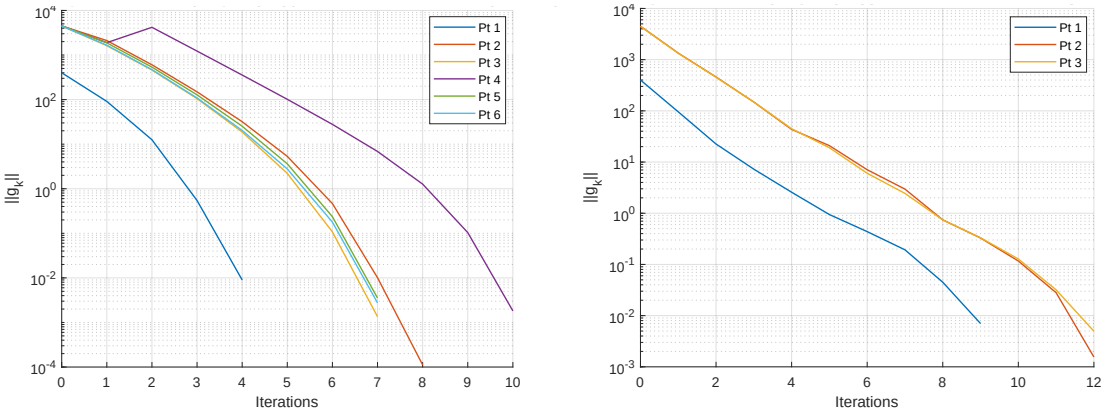
(a) Modified Newton (b) Truncated Newton

Figure 85: Rates of convergence Broyden with approximated Hessian, k = 12, mode = fixed



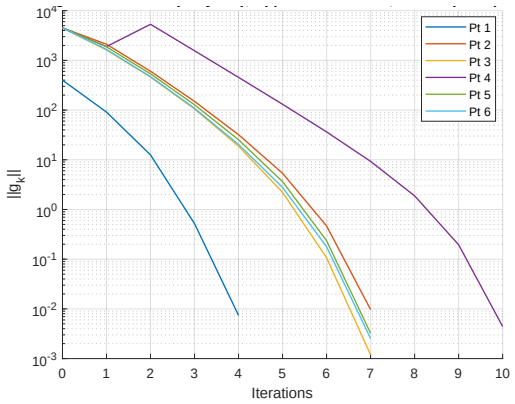
(a) Modified Newton (b) Truncated Newton

Figure 86: Rates of convergence Broyden with approximated Hessian, k = 12, mode = adaptive

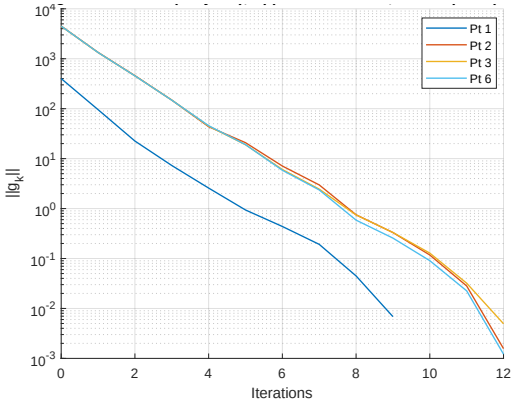


(a) Modified Newton (b) Truncated Newton

Figure 87: Rates of convergence Broyden with all derivatives approximated, k = 4, mode = fixed

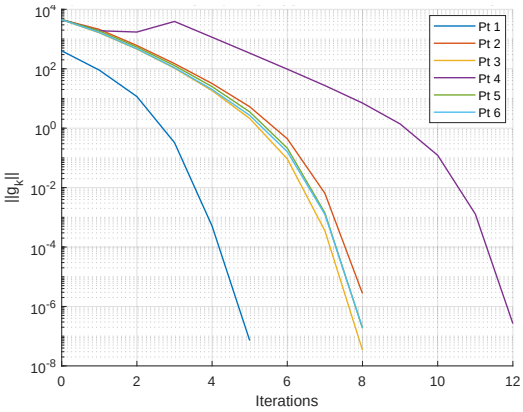


(a) Modified Newton

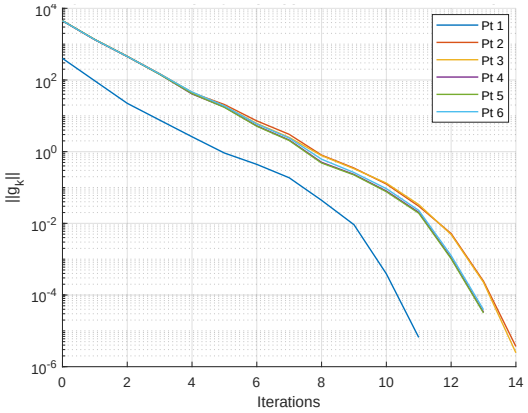


(b) Truncated Newton

Figure 88: Rates of convergence Broyden with all derivatives approximated, k = 4, mode = adaptive

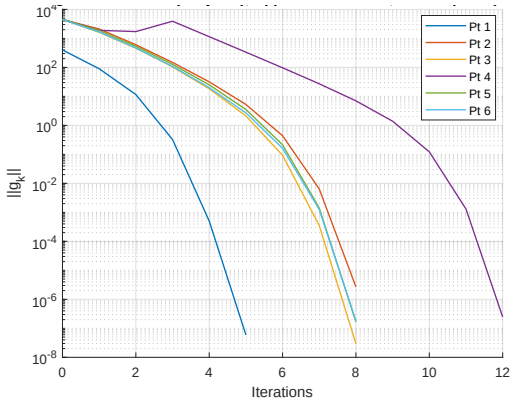


(a) Modified Newton

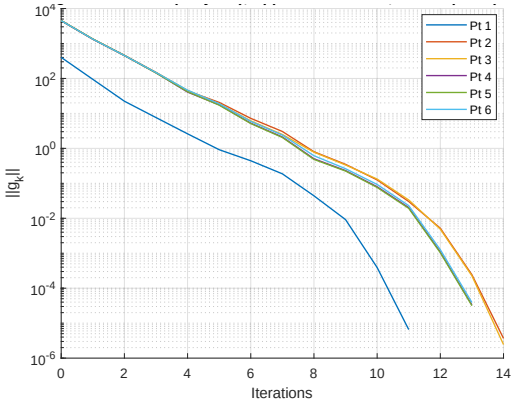


(b) Truncated Newton

Figure 89: Rates of convergence Broyden with all derivatives approximated, k = 8, mode = fixed

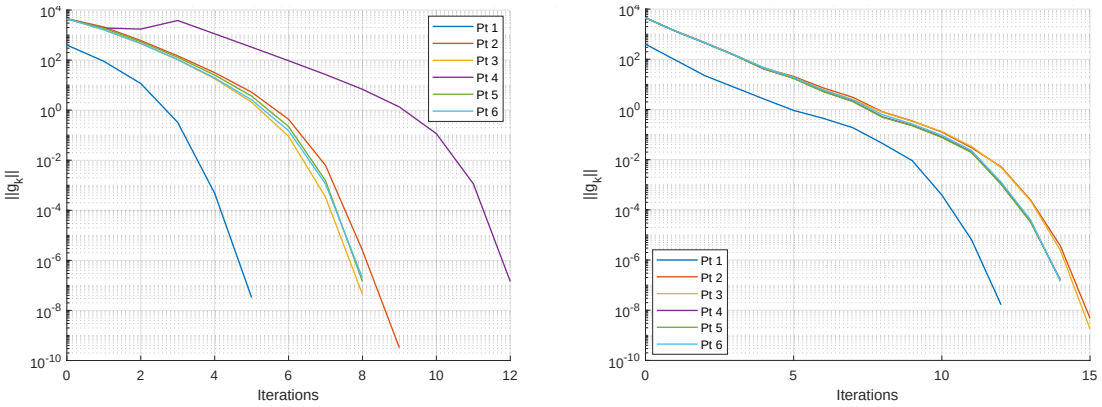


(a) Modified Newton



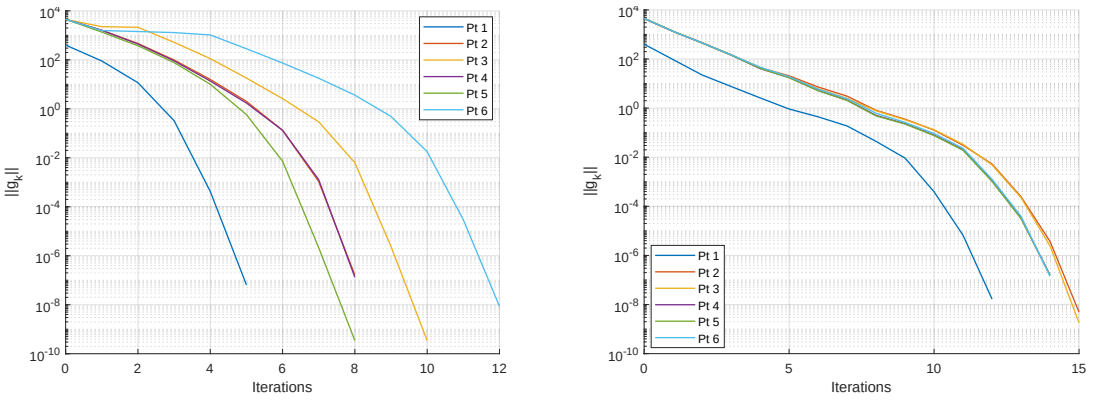
(b) Truncated Newton

Figure 90: Rates of convergence Broyden with all derivatives approximated, k = 8, mode = adaptive



(a) Modified Newton (b) Truncated Newton

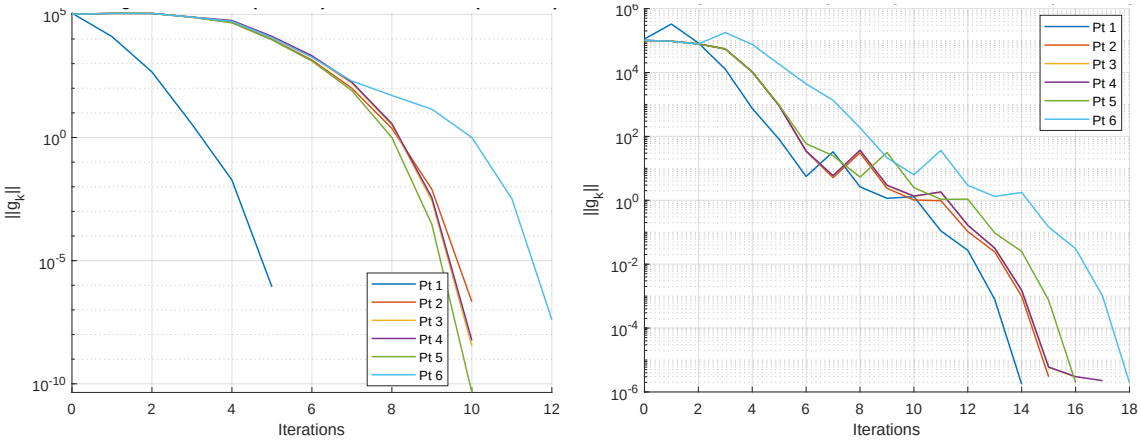
Figure 91: Rates of convergence Broyden with all derivatives approximated, k = 12, mode = fixed



(a) Modified Newton (b) Truncated Newton

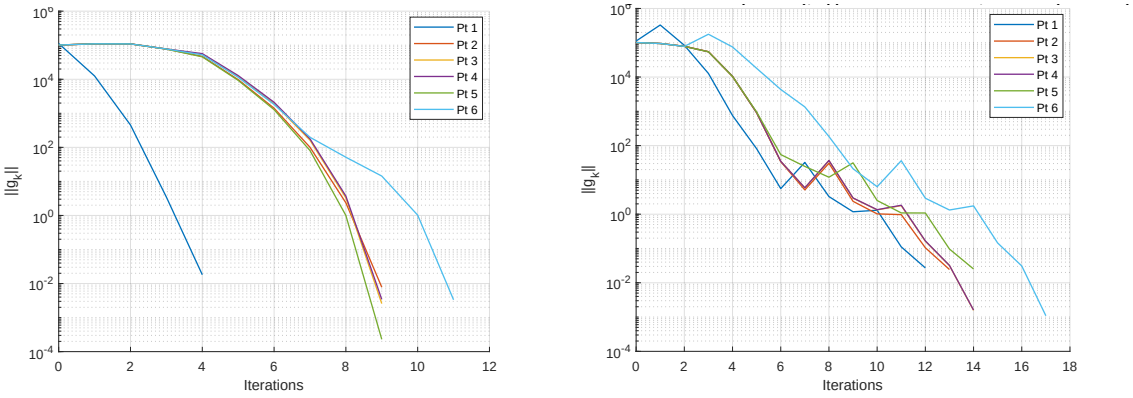
Figure 92: Rates of convergence Broyden with all derivatives approximated, k = 12, mode = adaptive

6.3.3 Chained Freudstain-Roth, Rates of convergence ($n = 10^5$)



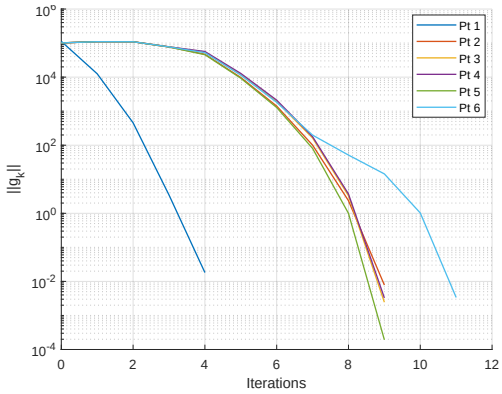
(a) Modified Newton (b) Truncated Newton

Figure 93: Rates of convergence Chained with exact derivatives

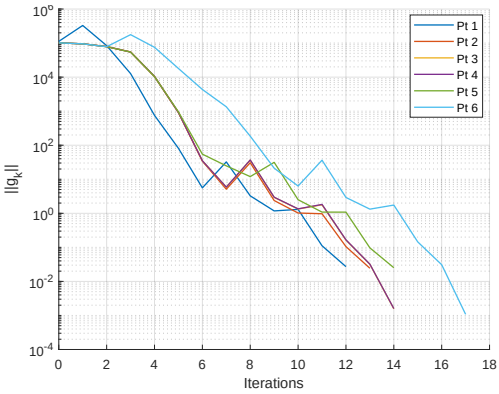


(a) Modified Newton (b) Truncated Newton

Figure 94: Rates of convergence Chained with approximated Hessian, k = 4, mode = fixed

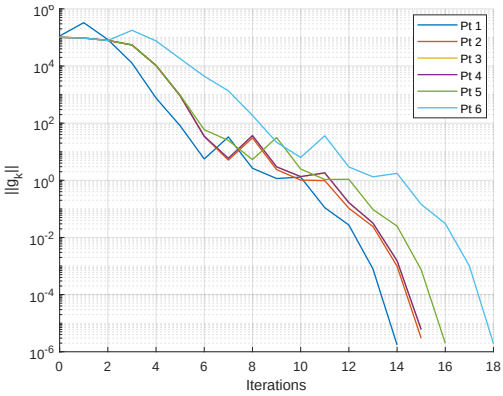


(a) Modified Newton



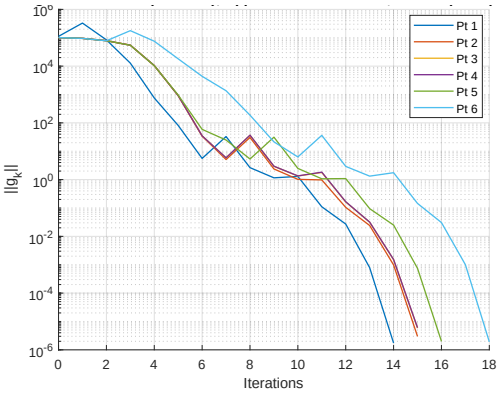
(b) Truncated Newton

Figure 95: Rates of convergence Chained with approximated Hessian, k = 4, mode = adaptive



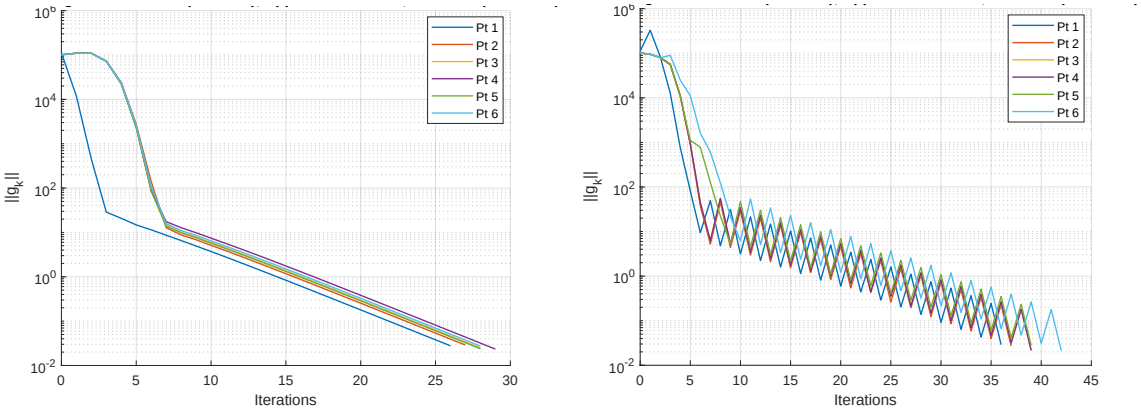
(a) Truncated Newton

Figure 96: Rates of convergence Chained with approximated Hessian, k = 8, mode = fixed



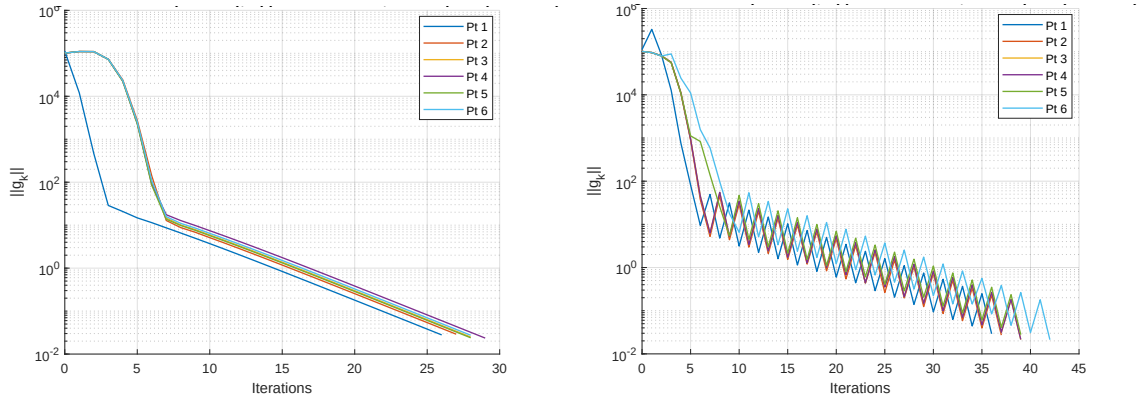
(a) Truncated Newton

Figure 97: Rates of convergence Chained with approximated Hessian, k = 8, mode = adaptive



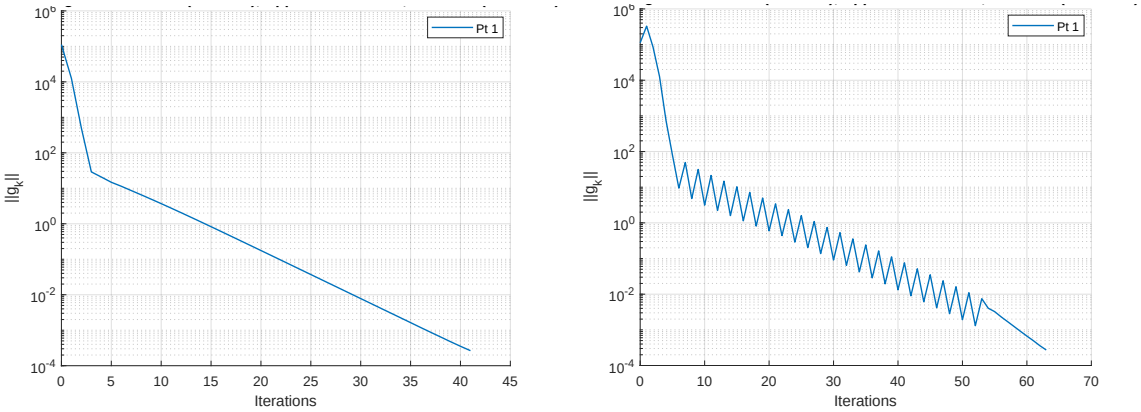
(a) Modified Newton (b) Truncated Newton

Figure 98: Rates of convergence Chained with all derivatives approximated, k = 4, mode = fixed



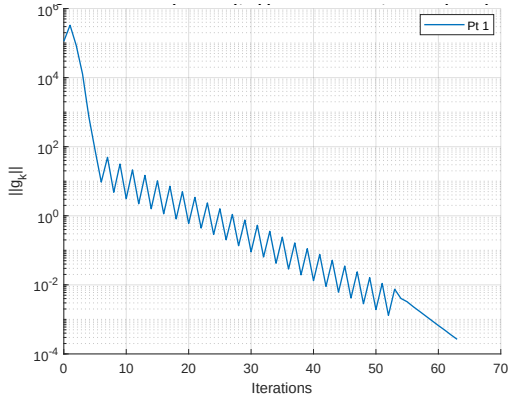
(a) Modified Newton (b) Truncated Newton

Figure 99: Rates of convergence Chained with all derivatives approximated, k = 4, mode = adaptive



(a) Modified Newton (b) Truncated Newton

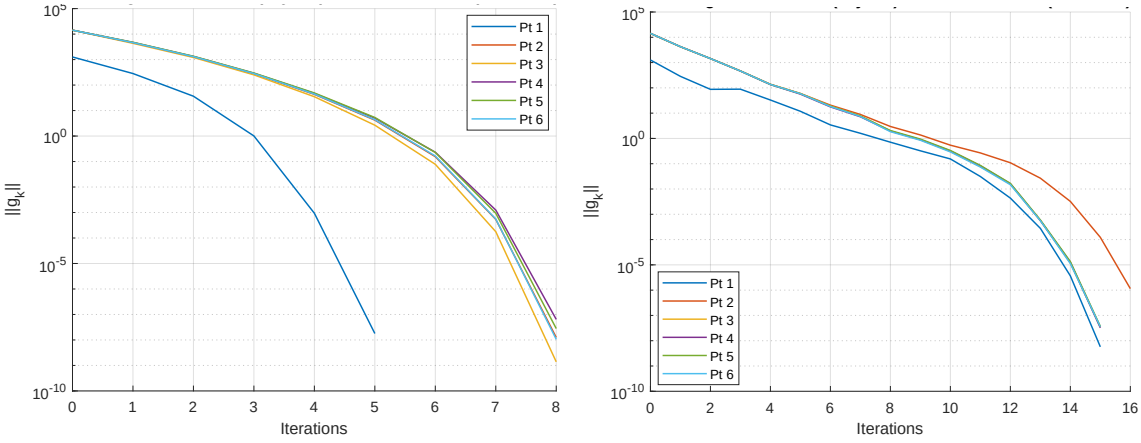
Figure 100: Rates of convergence Chained with all derivatives approximated, k = 8, mode = fixed



(a) Truncated Newton

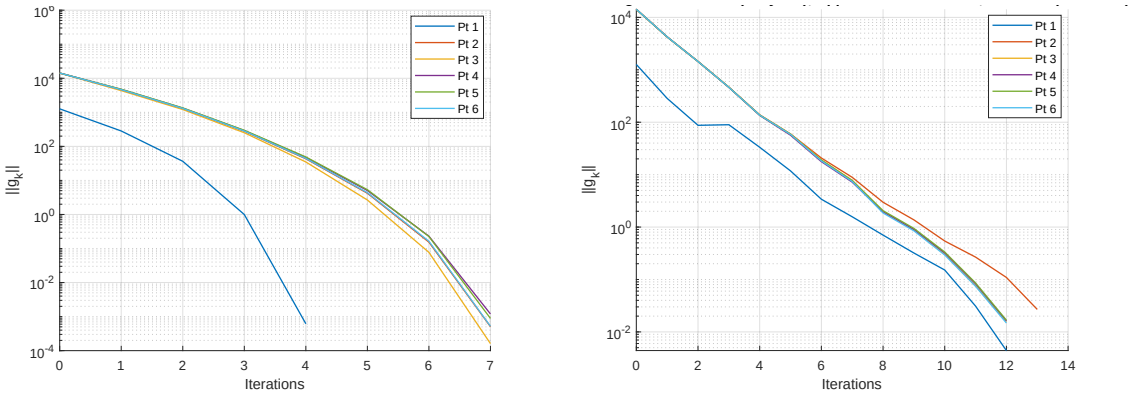
Figure 101: Rates of convergence Chained with all derivatives approximated, k = 8, mode = adaptive

6.3.4 Broyden Tridiagonal, Rates of convergence ($n = 10^5$)



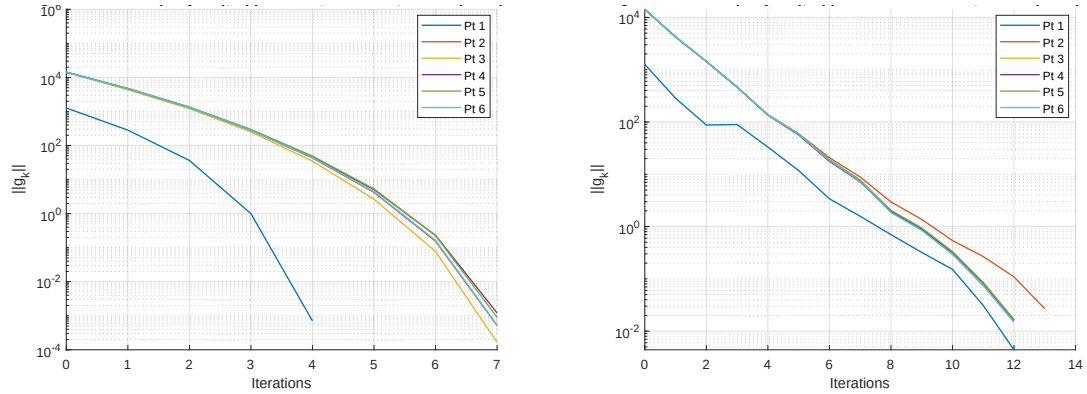
(a) Modified Newton (b) Truncated Newton

Figure 102: Rates of convergence Broyden with exact derivatives



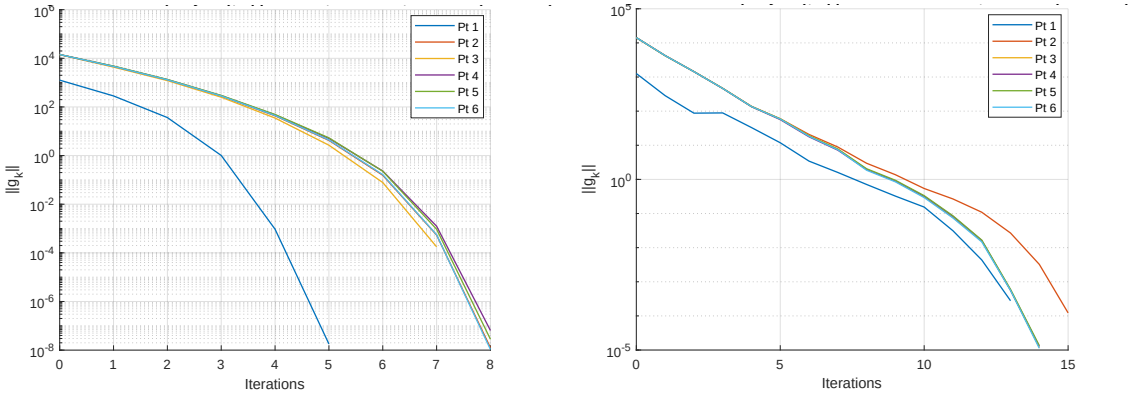
(a) Modified Newton (b) Truncated Newton

Figure 103: Rates of convergence Broyden with approximated Hessian, k = 4, mode = fixed



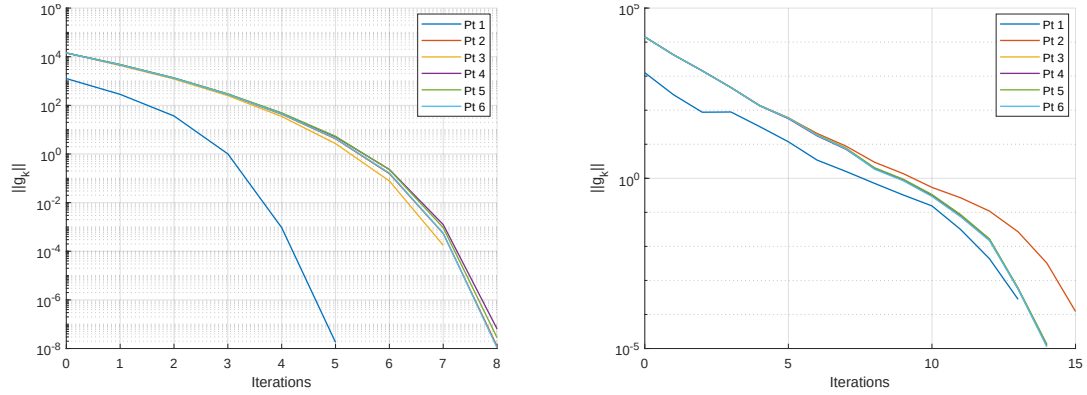
(a) Modified Newton (b) Truncated Newton

Figure 104: Rates of convergence Broyden with approximated Hessian, k = 4, mode = adaptive



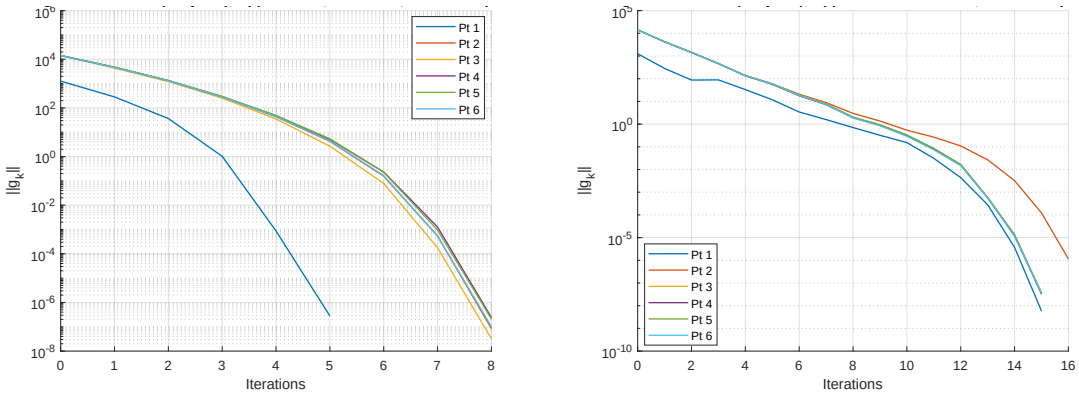
(a) Modified Newton (b) Truncated Newton

Figure 105: Rates of convergence Broyden with approximated Hessian, k = 8, mode = fixed



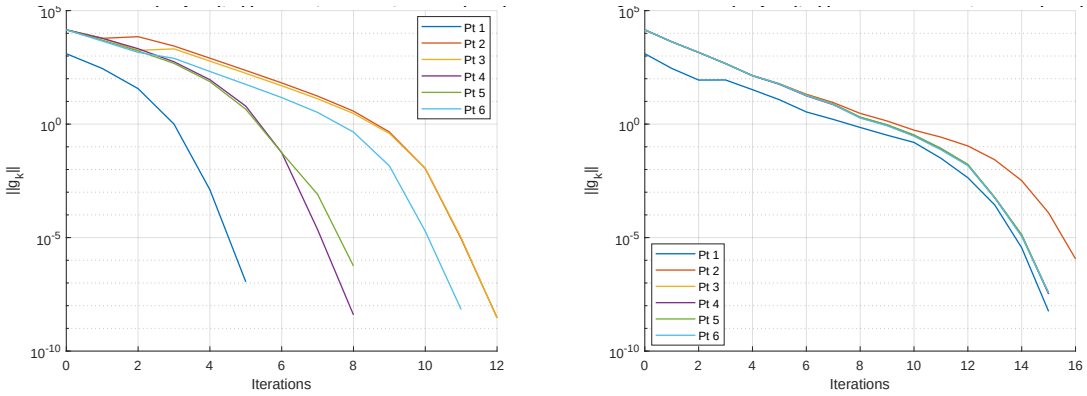
(a) Modified Newton (b) Truncated Newton

Figure 106: Rates of convergence Broyden with approximated Hessian, k = 8, mode = adaptive



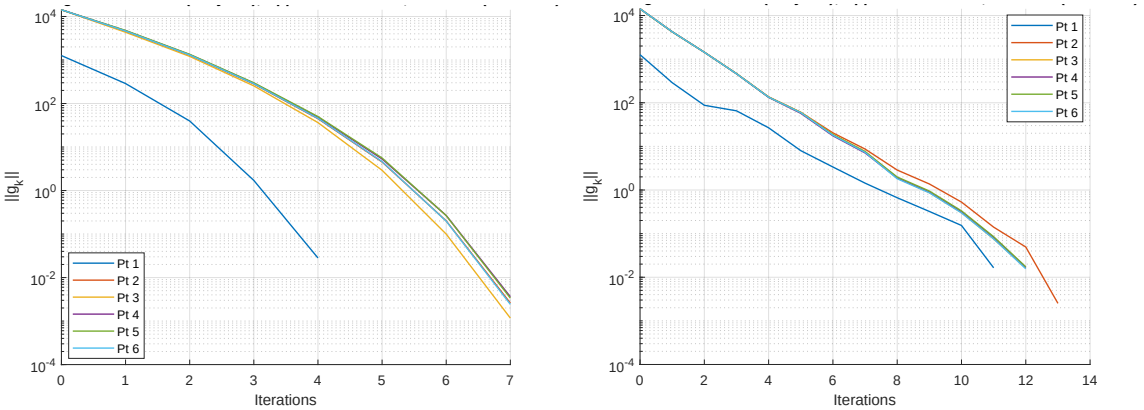
(a) Modified Newton (b) Truncated Newton

Figure 107: Rates of convergence Broyden with approximated Hessian, k = 12, mode = fixed



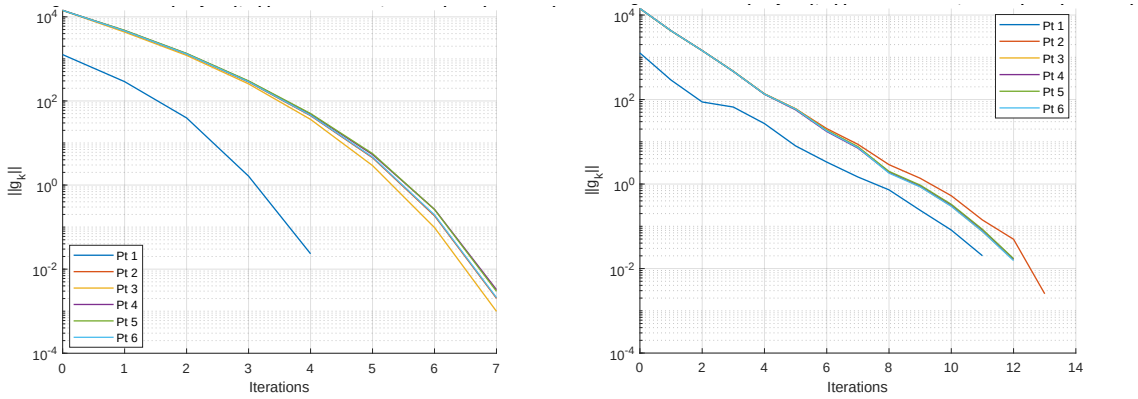
(a) Modified Newton (b) Truncated Newton

Figure 108: Rates of convergence Broyden with approximated Hessian, k = 12, mode = adaptive



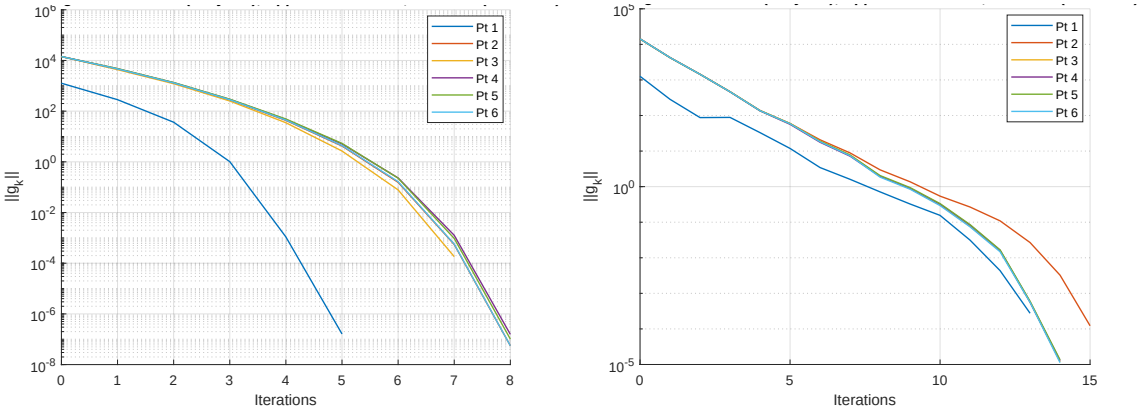
(a) Modified Newton (b) Truncated Newton

Figure 109: Rates of convergence Broyden with all derivatives approximated, k = 4, mode = fixed



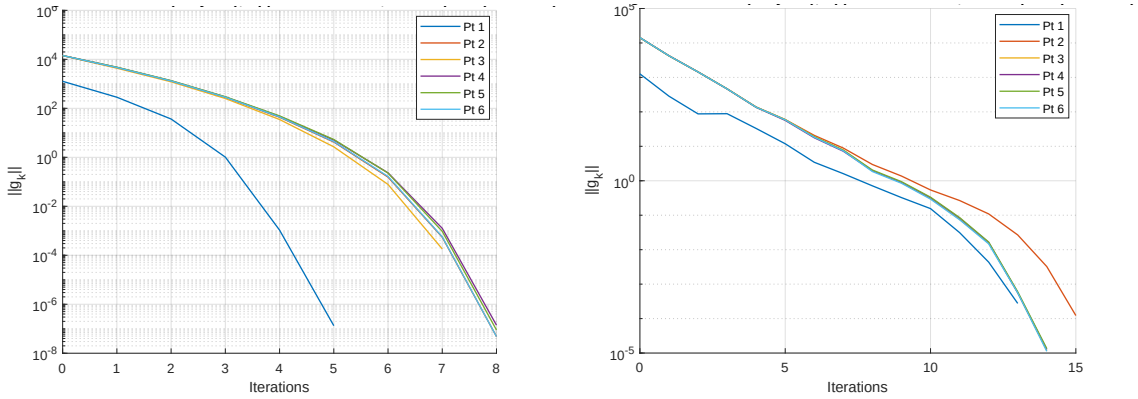
(a) Modified Newton (b) Truncated Newton

Figure 110: Rates of convergence Broyden with all derivatives approximated, k = 4, mode = adaptive



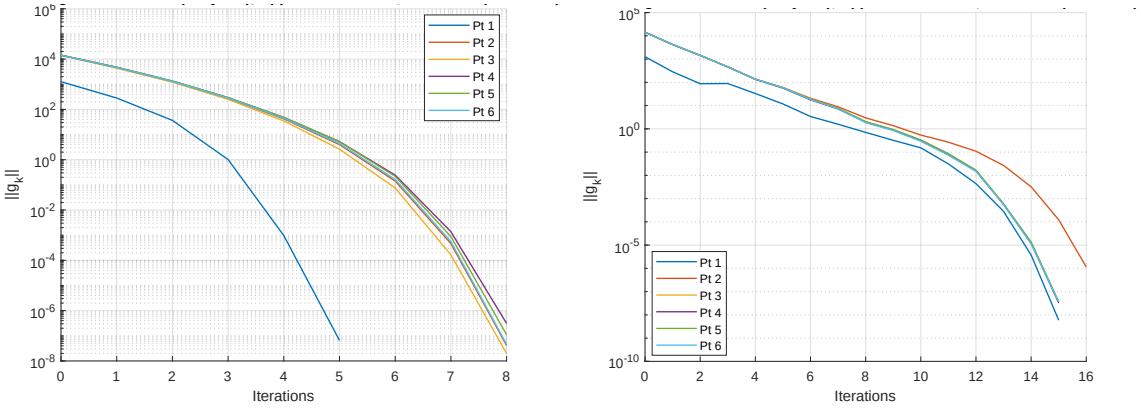
(a) Modified Newton (b) Truncated Newton

Figure 111: Rates of convergence Broyden with all derivatives approximated, k = 8, mode = fixed



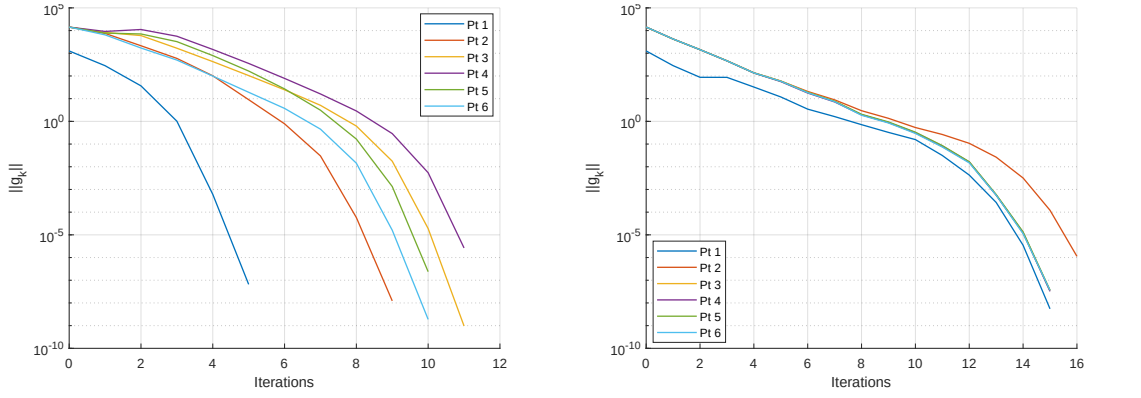
(a) Modified Newton (b) Truncated Newton

Figure 112: Rates of convergence Broyden with all derivatives approximated, k = 8, mode = adaptive



(a) Modified Newton (b) Truncated Newton

Figure 113: Rates of convergence Broyden with all derivatives approximated, k = 12, mode = fixed



(a) Modified Newton (b) Truncated Newton

Figure 114: Rates of convergence Broyden with all derivatives approximated, k = 12, mode = adaptive

7 Code

7.1 Newton methods

7.1.1 Modified Newton

```
function [xk, fk, k, gradfk_norm, xseq, btseq] = MN_backtrack(x0, f, gradf, Hessf
    ,...
                                                          c1, rho, kmax, btmax,
    jmax, tolgrad)
%Initialization
n = length(x0);
xseq = zeros(n,kmax);
btseq = zeros(1, kmax);

xk = x0;
k = 0;
fk = f(xk);
gradfk = gradf(xk);
gradfk_norm = norm(gradfk);

% parameters to catch stagnating behaviour
tolX = 1e-12;       % min treshold for x
tolF = 1e-12;       % min treshold f
max_patience = 150;   % how many iterations to wait before addressing stagnation
patience_cnt = 0;   % counter

%NEWTON LOOP
while k < kmax && gradfk_norm >= tolgrad

    %Compute the descent direction
    beta = 1e-4;
    hessfk = Hessf(xk);
    lambda_min = min(diag(hessfk));
    successo = false;

    if(lambda_min > 0 )
        tao = 0;
    else
        tao = beta - lambda_min;
    end

    for j = 0:jmax

        Bk = hessfk + tao * speye(n);
        % Cholesky attempt (flag = 0 if success, != 0 otherwise)
        [L, flag] = chol(Bk, 'lower');

        if flag == 0
            successo = true;
            break; %If the decomposition succeeds
        else
            tao = max(2*tao, beta);
        end
    end
    if ~successo
        error('Impossible to let the matrix be positive definite.');
        break;
    end

    y = L \ (-gradfk); % direct solver exploiting Cholesky

    pk = L' \ y; % backward sobstitution


    % Backtracking (Armijo Line Search)
    alpha = 1;
    xnew = xk + alpha*pk;
    fnew = f(xnew);

    c1_gradfk_pk = c1 * gradfk' * pk;

    bt = 0;
```

```
    while bt < btmax && fnew > (fk + alpha * c1_gradfk_pk)
        alpha = rho * alpha;
        xnew = xk + alpha * pk;
        fnew = f(xnew);
        bt = bt+1;
    end

    if bt == btmax && fnew > (fk + alpha * c1_gradfk_pk)
        fprintf('Backtracking failed.\n');
        break
    end

    % if algorithm stagnates, interrupt execution
    rel_step_x = norm(xnew - xk) / (1 + norm(xk));
    rel_diff_f = abs(fnew - fk) / (1 + abs(fk));

    if rel_step_x < tolX || rel_diff_f < tolF
        patience_cnt = patience_cnt + 1;
    else
        patience_cnt = 0;
    end

    if patience_cnt >= max_patience
           fprintf('Stagnation at iteration %d.\n', k);
           break;
    end

    % Updating variables
    xk = xnew;
    fk = fnew;
    gradfk = gradf(xk);
    gradfk_norm = norm(gradfk);

    k = k+1;

    xseq(:, k) = xk;
    btseq(k) = bt;
end

xseq = xseq(:,1:k);  %trim
btseq = btseq(:,1:k);
end
```

Listing 1: Modified Newton with backtracking

7.1.2 Truncated Newton

```
function [xk, fk, k, gradfk_norm, xseq, btseq] = truncated_newton(x0, f, gradf,
    Hessf, ...
                                                          c1, rho, kmax, btmax,
    tolgrad)
    % Initialization
    xseq = zeros(length(x0), kmax);
    btseq = zeros(1, kmax);

    xk = x0;
    k = 0;
    fk = f(xk);
    gradfk = gradf(xk);
    gradfk_norm = norm(gradfk);

    % parameters to catch stagnating behaviour
    tolX = 1e-12;       % min treshold for x
    tolF = 1e-12;       % min treshold f
    max_patience = 150;   % how many iterations to wait before addressing
    stagnation
    patience_cnt = 0;   % counter

    % NEWTON (External loop)
    while k < kmax && gradfk_norm >= tolgrad

        %  CONJUGATE GRADIENT (Inner Loop)
```

```
        Bk = Hessf(xk);

        z = zeros(length(xk), 1);
        r = -gradfk;
        d = r;

        eta_k = (min(0.5, sqrt(gradfk_norm))); % sqrt for superlinear convergence

        % if CG fails immediatly, use the gradient (Steepest Descent)
        pk = -gradfk;

        for j = 1:length(xk)
            Bd = Bk * d;
            curvature = d' * Bd;

            % Curvature check
            if curvature > 0
                alpha_cg = (r' * r) / curvature;
                z_new = z + alpha_cg * d;
                r_new = r - alpha_cg * Bd;
            else
                % Negative curvature
                if j == 1
                    pk = -gradfk;
                else
                    pk = z;
                end
                break;
            end

            %  Convergence check
            if norm(r_new) <= eta_k * gradfk_norm
                pk = z_new;
                break;  % Convergenza reached, exit
            end

            % If not finished, update the direction
            beta_cg = (r_new' * r_new) / (r' * r);
            d = r_new + beta_cg * d;

            % Updating variables for next iteration
            z = z_new;
            r = r_new;

            % If we reach last iteration, without break
            if j == length(xk)
                pk = z;
            end
        end
        % END INNER LOOP

        % Backtracking (Armijo Line Search)
        alpha = 1;
        xnew = xk + alpha * pk;
        fnew = f(xnew);

        c1_gradfk_pk = c1 * gradfk' * pk;

        bt = 0;
        while bt < btmax && fnew > (fk + alpha * c1_gradfk_pk)
            alpha = rho * alpha;
            xnew = xk + alpha * pk;
            fnew = f(xnew);
            bt = bt + 1;
        end

        if bt == btmax && fnew > (fk + alpha * c1_gradfk_pk)
            fprintf('Backtracking failed.\n');
            break
        end

        % if algorithm stagnates, interrupt execution
        rel_step_x = norm(xnew - xk) / (1 + norm(xk));
```

```
        rel_diff_f = abs(fnew - fk) / (1 + abs(fk));

        if rel_step_x < tolX || rel_diff_f < tolF
            patience_cnt = patience_cnt + 1;
        else
            patience_cnt = 0;
        end

        if patience_cnt >= max_patience
           fprintf('Stagnation at iteration %d.\n', k);
           break;
        end

        % Updating variables
        xk = xnew;
        fk = fnew;
        gradfk = gradf(xk);
        gradfk_norm = norm(gradfk);

        k = k + 1;

        xseq(:, k) = xk;
        btseq(k) = bt;
    end

    xseq = xseq(:, 1:k); %trim
    btseq = btseq(1:k);
end
```

Listing 2: Truncated Newton with Conjugate Gradient

7.2 Functions

7.2.1 Broyden Tridiagonal

```
function [F, g, Hess] = broyden_tridiagonal(x)
    n = length(x);

    x_prev = [0; x(1:n-1)]; %  x_{k-1}
    x_next = [x(2:n); 0];   %  x_{k+1}

    f = (3 - 2*x) .* x - x_prev - 2.*x_next + 1;

    F = 0.5 * sum(f.^2);

    if nargout > 1
        d = 3 - 4*x;
        l = -ones(n, 1);
        u = -2 * ones(n, 1);

        % Building the Jacobian
        J = spdiags([l d u], -1:1, n, n);
    end

    % Computing the gradient: g = J' * f
    if nargout >= 2   % compute the gradient only if needed
        % g_k = -2*f_{k-1} + (3-4x_k)*f_k - f_{k+1}
        f_prev = [0; f(1:n-1)];
        f_next = [f(2:n); 0];
        g = (3 - 4*x) .* f - 2*f_prev - f_next;
    end

    % evaluating the Hessian: Hess = J' * J + sum(f_k * nabla^2 f_k)
    if nargout == 3  % compute the Hessian only if needed
        % Curvature term (diagonal)
        curvature_term = spdiags(-4 * f, 0, n, n);
        Hess = J' * J + curvature_term;
    end
end
```

Listing 3: Broyden Tridiagonal function implementation

7.2.2 Chained Freudenstein-Roth function

```
function [F, g, Hess] = chained_freudenstein_roth(x)
    [n, ~] = size(x);

    x_i    = x(1:n-1,:); %  x_{k}
    x_next = x(2:n,:);   %  x_{k+1}

    f_odd = x_i + x_next .* ((5 - x_next) .* x_next - 2) - 13;
    f_even    = x_i + x_next .* ((1 + x_next) .* x_next - 14) - 29;

    F = 0.5 * (sum(f_odd.^2, 1) + sum(f_even.^2, 1));

    if nargout > 1
        % First order derivatives w.r.t. x_{i+1}
        d_odd  = 10*x_next - 3*x_next.^2 - 2;
        d_even = 3*x_next.^2 + 2*x_next - 14;

        % Building the Jacobian
        m = 2*(n-1);
        r = (1:m)';
        c1 = ceil(r/2);   % Index for x_i
        c2 = c1 + 1;      % Index for x_{i+1}

        v1 = ones(m, 1);  % df/dxi are 1
        v2 = zeros(m, 1); % df/dx_next (alterned among odd and even)
        v2(1:2:end) = d_odd;
        v2(2:2:end) = d_even;

        rows = [r; r];
        cols = [c1; c2];
        vals = [v1; v2];

        J = sparse(rows, cols, vals, m, n);

        % Computing the gradient: g = J' * f
        if nargout >= 2    % compute the gradient only if needed
            f_grad = zeros(m, 1);
            f_grad(1:2:end) = f_odd;
            f_grad(2:2:end) = f_even;

            g = J' * f_grad;
        end

    % Computing the Hessian: Hess = J' * J + sum(f_k * nabla^2 f_k)
    if nargout == 3 % compute the Hessian only if needed
        % Second order derivatives w.r.t x_{i+1}
        d2_odd  = 10 - 6*x_next;
        d2_even = 6*x_next + 2;

        % Curvature term (diagonal)
        curvature_term = zeros(n, 1);
        curvature_term(2:end) = f_odd .* d2_odd + f_even .* d2_even;

        Hess = J' * J + spdiags(curvature_term, 0, n, n);
    end
end
```

Listing 4: Chained Freudenstein-Roth function implementation

7.3 Efficient implementation of functions' derivatives using the exact gradient

7.3.1 Efficient implementation - Broyden Tridiagonal/ mode 1

```
function H_fd = broyden_derivatives_grad(x, k, mode)
    n = length(x);
    base_h = 10^(-k);

    % gradient evaluation
    [~, g_exact] = broyden_tridiagonal(x);
```

```
7
8       %creating vector h according to the mode (fixed/adaptive)
9       if strcmp(mode, 'fixed')
10          h = ones(n, 1) * base_h;
11      else
12          h = base_h * max(abs(x), 1e-8);
13          h(h < eps) = base_h; % if x_i is zero, h_i would be zero
14      end
15
16      % Exploiting pentadiagonal structure, we compute only the local
17      % interactions
18      H_diag0 = zeros(n, 1);
19      H_diag1 = zeros(n-1, 1);
20      H_diag2 = zeros(n-2, 1);
21
22      % Defining 5 groups for pentadiagonal structure
23      for group = 1:5
24          % Columns indexes for this group
25          idx = group:5:n;
26
27          h_vec = zeros(n, 1); % Perturbation vector
28          h_vec(idx) = h(idx);
29
30          % Evaluation of the gradient for 1/5 of total columns
31          [~, g_plus] = broyden_tridiagonal(x + h_vec);
32
33          y = g_plus - g_exact;
34
35          %creating diagonal vectors
36          for j = idx
37              H_diag0(j) = y(j) / h(j);
38
39              if j <= n
40                  H_diag1(j) = y(j+1) / h(j);   % Superdiagonal 1
41              end
42
43              if j < n-1
44                  H_diag2(j) = y(j+2) / h(j);   % Superdiagonal 2
45              end
46          end
47      end
48
49      % Creating sparse tridiagonal matrix
50      H_fd = spdiags([[H_diag2;0;0], [H_diag1;0], H_diag0, [0;H_diag1], [0;0;H_diag2
        ]], -2:2, n, n);
51
52      % imposing symmetry to avoid numerical errors
53      H_fd = (H_fd + H_fd') / 2;
54  end
```

Listing 5: Efficient implementation of Broyden Tridiagonal using the exact gradient

7.3.2 Efficient implementation - Chained-Freudenstein-Roth/ mode 1

```
1   function H_fd = chained_derivatives_grad(x, k, mode)
2       n = length(x);
3       base_h = 10^(-k);
4
5       % Gradient evaluation
6       [~, g_exact] = chained_freudenstein_roth(x);
7
8       %creating vector h according to the mode (fixed/adaptive)
9       if strcmp(mode, 'fixed')
10          h = ones(n, 1) * base_h;
11      else
12          h = base_h * max(abs(x), 1e-8);
13          h(h < eps) = base_h; % if x_i is zero, h_i would be zero
14
15      end
16
17      % Exploiting tridiagoal structure, we compute only the local
18      % interactions
```

```
    H_diag0 = zeros(n, 1);
    H_diag1 = zeros(n-1, 1);

    % Defining 3 groups for tridiagonal structure
    for group = 1:3
        % Columns indexes for this group
        idx = group:3:n;

        h_vec = zeros(n, 1); % Perturbation vector
        h_vec(idx) = h(idx);

        % Evaluating the gradient for 1/3 of total columns
        [~, g_plus] = chained_freudenstein_roth(x + h_vec);

        y = g_plus - g_exact;

        %creating diagonal vectors
        for j = idx
            H_diag0(j) = y(j) / h(j);   % Main diaginal

            if j < n
                H_diag1(j) = y(j+1) / h(j);  % Superdiagonal
            end
        end
    end

    % Creating sparse tridiagonal matrix
    H_fd = spdiags([[H_diag1; 0], H_diag0, [0; H_diag1]], -1:1, n, n);

    % imposing symmetry to avoid numerical errors
    H_fd = (H_fd + H_fd') / 2;

end
```

Listing 6: Efficient implementation of Chained-Freudenstein-Roth using the exact gradient

7.4 Efficient implementation of ALL functions' derivatives

7.4.1 Efficient implementation - Broyden Tridiagonal / mode 2

```
function g_fd = fd_gradBroyden(x, k, mode)
    n = length(x);
    base_h = 10^(-k);

   %creating vector h according to the mode (fixed/adaptive)
    if strcmp(mode, 'fixed')
        h = ones(n, 1) * base_h;
    else
        h = base_h * max(abs(x), 1e-8);
        h(h < eps) = base_h; % if x_i is zero, h_i would be zero

    end

    % Defining F, it depends by x_previous, x_current and x_next
    calc_fk = @(xp, xc, xn) (3 - 2*xc).*xc - xp - 2*xn + 1;

    % To handle (i-1 and i+1)
    X = [0; x; 0];

    % Base (non perturbed terms)
    f_base = calc_fk(X(1:n), X(2:n+1), X(3:n+2));
    F_base = 0.5 * f_base.^2;

    H = [0; h; 0];

    % Term a: x_i is x_next
    f_prev_h = calc_fk(X(1:n), X(2:n+1), X(3:n+2) + H(3:n+2));

    % Term b: x_i is x_curent
    f_curr_h = calc_fk(X(1:n), X(2:n+1) + H(2:n+1), X(3:n+2));

    % Term c: x_i is x_prev
```

```
    f_next_h = calc_fk(X(1:n) + H(1:n), X(2:n+1), X(3:n+2));

    % Evaluate the perturbed F
    F_prev_h = 0.5 * f_prev_h.^2;
    F_curr_h = 0.5 * f_curr_h.^2;
    F_next_h = 0.5 * f_next_h.^2;

    % Assembling the gradient
    diff_prev = [0; F_prev_h(1:n-1) - F_base(1:n-1)];
    diff_curr = F_curr_h - F_base;
    diff_next = [F_next_h(2:n) - F_base(2:n); 0];

    % Assembling the gradient
    g_fd = (diff_prev + diff_curr + diff_next) ./ h;

end
```

Listing 7: Broyden Tridiagonal efficient finite differences implementation for the gradient

```
function H_fd = fd_HessBroyden(x, k, mode)
    n = length(x);
    base_h = 10^(-k);

    %creating vector h according to the mode (fixed/adaptive)
    if strcmp(mode, 'fixed')
        h = ones(n, 1) * base_h;
    else
        h = base_h * max(abs(x), 1e-8);
        h(h < eps) = base_h; % if x_i is zero, h_i would be zero

    end
    h = sqrt(h); % Step for the Hessian

    % function handles
    calc_fk = @(x_prev, x, x_next) (3 - 2*x).*x - x_prev - 2*x_next + 1;
    calc_F  = @(x_prev, x, x_next) 0.5 * calc_fk(x_prev, x, x_next).^2;

    X = [0; x; 0];
    H_vec = [0; h; 0];

    % Base
    F0 = calc_F(X(1:n), X(2:n+1), X(3:n+2));

    %%  MAIN DIAGONAL
    % Forward (x + h)
    Fp_prev = calc_F(X(1:n), X(2:n+1), X(3:n+2) + H_vec(3:n+2)); % Perturbation of
    x_{k+1} with h_{k+1}
    Fp_curr = calc_F(X(1:n), X(2:n+1) + H_vec(2:n+1), X(3:n+2)); % Perturbation of
    x_k with h_k
    Fp_next = calc_F(X(1:n) + H_vec(1:n), X(2:n+1), X(3:n+2));   % Perturbation of
    x_{k-1} with h_{k-1}

    % Backward (x - h)
    Fm_prev = calc_F(X(1:n), X(2:n+1), X(3:n+2) - H_vec(3:n+2));
    Fm_curr = calc_F(X(1:n), X(2:n+1) - H_vec(2:n+1), X(3:n+2));
    Fm_next = calc_F(X(1:n) - H_vec(1:n), X(2:n+1), X(3:n+2));

    % Central Difference: (F(x+h) - 2F(x) + F(x-h)) / h^2
    D_prev = (Fp_prev - 2*F0 + Fm_prev) ./ (H_vec(3:n+2).^2);
    D_curr = (Fp_curr - 2*F0 + Fm_curr) ./ (H_vec(2:n+1).^2);
    D_next = (Fp_next - 2*F0 + Fm_next) ./ (H_vec(1:n).^2);

    % Assembling the terms for the main diagonal
    diag0 = ([0; D_prev(1:n-1)] + D_curr + [D_next(2:n); 0]);


    %% FIRST SUPERDIAGONAL: interaction among x_i and x_{i+1}

    % Denominator: h_i * h_{i+1}
    h_h1 = H_vec(2:n) .* H_vec(3:n+1);

    % TERM a: x_i is current, x_{i+1} is next
    F_a = calc_F(X(1:n-1), ...
                      X(2:n) + H_vec(2:n), ...      % x_i + h_i
```

```
                        X(3:n+1) + H_vec(3:n+1));     % x_{i+1} + h_{i+1}

    % F(++h) - F(+hi) - F(+hj) + F0
    f_a = F_a - Fp_curr(1:n-1) - Fp_prev(1:n-1) + F0(1:n-1);


    % TERM b: x_i is current, x_{i+1} is next
    F_b = calc_F(X(2:n) + H_vec(2:n), ...      % x_i + h_i
                        X(3:n+1) + H_vec(3:n+1), ...  % x_{i+1} + h_{i+1}
                        X(4:n+2));

    % F(++h) - F(+hi) - F(+hj) + F0
    f_b = F_b - Fp_next(2:n) - Fp_curr(2:n) + F0(2:n);

    diag1 = (f_a + f_b) ./ h_h1;


    %% SECOND SUPERDIAGONAL: x_i and x_i+2 are perturbed
    h_h2 = H_vec(2:n-1) .* H_vec(4:n+1); % h_i * h_{i+2}

    F_c = calc_F(X(2:n-1) + H_vec(2:n-1), ...  % x_i + h_i
                        X(3:n), ...
                        X(4:n+1) + H_vec(4:n+1));     % x_{i+2} + h_{i+2}

    % F(++h) - F(+hi) - F(+hj) + F0
    f_c = F_c - Fp_next(2:n-1) - Fp_prev(2:n-1) + F0(2:n-1);

    diag2 = f_c ./ h_h2;


    % Assembling the Hessian
    H_fd = spdiags([ ...
        [diag2; 0; 0], ...          % -2
        [diag1; 0], ...             % -1
        diag0, ...                  %  0
        [0; diag1], ...             % +1
        [0; 0; diag2]], ...         % +2
        -2:2, n, n);

    % imposing symmetry to avoid numerical errors
    H_fd = (H_fd + H_fd') / 2;
end
```

Listing 8: Broyden Tridiagonal efficient finite differences implementation for the Hessian

7.4.2 Efficient implementation - Chained Freudenstein Roth / mode 2

```
function g_fd = fd_gradChained(x, k, mode)
    n = length(x);
    base_h = 10^(-k);

    %creating vector h according to the mode (fixed/adaptive)
    if strcmp(mode, 'fixed')
        h = ones(n, 1) * base_h;
    else
        h = base_h * max(abs(x), 1e-8);
        h(h < eps) = base_h; % if x_i is zero, h_i would be zero

    end

    % Defining F, it depends by x_current and x_next
    calc_F = @(x, x_next) ...
        0.5 * ( ...
            (x + x_next .* ((5 - x_next) .* x_next - 2) - 13).^2 + ...     % f_odd^2
            (x + x_next .* ((1 + x_next) .* x_next - 14) - 29).^2 ...      % f_even
    ^2
        );

    % Base (non perturbed terms)
    F_base = calc_F(x(1:n-1), x(2:n));

    g_fd = zeros(n, 1);
```

```
25
26      % term a: x_i is c_curr
27      h_curr = h(1:n-1);
28      F_pert_curr = calc_F(x(1:n-1) + h_curr, x(2:n));
29
30      diff_curr = (F_pert_curr - F_base) ./ h_curr;
31      g_fd(1:n-1) = g_fd(1:n-1) + diff_curr;
32
33      % term b: x_i is x_next
34      h_next = h(2:n);
35      F_pert_next = calc_F(x(1:n-1), x(2:n) + h_next);
36
37      diff_next = (F_pert_next - F_base) ./ h_next;
38
39      g_fd(2:n) = g_fd(2:n) + diff_next;    % Assembling the gradient
40
41
42  end
```

Listing 9: Chained Freudenstein-Roth efficient finite differences implementation for the gradient

```
function H_fd = fd_HessChained(x, k, mode)
    n = length(x);
    base_h = 10^(-k);

   %creating vector h according to the mode (fixed/adaptive)
    if strcmp(mode, 'fixed')
        h = ones(n, 1) * base_h;
    else
        h = base_h * max(abs(x), 1e-8);
        h(h < eps) = base_h; % if x_i is zero, h_i would be zero

    end

    h = sqrt(h); % Step for the Hessian

    % Defining function handles
    calc_F = @(x, x_next) ...
        0.5 * ( ...
            (x + x_next .* ((5 - x_next) .* x_next - 2) - 13).^2 + ...
            (x + x_next .* ((1 + x_next) .* x_next - 14) - 29).^2 ...
        );

    % Base
    F0 = calc_F(x(1:n-1), x(2:n));

    %%  MAIN DIAGONAL
    h_next = h(2:n);
    h_curr = h(1:n-1);

    % Forward (x + h)
    Fp_curr = calc_F(x(1:n-1) + h_curr, x(2:n));
    Fp_next = calc_F(x(1:n-1), x(2:n) + h_next);

    % Backward (x - h)
    Fm_next = calc_F(x(1:n-1), x(2:n) - h_next);
    Fm_curr = calc_F(x(1:n-1) - h_curr, x(2:n));

    % Central Difference: (F(x+h) - 2F(x) + F(x-h)) / h^2
    D2_curr = (Fp_curr - 2*F0 + Fm_curr) ./ (h_curr.^2);
    D2_next = (Fp_next - 2*F0 + Fm_next) ./ (h_next.^2);

    % Assembling the terms for the main diagonal
    diag0 = zeros(n, 1);
    diag0(1:n-1) = diag0(1:n-1) + D2_curr;
    diag0(2:n)   = diag0(2:n)   + D2_next;

    %% SUPERDIAGONAL
    % x_prev=x_i, x_next=x_{i+1}

    % F(x+h, x_next+h)
    F_cross = calc_F(x(1:n-1) + h_curr, x(2:n) + h_next);

    % (F(++h) - F(+h_x) - F(+h_xnext) + F0) / (h_x * h_xnext)
    num = F_cross - Fp_curr - Fp_next + F0;
    den = h_curr .* h_next;

    diag1 = num ./ den;

    % Assembling the Hessian
    H_fd = spdiags([ ...
        [0; diag1], ...     % -1
        diag0, ...          %  0
        [diag1; 0]], ...    % +1
        -1:1, n, n);

    % imposing symmetry to avoid numerical errors
    H_fd = (H_fd + H_fd') / 2;

end
```

Listing 10: Chained Freudenstein-Roth efficient finite differences implementation for the Hessian

7.5 Main

```
clear;
clc;

%to import Matlab functions from different folders
root = fileparts(mfilename('fullpath'));

addpath(genpath(fullfile(root, 'newton_methods')));
addpath(genpath(fullfile(root, 'derivatives')));
addpath(genpath(fullfile(root, 'functions')));
addpath(genpath(fullfile(root, 'data')));

%to store results in "results.txt"
fname = "results.txt";
if exist(fname, 'file'); delete(fname); end
diary(fname)

%setting a random seed for reproducibility
students_id = [359693; 362974];
rng(min(students_id));

%variables initialization
n = [2, 1e3, 1e4, 1e5];
num_random_points = 5;
k_vals = [4,8,12];
modes = {'fixed','adaptive'};

get_approx_derivatives;  %to import function handles
get_functions;

for i=1:length(n)

    %STARTING POINTS FOR CHAINED FREUDSTEIN ROTH FUNCTION

    %x_0 suggested in the pdf
    x0_chained_part = 0.5*ones(n(i)-1,1);
    x0_chained_sugg = [x0_chained_part; -2];
    start_points_chained = zeros(n(i), num_random_points+1);
    start_points_chained(:, 1) = x0_chained_sugg;

    %generating other starting points in [-1,1] hypercube
    for j = 1:num_random_points
        perturbation = 2 * rand(n(i), 1) - 1;
        start_points_chained(:, j+1) = x0_chained_sugg + perturbation;
    end

    %STARTING POINTS FOR BROYDEN TRIDIAGONAL FUNCTION

    %x_0 suggested in the pdf
    x0_broyden_sugg = -ones(n(i),1);
    start_points_broyden = zeros(n(i), num_random_points + 1);
    start_points_broyden(:, 1) = x0_broyden_sugg;

    %generating other starting points in [-1,1] hypercube
    for j = 1:num_random_points
        perturbation = 2 * rand(n(i), 1) - 1;
        start_points_broyden(:, j+1) = x0_broyden_sugg + perturbation;
    end


    % Parameters setting for Newton methods
    c1 = 1e-4;
    kmax = 1500;
    btmax = 50;
    jmax = 80;

    rho_Cmn = 0.5;
    rho_Ctn = 0.5;
    rho_Bmn = 0.1;
    rho_Btn = 0.5;

    %Scaling tollerance w.r.t. n
```

```
    if n(i) == 2
        tolgrad = 1e-8;
    else
        raw_tol = 1e-8 * sqrt(n(i));
        tolgrad = round(raw_tol, 1, 'significant');
    end

    fprintf("\n=== ANALISI PER n = %d | tolgrad = %2.e===\n", n(i), tolgrad)

    for ik = 1:length(k_vals)
        k = k_vals(ik);

        %Scaling tollerances w.r.t. k
        if k==4
            base_tol = 1e-4;
        elseif k==8
            base_tol = 1e-6;
        elseif k==12
            base_tol = 1e-8;
        end
        row_tol_k = base_tol * sqrt(n(i));

        tolgrad_k = round(row_tol_k,1, 'significant');

        %iteration to evaulate derivatives in 'fixed' and 'adaptive' mode
        for im = 1:length(modes)
            mode = modes{im};

            fprintf('\n==============================\n');
            fprintf('k = %d | mode = %s\n', k, mode);
            fprintf('==============================\n');

            % Uploading approx function handles
            filename = fullfile(root, 'data', ...
            sprintf('dati_approx_k%d_%s.mat', k, mode));
            load(filename);


            %INITIALATING VARIABLES TO COUNT SUCCESSES

            %Chained - modified newton
            successi_Cmn = 0;
            iter_Cmn =0;
            successi_Cmn_a1 = 0;
            iter_Cmn_a1 = 0;
            successi_Cmn_a2 = 0;
            iter_Cmn_a2 = 0;

            %Chained - truncated newton
            successi_Ctn = 0;
            iter_Ctn = 0;
            successi_Ctn_a1 = 0;
            iter_Ctn_a1=0;
            successi_Ctn_a2 = 0;
            iter_Ctn_a2=0;

            %Broyden - modified newton
            successi_Bmn = 0;
            iter_Bmn =0;
            successi_Bmn_a1 = 0;
            iter_Bmn_a1 = 0;
            successi_Bmn_a2 = 0;
            iter_Bmn_a2 = 0;

            %Chained - truncated newton
            successi_Btn = 0;
            iter_Btn = 0;
            successi_Btn_a1 = 0;
            iter_Btn_a1=0;
            successi_Btn_a2 = 0;
            iter_Btn_a2=0;

        %% Evaluating Chained Freudstein Roth
```

```
145         fprintf("\n--CHAINED FREUDSTEIN ROTH
    ----------------------------------------\n")
146
147         %Evaluation of the optimization methods for each starting point
148         for j = 1:size(start_points_chained, 2)
149                 fprintf("STARTING POINT %d\n",j);
150                 fprintf("\n--Modified newton--\n")
151                 x0 = start_points_chained(:, j);
152
153                 %Modified Newton with exact derivatives
154                 t_start = tic;
155                 [xk_Cmn, fk_Cmn, k_Cmn, gradfk_norm_Cmn, xseq_Cmn, btseq_Cmn] =
    MN_backtrack(x0, f1, gradf1, Hessf1, c1, rho_Cmn, kmax, btmax, jmax, tolgrad);
156                  x_best_ch = xseq_Cmn(:, k_Cmn);
157                 p_exp = compute_roc(xseq_Cmn, x_best_ch);  %evaluating rate of
    convergence
158
159                 %printing statistics
160                 if gradfk_norm_Cmn < tolgrad
161                     fprintf('|MN Chained Exact derivatives | F: %.4e | ||g||: %.4e
    | Iters: %d | tolgrad = %2.e | ROC: %.4f | time: %.4f | CONVERGENCE \n', fk_Cmn
    , gradfk_norm_Cmn, k_Cmn, tolgrad, p_exp, toc(t_start));
162                     successi_Cmn = successi_Cmn +1;
163                     iter_Cmn = iter_Cmn + k_Cmn;
164                 else
165                     fprintf('|MN Chained Exact derivatives | F: %.4e | ||g||: %.4e
    | Iters: %d | tolgrad = %2.e | ROC: N/A | time: %.4f| NO  CONVERGENCE \n',
    fk_Cmn, gradfk_norm_Cmn, k_Cmn, tolgrad, toc(t_start));
166                 end
167
168                 %Modified Newton with derivatives approximated using exact gradient
169                 t_start = tic;
170                 [xk_Cmn_a1, fk_Cmn_a1, k_Cmn_a1, gradfk_norm_Cmn_a1, xseq_Cmn_a1,
    btseq_Cmn_a1] = MN_backtrack(x0, f1, gradf1, Hessf1_approx1, c1, rho_Cmn, kmax,
     btmax, jmax, tolgrad_k);
171                 p_exp = compute_roc(xseq_Cmn_a1, x_best_ch);
172
173                 % printing results
174                 if gradfk_norm_Cmn_a1 < tolgrad_k
175                     fprintf('|MN Chained Approx Hess, exact. Grad | F: %.4e | ||g
    ||: %.4e | Iters: %d | tolgrad = %2.e | ROC: %.4f  | time: %.4f | CONVERGENCE\n
    ', fk_Cmn_a1, gradfk_norm_Cmn_a1, k_Cmn_a1, tolgrad_k, p_exp, toc(t_start));
176                     successi_Cmn_a1 = successi_Cmn_a1 +1;
177                     iter_Cmn_a1 = iter_Cmn_a1 + k_Cmn_a1;
178                 else
179                     fprintf('|MN Chained Approx Hess, exact. Grad | F: %.4e | ||g
    ||: %.4e | Iters: %d | tolgrad = %2.e | ROC: N/A | time: %.4f | NO CONVERGENCE\
    n', fk_Cmn_a1, gradfk_norm_Cmn_a1, k_Cmn_a1, tolgrad_k, toc(t_start));
180                 end
181                 %
182                 %Modified Newton with all derivatives approimated
183                 t_start = tic;
184                 [xk_Cmn_a2, fk_Cmn_a2, k_Cmn_a2, gradfk_norm_Cmn_a2, xseq_Cmn_a2,
    btseq_Cmn_a2] = MN_backtrack(x0, f1, gradf1_approx2, Hessf1_approx2, c1,
    rho_Cmn, kmax, btmax, jmax, tolgrad_k);
185                 p_exp = compute_roc(xseq_Cmn_a2, x_best_ch);
186
187                  % printing results
188                 if gradfk_norm_Cmn_a2 < tolgrad_k
189                     fprintf('|MN Chained Approx all derivatives | F: %.4e | ||g||:
    %.4e | Iters: %d | tolgrad = %2.e | ROC: %.4f  | time: %.4f | CONVERGENCE\n',
    fk_Cmn_a2, gradfk_norm_Cmn_a2, k_Cmn_a2, tolgrad_k, p_exp, toc(t_start));
190                     successi_Cmn_a2 = successi_Cmn_a2 +1;
191                     iter_Cmn_a2 = iter_Cmn_a2 + k_Cmn_a2;
192                 else
193                     fprintf('|MN Chained Approx all derivatives | F: %.4e | ||g||:
    %.4e | Iters: %d | tolgrad = %2.e | ROC: N/A | time: %.4f | NO CONVERGENCE\n',
    fk_Cmn_a2, gradfk_norm_Cmn_a2, k_Cmn_a2, tolgrad_k, toc(t_start));
194                 end
195                 %
196                 fprintf("--Truncated newton--\n")
197                 t_start = tic;
198
```

```
199                 %Truncated Newton with exact derivatives
200                 [xk_Ctn, fk_Ctn, k_Ctn, gradfk_norm_Ctn, xseq_Ctn, btseq_Ctn] =
    truncated_newton(x0, f1, gradf1, Hessf1, c1, rho_Ctn, kmax, btmax, tolgrad);
201                 p_exp = compute_roc(xseq_Ctn, x_best_ch);
202
203                 % printing results
204                 if gradfk_norm_Ctn < tolgrad
205                     fprintf('|TN Chained Exact derivatives | F: %.4e | ||g||: %.4e
    | Iters: %d | tolgrad = %2.e | roc: %.4f  | time: %.4f | CONVERGENCE \n',
    fk_Ctn, gradfk_norm_Ctn, k_Ctn, tolgrad, p_exp, toc(t_start));
206                     successi_Ctn = successi_Ctn +1;
207                     iter_Ctn = iter_Ctn + k_Ctn;
208                 else
209                     fprintf('|TN Chained Exact derivatives | F: %.4e | ||g||: %.4e
    | Iters: %d | tolgrad = %2.e | ROC: N/A | time: %.4f |NO CONVERGENCE \n',
    fk_Ctn, gradfk_norm_Ctn, k_Ctn, tolgrad, toc(t_start));
210                 end
211
212
213                 %Truncated Newton with derivatives approximated using exact
    gradient
214                 t_start = tic;
215                 [xk_Ctn_a1, fk_Ctn_a1, k_Ctn_a1, gradfk_norm_Ctn_a1, xseq_Ctn_a1,
    btseq_Ctn_a1] = truncated_newton(x0, f1, gradf1, Hessf1_approx1, c1, rho_Ctn,
    kmax, btmax, tolgrad_k);
216                 p_exp = compute_roc(xseq_Ctn_a1, x_best_ch);
217
218                 % printing results
219                 if gradfk_norm_Ctn_a1 < tolgrad_k
220                     fprintf('|TN Chained Approx Hess, exact. Grad | F: %.4e | ||g
    ||: %.4e | Iters: %d | tolgrad = %2.e | roc: %.4f  | time: %.4f | CONVERGENCE\n
    ', fk_Ctn_a1, gradfk_norm_Ctn_a1, k_Ctn_a1, tolgrad_k, p_exp, toc(t_start));
221                     successi_Ctn_a1 = successi_Ctn_a1 +1;
222                     iter_Ctn_a1 = iter_Ctn_a1 + k_Ctn_a1;
223                 else
224                     fprintf('|TN Chained Approx Hess, exact. Grad | F: %.4e | ||g
    ||: %.4e | Iters: %d | tolgrad = %2.e | ROC: N/A | time: %.4f | NO CONVERGENCE\
    n', fk_Ctn_a1, gradfk_norm_Ctn_a1, k_Ctn_a1, tolgrad_k, toc(t_start));
225                 end
226
227                 %Truncated Newton with all derivatives approimated
228                 t_start = tic;
229                 [xk_Ctn_a2, fk_Ctn_a2, k_Ctn_a2, gradfk_norm_Ctn_a2, xseq_Ctn_a2,
    btseq_Ctn_a2] = truncated_newton(x0, f1, gradf1_approx2, Hessf1_approx2, c1,
    rho_Ctn, kmax, btmax, tolgrad_k);
230                 p_exp = compute_roc(xseq_Ctn_a2, x_best_ch);
231
232                 % printing results
233                 if gradfk_norm_Ctn_a2 < tolgrad_k
234                     fprintf('|TN Chained Approx all derivatives | F: %.4e | ||g||:
    %.4e | Iters: %d | tolgrad = %2.e | roc: %.4f  | time: %.4f | CONVERGENCE\n',
    fk_Ctn_a2, gradfk_norm_Ctn_a2, k_Ctn_a2, tolgrad_k, p_exp, toc(t_start));
235                     successi_Ctn_a2 = successi_Ctn_a2 +1;
236                     iter_Ctn_a2 = iter_Ctn_a2 + k_Ctn_a2;
237                 else
238                     fprintf('|TN Chained Approx all derivatives | F: %.4e | ||g||:
    %.4e | Iters: %d | tolgrad = %2.e | ROC: N/A | time: %.4f | NO CONVERGENCE\n',
    fk_Ctn_a2, gradfk_norm_Ctn_a2, k_Ctn_a2, tolgrad_k, toc(t_start));
239                 end
240
241
242             end
243
244             %Printing overall statistics
245             if successi_Cmn > 0
246                 iter_Cmn = iter_Cmn/successi_Cmn;
247                 fprintf('\nMN Chained Exact derivatives  | Punti convergenti: %d/%d |
    Iterazioni medie (sui successi): %.1f\n', ...
248                 successi_Cmn, size(start_points_chained, 2), iter_Cmn);
249             else
250                 fprintf('\nMN Chained Exact derivatives  | Punti convergenti: 0/%d\n',
    ...
251                 size(start_points_chained, 2));
```

```
252         end
253         if successi_Cmn_a1 >0
254             iter_Cmn_a1 = iter_Cmn_a1/successi_Cmn_a1;
255             fprintf('\nMN Chained Approx Hess, exact. Grad | Punti convergenti: %d
    /%d | Iterazioni medie (sui successi): %.1f\n', ...
256                 successi_Cmn_a1, size(start_points_chained, 2), iter_Cmn_a1);
257         else
258             fprintf('\nMN Chained Approx Hess, exact. Grad | Punti convergenti: 0/%
    d\n', ...
259                 size(start_points_chained, 2));
260         end
261         if successi_Cmn_a2 >0
262             iter_Cmn_a2 = iter_Cmn_a2/successi_Cmn_a2;
263             fprintf('\nMN Chained Approx all derivatives | Punti convergenti: %d/%d
     | Iterazioni medie (sui successi): %.1f\n', ...
264                 successi_Cmn_a2, size(start_points_chained, 2), iter_Cmn_a2);
265         else
266             fprintf('\nMN Chained Approx all derivatives | Punti convergenti: 0/%d\
    n', ...
267                 size(start_points_chained, 2));
268         end
269
270
271         if successi_Ctn > 0
272             iter_Ctn = iter_Ctn/successi_Ctn;
273             fprintf('\nTN Chained Exact derivatives  | Punti convergenti: %d/%d |
    Iterazioni medie (sui successi): %.1f\n', ...
274                 successi_Ctn, size(start_points_chained, 2), iter_Ctn);
275         else
276             fprintf('\nTN Chained Exact derivatives  | Punti convergenti: 0/%d\n',
    ...
277                 size(start_points_chained, 2));
278         end
279         if successi_Ctn_a1 >0
280             iter_Ctn_a1 = iter_Ctn_a1/successi_Ctn_a1;
281             fprintf('\nTN Chained Approx. Hess, exact. Grad | Punti convergenti: %d
    /%d | Iterazioni medie (sui successi): %.1f\n', ...
282                 successi_Ctn_a1, size(start_points_chained, 2), iter_Ctn_a1);
283         else
284             fprintf('\nTN Chained Approx. Hess, exact. Grad | Punti convergenti:
    0/%d\n', ...
285                 size(start_points_chained, 2));
286         end
287         if successi_Ctn_a2 >0
288             iter_Ctn_a2 = iter_Ctn_a2/successi_Ctn_a2;
289             fprintf('\nTN Chained Approx all derivatives | Punti convergenti: %d/%d
     | Iterazioni medie (sui successi): %.1f\n', ...
290                 successi_Ctn_a2, size(start_points_chained, 2), iter_Ctn_a2);
291         else
292             fprintf('\nTN Chained Approx all derivatives | Punti convergenti: 0/%d\
    n', ...
293                 size(start_points_chained, 2));
294         end
295
296
297        %% DOING THE SAME FOR BROYDEN FUNCTION
298
299         fprintf("\n--BROYDEN TRIDIAGONAL----------------------------------------\n")
300         for j = 1:size(start_points_broyden, 2)
301             x0 = start_points_broyden(:, j);
302             fprintf("STARTING POINT %d: [%.2f; %.2f]\n",j, start_points_broyden
    (1, j), start_points_broyden(2, j));
303             fprintf("--Modified newton--\n")
304
305             %Modified Newton with exact derivatives
306             t_start = tic;
307             [xk_Bmn, fk_Bmn, k_Bmn, gradfk_norm_Bmn, xseq_Bmn, btseq_Bmn] =
    MN_backtrack(x0, f2, gradf2, Hessf2, c1, rho_Bmn, kmax, btmax, jmax, tolgrad);
308             x_best_br = xseq_Bmn(:, k_Bmn);
309             p_exp = compute_roc(xseq_Bmn, x_best_br);
310
311             % printing results
312             if gradfk_norm_Bmn < tolgrad
```

```
                    fprintf('|MN Broyden Exact derivatives | F: %.4e | ||g||: %.4e | Iters: %d | tolgrad = %2.e | roc: %.4f  | time: %.4f | CONVERGENCE \n', fk_Bmn, gradfk_norm_Bmn, k_Bmn, tolgrad, p_exp, toc(t_start));
                    successi_Bmn = successi_Bmn +1;
                    iter_Bmn = iter_Bmn + k_Bmn;
                else
                    fprintf('|MN Broyden Exact derivatives | F: %.4e | ||g||: %.4e | Iters: %d | tolgrad = %2.e | ROC: N/A | time: %.4f | NO CONVERGENCE \n', fk_Bmn, gradfk_norm_Bmn, k_Bmn, tolgrad, toc(t_start));

                end

                %Modified Newton with derivatives approximated using exact gradient
                t_start = tic;
                [xk_Bmn_a1, fk_Bmn_a1, k_Bmn_a1, gradfk_norm_Bmn_a1, xseq_Bmn_a1, btseq_Bmn_a1] = MN_backtrack(x0, f2, gradf2, Hessf2_approx1, c1, rho_Bmn, kmax, btmax, jmax, tolgrad_k);
                p_exp = compute_roc(xseq_Bmn_a1, x_best_br);

                % printing results
                if gradfk_norm_Bmn_a1 < tolgrad_k
                    fprintf('|MN Broyden Approx Hess, exact. Grad | F: %.4e | ||g||: %.4e | Iters: %d | tolgrad = %2.e | roc: %.4f  | time: %.4f | CONVERGENCE\n', fk_Bmn_a1, gradfk_norm_Bmn_a1, k_Bmn_a1, tolgrad_k, p_exp, toc(t_start));
                    successi_Bmn_a1 = successi_Bmn_a1 +1;
                    iter_Bmn_a1 = iter_Bmn_a1 +k_Bmn_a1;
                else
                    fprintf('|MN Broyden Approx Hess, exact. Grad | F: %.4e | ||g||: %.4e | Iters: %d | tolgrad = %2.e | ROC: N/A | time: %.4f | NO CONVERGENCE\n', fk_Bmn_a1, gradfk_norm_Bmn_a1, k_Bmn_a1, tolgrad_k, toc(t_start));
                end


                %Modified Newton with all derivatives approximated
                t_start = tic;
                [xk_Bmn_a2, fk_Bmn_a2, k_Bmn_a2, gradfk_norm_Bmn_a2, xseq_Bmn_a2, btseq_Bmn_a2] = MN_backtrack(x0, f2, gradf2_approx2, Hessf2_approx2, c1, rho_Bmn, kmax, btmax, jmax, tolgrad_k);
                p_exp = compute_roc(xseq_Bmn_a2, x_best_br);

                % printing results
                if gradfk_norm_Bmn_a2 < tolgrad_k
                    fprintf('|MN Broyden Approx all derivatives | F: %.4e | ||g||: %.4e | Iters: %d | tolgrad = %2.e | roc: %.4f  | time: %.4f | CONVERGENCE\n', fk_Bmn_a2, gradfk_norm_Bmn_a2, k_Bmn_a2, tolgrad_k, p_exp, toc(t_start));
                    successi_Bmn_a2 = successi_Bmn_a2 +1;
                    iter_Bmn_a2 = iter_Bmn_a2 + k_Bmn_a2;
                else
                    fprintf('|MN Broyden Approx all derivatives | F: %.4e | ||g||: %.4e | Iters: %d | tolgrad = %2.e | ROC: N/A | time: %.4f | NO CONVERGENCE\n', fk_Bmn_a2, gradfk_norm_Bmn_a2, k_Bmn_a2, tolgrad_k, toc(t_start));
                end

                fprintf("--Truncated newton--\n")

                %Truncated Newton with exact derivatives
                t_start = tic;
                [xk_Btn, fk_Btn, k_Btn, gradfk_norm_Btn, xseq_Btn, btseq_Btn] = truncated_newton(x0, f2, gradf2, Hessf2, c1, rho_Btn, kmax, btmax, tolgrad);
                p_exp = compute_roc(xseq_Btn, x_best_br);

                 % printing results
                if gradfk_norm_Btn < tolgrad
                    fprintf('|TN Broyden Exact derivatives | F: %.4e | ||g||: %.4e | Iters: %d | tolgrad = %2.e | roc: %.4f  | time: %.4f | CONVERGENCE \n', fk_Btn, gradfk_norm_Btn, k_Btn, tolgrad, p_exp, toc(t_start));
                    successi_Btn = successi_Btn +1;
                    iter_Btn = iter_Btn + k_Btn;
                else
                    fprintf('|TN Broyden Exact derivatives | F: %.4e | ||g||: %.4e | Iters: %d | tolgrad = %2.e | ROC: N/A | time: %.4f | NO CONVERGENCE \n', fk_Btn, gradfk_norm_Btn, k_Btn, tolgrad, toc(t_start));
                end
```

```
365
366
367             %Truncated Newton with derivatives approximated using exact
    gradient
368             t_start = tic;
369             [xk_Btn_a1, fk_Btn_a1, k_Btn_a1, gradfk_norm_Btn_a1, xseq_Btn_a1,
    btseq_Btn_a1] = truncated_newton(x0, f2, gradf2, Hessf2_approx1, c1, rho_Btn,
    kmax, btmax, tolgrad_k);
370             p_exp = compute_roc(xseq_Btn_a1, x_best_br);
371
372             % printing results
373             if gradfk_norm_Btn_a1 < tolgrad_k
374                 fprintf('|TN Broyden Approx Hess, exact. Grad | F: %.4e | ||g
    ||: %.4e | Iters: %d | tolgrad = %2.e | roc: %.4f | time: %.4f | CONVERGENCE\n'
    , fk_Btn_a1, gradfk_norm_Btn_a1, k_Btn_a1, tolgrad_k, p_exp, toc(t_start));
375                 successi_Btn_a1 = successi_Btn_a1 +1;
376                 iter_Btn_a1 = iter_Btn_a1 + k_Btn_a1;
377             else
378                 fprintf('|TN Broyden Approx Hess, exact. Grad | F: %.4e | ||g
    ||: %.4e | Iters: %d | tolgrad = %2.e | ROC: N/A | time: %.4f | NO CONVERGENCE\
    n', fk_Btn_a1, gradfk_norm_Btn_a1, k_Btn_a1, tolgrad_k, toc(t_start));
379             end
380
381
382             %Truncated Newton with all derivatives approimated
383             t_start = tic;
384             [xk_Btn_a2, fk_Btn_a2, k_Btn_a2, gradfk_norm_Btn_a2, xseq_Btn_a2,
    btseq_Btn_a2] = truncated_newton(x0, f2, gradf2_approx2, Hessf2_approx2, c1,
    rho_Btn, kmax, btmax, tolgrad_k);
385             p_exp = compute_roc(xseq_Btn_a2, x_best_br);
386
387             % printing results
388             if gradfk_norm_Btn_a2 < tolgrad_k
389                 fprintf('|TN Broyden Approx all derivatives | F: %.4e | ||g||:
    %.4e | Iters: %d | tolgrad = %2.e | roc: %.4f  | time: %.4f | CONVERGENCE\n',
    fk_Btn_a2, gradfk_norm_Btn_a2, k_Btn_a2, tolgrad_k, p_exp, toc(t_start));
390                 successi_Btn_a2 = successi_Btn_a2 +1;
391                 iter_Btn_a2 = iter_Btn_a2 + k_Btn_a2;
392             else
393                 fprintf('|TN Broyden Approx all derivatives | F: %.4e | ||g||:
    %.4e | Iters: %d | tolgrad = %2.e | ROC: N/A | time: %.4f | NO CONVERGENCE\n',
    fk_Btn_a2, gradfk_norm_Btn_a2, k_Btn_a2, tolgrad_k, toc(t_start));
394             end
395
396
397         end
398
399         % Printing overall statistics
400
401         % For Modified Newton
402         if successi_Bmn > 0
403             iter_Bmn = iter_Bmn/successi_Bmn;
404             fprintf('\nMN Broyden Exact derivatives  | Punti convergenti: %d/%d |
    Iterazioni medie (sui successi): %.1f\n', ...
405                 successi_Bmn, size(start_points_broyden, 2), iter_Bmn);
406         else
407             fprintf('\nMN Broyden Exact derivatives  | Punti convergenti: 0/%d\n',
    ...
408                 size(start_points_broyden, 2));
409         end
410         if successi_Bmn_a1 >0
411             iter_Bmn_a1 = iter_Bmn_a1/successi_Bmn_a1;
412             fprintf('\nMN Broyden Approx Hess, exact. Grad | Punti convergenti: %d
    /%d | Iterazioni medie (sui successi): %.1f\n', ...
413                 successi_Bmn_a1, size(start_points_broyden, 2), iter_Bmn_a1);
414         else
415             fprintf('\nMN Broyden Approx Hess, exact. Grad | Punti convergenti: 0/%
    d\n', ...
416                 size(start_points_broyden, 2));
417         end
418         if successi_Bmn_a2 >0
419             iter_Bmn_a2 = iter_Bmn_a2/successi_Bmn_a2;
420             fprintf('\nMN Broyden Approx all derivatives | Punti convergenti: %d/%d
```

```
     | Iterazioni medie (sui successi): %.1f\n', ...
421                     successi_Bmn_a2, size(start_points_broyden, 2), iter_Bmn_a2);
422         else
423             fprintf('\nMN Broyden Approx all derivatives | Punti convergenti: 0/%d\
    n', ...
424                     size(start_points_broyden, 2));
425         end
426
427         %For Truncated Newton
428         if successi_Btn > 0
429             iter_Btn = iter_Btn/successi_Btn;
430             fprintf('\nTN Bryden Exact derivatives  | Punti convergenti: %d/%d |
    Iterazioni medie (sui successi): %.1f\n', ...
431                 successi_Btn, size(start_points_broyden, 2), iter_Btn);
432         else
433             fprintf('\nTN Bryden Exact derivatives  | Punti convergenti: 0/%d\n',
    ...
434                 size(start_points_broyden, 2));
435         end
436         if successi_Btn_a1 >0
437             iter_Btn_a1 = iter_Btn_a1/successi_Btn_a1;
438             fprintf('\nTN Bryden Approx. Hess, exact. Grad | Punti convergenti: %d
    /%d | Iterazioni medie (sui successi): %.1f\n', ...
439                 successi_Btn_a1, size(start_points_broyden, 2), iter_Btn_a1);
440         else
441             fprintf('\nTN Chained Approx. Hess, exact. Grad | Punti convergenti:
    0/%d\n', ...
442                 size(start_points_broyden, 2));
443         end
444         if successi_Btn_a2 >0
445             iter_Btn_a2 = iter_Btn_a2/successi_Btn_a2;
446             fprintf('\nTN Bryden Approx all derivatives | Punti convergenti: %d/%d
    | Iterazioni medie (sui successi): %.1f\n', ...
447                 successi_Btn_a2, size(start_points_broyden, 2), iter_Btn_a2);
448         else
449             fprintf('\nTN Bryden Approx all derivatives | Punti convergenti: 0/%d\n
    ', ...
450                 size(start_points_broyden, 2));
451         end
452
453
454         end
455      end
456
457 end
458
459 diary off
```

Listing 11: Main

8 Tables

Analysis for Dimension $n = 2$

pt ID	grad	iter/max	succ	flag	rate	time
1	8.56e-14	10	yes	-	1.3216	0.2182s
2	7.75e-12	5	yes	-	1.8790	0.0051s
3	6.78e-09	8	yes	-	2.1361	0.0051s
4	5.19e-10	6	yes	-	1.8727	0.0026s
5	9.97e-14	7	yes	-	0.0629	0.0029s
6	1.42e-14	8	yes	-	2.0009	0.0032s
Avg	-	7.33	6/6	-	1.5455	0.0395s

Table 1: $n = 2$: Chained (MN) - Exact Derivatives

pt	k	mode	grad	iter/max	succ	flag	rate	time
1	4	adaptive	9.12e-05	109	yes	-	0.2376	0.0282s
2	4	adaptive	9.44e-05	108	yes	-	0.4776	0.0224s
3	4	adaptive	9.62e-05	115	yes	-	0.4753	0.0246s
4	4	adaptive	8.40e-05	107	yes	-	0.1886	0.0226s
5	4	adaptive	8.13e-05	111	yes	-	0.5065	0.0216s
6	4	adaptive	8.84e-05	117	yes	-	0.5504	0.0303s
1	4	fixed	9.11e-05	109	yes	-	0.2382	0.0751s
2	4	fixed	9.44e-05	108	yes	-	0.4786	0.0188s
3	4	fixed	9.61e-05	115	yes	-	0.4756	0.0318s
4	4	fixed	8.40e-05	107	yes	-	0.1892	0.0279s
5	4	fixed	8.12e-05	111	yes	-	0.5071	0.0258s
6	4	fixed	8.83e-05	117	yes	-	0.5508	0.0360s
1	8	adaptive	7.95e-07	145	yes	-	1.5374	0.0229s
2	8	adaptive	3.11e-08	144	yes	-	0.4363	0.0362s
3	8	adaptive	5.89e-07	147	yes	-	0.2930	0.0356s
4	8	adaptive	1.87e-07	152	yes	-	1.8178	0.0302s
5	8	adaptive	6.23e-08	152	yes	-	1.9563	0.0450s
6	8	adaptive	6.36e-07	147	yes	-	0.4524	0.0289s
1	8	fixed	7.11e-07	150	yes	-	0.0031	0.0385s
2	8	fixed	7.94e-07	145	yes	-	0.0050	0.0299s
3	8	fixed	7.11e-07	170	yes	-	2.0003	0.0413s
4	8	fixed	5.02e-07	150	yes	-	1.7942	0.0353s
5	8	fixed	3.55e-07	144	yes	-	0.1285	0.0287s
6	8	fixed	7.11e-07	146	yes	-	0.0041	0.0291s
1	12	adaptive	0.0	81	yes	-	0.6196	0.0130s
2	12	adaptive	0.0	71	yes	-	0.8798	0.0081s
3	12	adaptive	0.0	85	yes	-	0.7056	0.0090s
4	12	adaptive	0.0	87	yes	-	0.9686	0.0155s
5	12	adaptive	7.93e-03	223	no	fail	N/A	0.0766s
6	12	adaptive	1.58e-02	228	no	fail	N/A	0.0811s
1	12	fixed	1.91e-02	225	no	fail	N/A	0.0986s
2	12	fixed	2.27e-02	227	no	fail	N/A	0.1085s
3	12	fixed	2.51e-02	224	no	fail	N/A	0.0909s
4	12	fixed	0.0	84	yes	-	0.0001	0.0159s
5	12	fixed	5.73e-02	205	no	fail	N/A	0.0944s
6	12	fixed	6.79e-02	206	no	fail	N/A	0.0710s
Avg Success			-	121.86	29/36	-	0.6372	0.0286s

Table 2: $n = 2$: Chained (MN) - Finite Differences (With exact gradient)

pt	k	mode	grad	iter/max	succ	flag	rate	time
1	4	adaptive	9.12e-05	109	yes	-	0.2376	0.0282s
2	4	adaptive	9.44e-05	108	yes	-	0.4776	0.0224s
3	4	adaptive	9.62e-05	115	yes	-	0.4753	0.0246s
4	4	adaptive	8.40e-05	107	yes	-	0.1886	0.0226s
5	4	adaptive	8.13e-05	111	yes	-	0.5065	0.0216s
6	4	adaptive	8.84e-05	117	yes	-	0.5504	0.0303s
1	4	fixed	9.11e-05	109	yes	-	0.2382	0.0751s
2	4	fixed	9.44e-05	108	yes	-	0.4786	0.0188s
3	4	fixed	9.61e-05	115	yes	-	0.4756	0.0318s
4	4	fixed	8.40e-05	107	yes	-	0.1892	0.0279s
5	4	fixed	8.12e-05	111	yes	-	0.5071	0.0258s
6	4	fixed	8.83e-05	117	yes	-	0.5508	0.0360s
1	8	adaptive	7.95e-07	145	yes	-	1.5374	0.0229s
2	8	adaptive	3.11e-08	144	yes	-	0.4363	0.0362s
3	8	adaptive	5.89e-07	147	yes	-	0.2930	0.0356s
4	8	adaptive	1.87e-07	152	yes	-	1.3021	0.0302s
5	8	adaptive	6.23e-08	152	yes	-	1.7178	0.0450s
6	8	adaptive	6.36e-07	147	yes	-	0.4524	0.0289s
1	8	fixed	7.11e-07	150	yes	-	0.0031	0.0385s
2	8	fixed	7.94e-07	145	yes	-	0.0050	0.0299s
3	8	fixed	7.11e-07	170	yes	-	2.0003	0.0413s
4	8	fixed	5.02e-07	150	yes	-	0.0039	0.0353s
5	8	fixed	3.55e-07	144	yes	-	0.1285	0.0287s
6	8	fixed	7.11e-07	146	yes	-	0.0041	0.0291s
1	12	adaptive	0.0	81	yes	-	0.6196	0.0130s
2	12	adaptive	0.0	71	yes	-	0.9130	0.0081s
3	12	adaptive	0.0	85	yes	-	0.7056	0.0090s
4	12	adaptive	0.0	87	yes	-	0.5894	0.0155s
5	12	adaptive	7.93e-03	223	no	fail	N/A	0.0766s
6	12	adaptive	1.58e-02	228	no	fail	N/A	0.0811s
1	12	fixed	1.91e-02	225	no	fail	N/A	0.0986s
2	12	fixed	2.27e-02	227	no	fail	N/A	0.1085s
3	12	fixed	2.51e-02	224	no	fail	N/A	0.0909s
4	12	fixed	0.0	84	yes	-	0.8761	0.0159s
5	12	fixed	5.73e-02	205	no	fail	N/A	0.0944s
6	12	fixed	6.79e-02	206	no	fail	N/A	0.0710s
Avg Success			-	121.86	29/36	-	0.5677	0.0286s

Table 3: $n = 2$: Chained (MN) - Finite Differences (Approx All)

pt ID	grad	iter/max	succ	flag	rate	time
1	6.26e-13	9	yes	-	1.9368	0.0343s
2	1.34e-12	7	yes	-	1.8381	0.0032s
3	3.28e-13	9	yes	-	0.9942	0.0087s
4	2.71e-13	8	yes	-	1.2821	0.0010s
5	1.46e-14	7	yes	-	1.9459	0.0029s
6	0.0	10	yes	-	1.3437	0.0012s
Avg	-	8.33	6/6	-	1.5568	0.0086s

Table 4: $n = 2$: Chained (TN) - Exact Derivatives

pt	k	mode	grad	iter/max	succ	flag	rate	time
1	4	adaptive	9.90e-05	97	yes	-	0.0040	0.0043s
2	4	adaptive	9.38e-05	95	yes	-	0.0040	0.0081s
3	4	adaptive	9.91e-05	97	yes	-	0.0040	0.0042s
4	4	adaptive	9.96e-05	96	yes	-	0.0040	0.0060s
5	4	adaptive	9.96e-05	96	yes	-	0.0040	0.0072s
6	4	adaptive	8.38e-05	99	yes	-	0.0040	0.0106s
1	4	fixed	9.90e-05	97	yes	-	0.0044	0.0111s
2	4	fixed	9.37e-05	95	yes	-	0.0044	0.0131s
3	4	fixed	9.91e-05	97	yes	-	0.0044	0.0106s
4	4	fixed	9.96e-05	96	yes	-	0.0044	0.0085s
5	4	fixed	9.96e-05	96	yes	-	0.0044	0.0121s
6	4	fixed	8.38e-05	99	yes	-	0.0044	0.0041s
1	8	adaptive	1.56e-07	151	yes	-	0.8546	0.0100s
2	8	adaptive	9.70e-07	142	yes	-	0.8425	0.0072s
3	8	adaptive	8.18e-07	141	yes	-	0.0430	0.0056s
4	8	adaptive	8.14e-07	146	yes	-	0.1021	0.0119s
5	8	adaptive	4.38e-07	152	yes	-	2.0863	0.0120s
6	8	adaptive	9.04e-07	141	yes	-	0.2657	0.0096s
1	8	fixed	7.94e-07	150	yes	-	0.1465	0.0133s
2	8	fixed	2.84e-06	264	no	fail	N/A	0.0558s
3	8	fixed	0.0	150	yes	-	0.5207	0.0084s
4	8	fixed	3.55e-07	152	yes	-	0.0014	0.0123s
5	8	fixed	5.02e-07	143	yes	-	0.0048	0.0106s
6	8	fixed	7.11e-07	141	yes	-	0.0035	0.0045s
1	12	adaptive	0.0	84	yes	-	0.0012	0.0030s
2	12	adaptive	0.0	73	yes	-	0.0063	0.0037s
3	12	adaptive	0.0	75	yes	-	0.0051	0.0026s
4	12	adaptive	0.0	83	yes	-	0.0018	0.0027s
5	12	adaptive	2.38e-02	231	no	fail	N/A	0.1147s
6	12	adaptive	0.0	97	yes	-	0.0040	0.0114s
1	12	fixed	3.66e-02	219	no	fail	N/A	0.0682s
2	12	fixed	1.28e-02	214	no	fail	N/A	0.0685s
3	12	fixed	2.56e-02	208	no	fail	N/A	0.0622s
4	12	fixed	4.05e-02	217	no	fail	N/A	0.0599s
5	12	fixed	0.0	92	yes	-	0.0046	0.0041s
6	12	fixed	1.91e-02	218	no	fail	N/A	0.0667s
Avg Success			-	112.86	29/36	-	0.1705	0.0080s

Table 5: $n = 2$: Chained (TN) - Finite Differences (With exact gradient)

pt	k	mode	grad	iter/max	succ	flag	rate	time
1	4	adaptive	9.90e-05	97	yes	-	0.0040	0.0043s
2	4	adaptive	9.38e-05	95	yes	-	0.0040	0.0081s
3	4	adaptive	9.91e-05	97	yes	-	0.0040	0.0042s
4	4	adaptive	9.96e-05	96	yes	-	0.0040	0.0060s
5	4	adaptive	9.96e-05	96	yes	-	0.0040	0.0072s
6	4	adaptive	8.38e-05	99	yes	-	0.0040	0.0106s
1	4	fixed	9.90e-05	97	yes	-	0.0044	0.0111s
2	4	fixed	9.37e-05	95	yes	-	0.0044	0.0131s
3	4	fixed	9.91e-05	97	yes	-	0.0044	0.0106s
4	4	fixed	9.96e-05	96	yes	-	0.0044	0.0085s
5	4	fixed	9.96e-05	96	yes	-	0.0044	0.0121s
6	4	fixed	8.38e-05	99	yes	-	0.0044	0.0041s
1	8	adaptive	1.56e-07	151	yes	-	0.8546	0.0100s
2	8	adaptive	9.70e-07	142	yes	-	0.8425	0.0072s
3	8	adaptive	8.18e-07	141	yes	-	0.0430	0.0056s
4	8	adaptive	8.14e-07	146	yes	-	0.1021	0.0119s
5	8	adaptive	4.38e-07	152	yes	-	2.0863	0.0120s
6	8	adaptive	9.04e-07	141	yes	-	0.2657	0.0096s
1	8	fixed	7.94e-07	150	yes	-	0.1465	0.0133s
2	8	fixed	2.84e-06	264	no	fail	N/A	0.0558s
3	8	fixed	0.0	150	yes	-	0.5207	0.0084s
4	8	fixed	3.55e-07	152	yes	-	0.0014	0.0123s
5	8	fixed	5.02e-07	143	yes	-	0.0048	0.0106s
6	8	fixed	7.11e-07	141	yes	-	0.0035	0.0045s
1	12	adaptive	0.0	84	yes	-	0.0002	0.0030s
2	12	adaptive	0.0	73	yes	-	0.0063	0.0037s
3	12	adaptive	0.0	75	yes	-	0.0051	0.0026s
4	12	adaptive	0.0	83	yes	-	0.0042	0.0027s
5	12	adaptive	2.38e-02	231	no	fail	N/A	0.1147s
6	12	adaptive	0.0	97	yes	-	0.0060	0.0114s
1	12	fixed	3.66e-02	219	no	fail	N/A	0.0682s
2	12	fixed	1.28e-02	214	no	fail	N/A	0.0685s
3	12	fixed	2.56e-02	208	no	fail	N/A	0.0622s
4	12	fixed	4.05e-02	217	no	fail	N/A	0.0599s
5	12	fixed	0.0	92	yes	-	0.0069	0.0041s
6	12	fixed	1.91e-02	218	no	fail	N/A	0.0667s
Avg Success			-	112.86	29/36	-	0.1707	0.0080s

Table 6: $n = 2$: Chained (TN) - Finite Differences (Approx All)

pt ID	grad	iter/max	succ	flag	rate	time
1	2.61e-15	6	yes	-	1.9962	0.0426s
2	8.05e-10	7	yes	-	1.9748	0.0073s
3	6.75e-10	4	yes	-	2.0349	0.0033s
4	1.16e-15	7	yes	-	1.9978	0.0027s
5	5.89e-10	5	yes	-	1.9735	0.0032s
6	1.03e-15	6	yes	-	1.9987	0.0013s
Avg	-	5.83	6/6	-	1.9960	0.0101s

Table 7: $n = 2$: Broyden (MN) - Exact Derivatives

pt	k	mode	grad	iter/max	succ	flag	rate	time
1	4	adaptive	6.34e-06	5	yes	-	0.2950	0.0035s
2	4	adaptive	8.11e-07	7	yes	-	0.0711	0.0020s
3	4	adaptive	8.10e-07	5	yes	-	0.0710	0.0021s
4	4	adaptive	2.88e-06	6	yes	-	0.1777	0.0022s
5	4	adaptive	7.48e-07	5	yes	-	0.0668	0.0011s
6	4	adaptive	4.94e-06	5	yes	-	0.2539	0.0020s
1	4	fixed	1.08e-05	5	yes	-	0.2041	0.1829s
2	4	fixed	1.61e-06	7	yes	-	0.0523	0.0038s
3	4	fixed	7.21e-06	5	yes	-	0.1546	0.0058s
4	4	fixed	5.19e-06	6	yes	-	0.1225	0.0029s
5	4	fixed	1.50e-06	5	yes	-	0.0498	0.0100s
6	4	fixed	8.53e-06	5	yes	-	0.1740	0.0013s
1	8	adaptive	1.72e-07	5	yes	-	1.8608	0.0015s
2	8	adaptive	3.68e-09	7	yes	-	1.2049	0.0014s
3	8	adaptive	4.04e-09	4	yes	-	1.2240	0.0015s
4	8	adaptive	3.53e-08	6	yes	-	1.6597	0.0017s
5	8	adaptive	3.23e-09	5	yes	-	1.1721	0.0018s
6	8	adaptive	1.16e-07	5	yes	-	1.8233	0.0014s
1	8	fixed	1.93e-07	5	yes	-	1.7719	0.0014s
2	8	fixed	5.26e-09	7	yes	-	1.0589	0.0017s
3	8	fixed	8.85e-09	4	yes	-	1.1753	0.0032s
4	8	fixed	4.35e-08	6	yes	-	1.5205	0.0019s
5	8	fixed	4.64e-09	5	yes	-	1.0278	0.0017s
6	8	fixed	1.31e-07	5	yes	-	1.7177	0.0038s
1	12	adaptive	1.18e-11	6	yes	-	1.1333	0.0034s
2	12	adaptive	1.35e-12	8	yes	-	0.3501	0.0014s
3	12	adaptive	2.46e-11	5	yes	-	0.6106	0.0010s
4	12	adaptive	4.51e-12	7	yes	-	0.8240	0.0015s
5	12	adaptive	1.67e-12	6	yes	-	0.3257	0.0018s
6	12	adaptive	9.24e-12	6	yes	-	1.1333	0.0012s
1	12	fixed	3.65e-12	6	yes	-	1.0077	0.0012s
2	12	fixed	7.81e-09	7	yes	-	1.6150	0.0011s
3	12	fixed	6.48e-09	4	yes	-	1.7407	0.0013s
4	12	fixed	4.41e-12	7	yes	-	0.8940	0.0041s
5	12	fixed	6.49e-09	5	yes	-	1.6080	0.0010s
6	12	fixed	5.54e-12	6	yes	-	0.9218	0.0026s
Avg Success			-	5.64	36/36	-	0.8632	0.0073s

Table 8: $n = 2$: Broyden (MN) - Finite Differences (With exact gradient)

pt	k	mode	grad	iter/max	succ	flag	rate	time
1	4	adaptive	6.34e-06	5	yes	-	0.2950	0.0035s
2	4	adaptive	8.11e-07	7	yes	-	0.0711	0.0020s
3	4	adaptive	8.10e-07	5	yes	-	0.0710	0.0021s
4	4	adaptive	2.88e-06	6	yes	-	0.1777	0.0022s
5	4	adaptive	7.48e-07	5	yes	-	0.0668	0.0011s
6	4	adaptive	4.94e-06	5	yes	-	0.2539	0.0020s
1	4	fixed	1.08e-05	5	yes	-	0.2041	0.1829s
2	4	fixed	1.61e-06	7	yes	-	0.0523	0.0038s
3	4	fixed	7.21e-06	5	yes	-	0.1546	0.0058s
4	4	fixed	5.19e-06	6	yes	-	0.1225	0.0029s
5	4	fixed	1.50e-06	5	yes	-	0.0498	0.0100s
6	4	fixed	8.53e-06	5	yes	-	0.1740	0.0013s
1	8	adaptive	1.72e-07	5	yes	-	1.8608	0.0015s
2	8	adaptive	3.68e-09	7	yes	-	1.2049	0.0014s
3	8	adaptive	4.04e-09	4	yes	-	1.2240	0.0015s
4	8	adaptive	3.53e-08	6	yes	-	1.6597	0.0017s
5	8	adaptive	3.23e-09	5	yes	-	1.1721	0.0018s
6	8	adaptive	1.16e-07	5	yes	-	1.8233	0.0014s
1	8	fixed	1.93e-07	5	yes	-	1.7719	0.0014s
2	8	fixed	5.26e-09	7	yes	-	1.0589	0.0017s
3	8	fixed	8.85e-09	4	yes	-	1.1753	0.0032s
4	8	fixed	4.35e-08	6	yes	-	1.5205	0.0019s
5	8	fixed	4.64e-09	5	yes	-	1.0278	0.0017s
6	8	fixed	1.31e-07	5	yes	-	1.7177	0.0038s
1	12	adaptive	1.18e-11	6	yes	-	1.1333	0.0034s
2	12	adaptive	1.35e-12	8	yes	-	0.3501	0.0014s
3	12	adaptive	2.46e-11	5	yes	-	0.6106	0.0010s
4	12	adaptive	4.51e-12	7	yes	-	0.8240	0.0015s
5	12	adaptive	1.67e-12	6	yes	-	0.3257	0.0018s
6	12	adaptive	9.24e-12	6	yes	-	1.1333	0.0012s
1	12	fixed	3.65e-12	6	yes	-	1.0077	0.0012s
2	12	fixed	7.81e-09	7	yes	-	1.6150	0.0011s
3	12	fixed	6.48e-09	4	yes	-	1.7407	0.0013s
4	12	fixed	4.41e-12	7	yes	-	0.8940	0.0041s
5	12	fixed	6.49e-09	5	yes	-	1.6080	0.0010s
6	12	fixed	5.54e-12	6	yes	-	0.9218	0.0026s
Avg Success			-	5.64	36/36	-	0.8632	0.0073s

Table 9: $n = 2$: Broyden (MN) - Finite Differences (Approx All)

pt ID	grad	iter/max	succ	flag	rate	time
1	6.17e-11	6	yes	-	2.0008	0.0121s
2	1.25e-10	6	yes	-	1.7379	0.0029s
3	0.0	6	yes	-	0.2723	0.0023s
4	6.93e-10	7	yes	-	2.0003	0.0015s
5	6.11e-15	6	yes	-	0.6829	0.0012s
6	9.28e-11	6	yes	-	2.0001	0.0005s
Avg	-	6.17	6/6	-	1.4490	0.0034s

Table 10: $n = 2$: Broyden (TN) - Exact Derivatives

pt	k	mode	grad	iter/max	succ	flag	rate	time
1	4	adaptive	8.30e-05	5	yes	-	0.9463	0.0015s
2	4	adaptive	4.38e-07	6	yes	-	0.0448	0.0009s
3	4	adaptive	3.73e-06	5	yes	-	0.2115	0.0010s
4	4	adaptive	7.22e-07	7	yes	-	0.0652	0.0008s
5	4	adaptive	7.56e-06	5	yes	-	0.3280	0.0030s
6	4	adaptive	9.64e-05	5	yes	-	0.9884	0.0028s
1	4	fixed	7.21e-07	6	yes	-	0.0299	0.0050s
2	4	fixed	9.15e-07	6	yes	-	0.0351	0.0012s
3	4	fixed	6.45e-05	6	yes	-	0.5668	0.0017s
4	4	fixed	1.43e-06	7	yes	-	0.0481	0.0037s
5	4	fixed	1.26e-05	5	yes	-	0.2266	0.0014s
6	4	fixed	8.29e-07	6	yes	-	0.0328	0.0023s
1	8	adaptive	8.86e-10	6	yes	-	0.8920	0.0022s
2	8	adaptive	1.34e-09	6	yes	-	0.9799	0.0006s
3	8	adaptive	2.28e-08	5	yes	-	1.5836	0.0023s
4	8	adaptive	3.36e-09	7	yes	-	1.1830	0.0026s
5	8	adaptive	2.81e-07	5	yes	-	1.8971	0.0014s
6	8	adaptive	1.13e-09	6	yes	-	0.9418	0.0009s
1	8	fixed	1.34e-09	6	yes	-	0.7610	0.0014s
2	8	fixed	2.00e-09	6	yes	-	0.8438	0.0028s
3	8	fixed	2.99e-08	5	yes	-	1.4444	0.0010s
4	8	fixed	4.82e-09	7	yes	-	1.0387	0.0011s
5	8	fixed	3.06e-07	5	yes	-	1.8257	0.0007s
6	8	fixed	1.69e-09	6	yes	-	0.8084	0.0007s
1	12	adaptive	1.67e-09	6	yes	-	1.4783	0.0011s
2	12	adaptive	5.46e-09	6	yes	-	1.3630	0.0014s
3	12	adaptive	4.93e-12	6	yes	-	0.5116	0.0007s
4	12	adaptive	3.48e-09	7	yes	-	1.7100	0.0019s
5	12	adaptive	4.05e-11	6	yes	-	0.6341	0.0008s
6	12	adaptive	1.33e-09	6	yes	-	1.6443	0.0007s
1	12	fixed	1.30e-09	6	yes	-	1.6929	0.0005s
2	12	fixed	1.45e-09	6	yes	-	1.8330	0.0011s
3	12	fixed	4.68e-13	6	yes	-	0.4058	0.0006s
4	12	fixed	7.10e-09	7	yes	-	1.5880	0.0008s
5	12	fixed	1.57e-11	6	yes	-	0.7513	0.0008s
6	12	fixed	1.37e-09	6	yes	-	1.7080	0.0007s
Avg Success			-	5.92	36/36	-	0.9179	0.0015s

Table 11: $n = 2$: Broyden (TN) - Finite Differences (With exact gradient)

pt	k	mode	grad	iter/max	succ	flag	rate	time
1	4	adaptive	8.30e-05	5	yes	-	0.9463	0.0015s
2	4	adaptive	4.38e-07	6	yes	-	0.0448	0.0009s
3	4	adaptive	3.73e-06	5	yes	-	0.2115	0.0010s
4	4	adaptive	7.22e-07	7	yes	-	0.0652	0.0008s
5	4	adaptive	7.56e-06	5	yes	-	0.3280	0.0030s
6	4	adaptive	9.64e-05	5	yes	-	0.9884	0.0028s
1	4	fixed	7.21e-07	6	yes	-	0.0299	0.0050s
2	4	fixed	9.15e-07	6	yes	-	0.0351	0.0012s
3	4	fixed	6.45e-05	6	yes	-	0.5668	0.0017s
4	4	fixed	1.43e-06	7	yes	-	0.0481	0.0037s
5	4	fixed	1.26e-05	5	yes	-	0.2266	0.0014s
6	4	fixed	8.29e-07	6	yes	-	0.0328	0.0023s
1	8	adaptive	8.86e-10	6	yes	-	0.8920	0.0022s
2	8	adaptive	1.34e-09	6	yes	-	0.9799	0.0006s
3	8	adaptive	2.28e-08	5	yes	-	1.5836	0.0023s
4	8	adaptive	3.36e-09	7	yes	-	1.1830	0.0026s
5	8	adaptive	2.81e-07	5	yes	-	1.8971	0.0014s
6	8	adaptive	1.13e-09	6	yes	-	0.9418	0.0009s
1	8	fixed	1.34e-09	6	yes	-	0.7610	0.0014s
2	8	fixed	2.00e-09	6	yes	-	0.8438	0.0028s
3	8	fixed	2.99e-08	5	yes	-	1.4444	0.0010s
4	8	fixed	4.82e-09	7	yes	-	1.0387	0.0011s
5	8	fixed	3.06e-07	5	yes	-	1.8257	0.0007s
6	8	fixed	1.69e-09	6	yes	-	0.8084	0.0007s
1	12	adaptive	1.67e-09	6	yes	-	1.4783	0.0011s
2	12	adaptive	5.46e-09	6	yes	-	1.3630	0.0014s
3	12	adaptive	4.93e-12	6	yes	-	0.5116	0.0007s
4	12	adaptive	3.48e-09	7	yes	-	1.7100	0.0019s
5	12	adaptive	4.05e-11	6	yes	-	0.6341	0.0008s
6	12	adaptive	1.33e-09	6	yes	-	1.6443	0.0007s
1	12	fixed	1.30e-09	6	yes	-	1.6929	0.0005s
2	12	fixed	1.45e-09	6	yes	-	1.8330	0.0011s
3	12	fixed	4.68e-13	6	yes	-	0.4058	0.0006s
4	12	fixed	7.10e-09	7	yes	-	1.5880	0.0008s
5	12	fixed	1.57e-11	6	yes	-	0.7513	0.0008s
6	12	fixed	1.37e-09	6	yes	-	1.7080	0.0007s
Avg Success			-	5.92	36/36	-	0.9179	0.0015s

Table 12: $n = 2$: Broyden (TN) - Finite Differences (Approx All)

Analysis for Dimension $n = 1000$

pt ID	grad	iter/max	succ	flag	rate	time
1	2.57e-12	6	yes	-	1.7319	0.1102s
2	2.60e-12	9	yes	-	2.1242	0.0566s
3	1.22e-07	14	yes	-	1.7290	0.0503s
4	1.01e-11	11	yes	-	1.9046	0.0435s
5	2.05e-08	14	yes	-	1.7473	0.0515s
6	2.92e-12	10	yes	-	1.8868	0.0444s
Avg	-	10.67	6/6	-	1.8540	0.0594s

Table 13: $n = 1000$: Chained (MN) - Exact Derivatives

pt	k	mode	grad	iter/max	succ	flag	rate	time
1	4	adaptive	2.32e-03	34	yes	-	0.7573	0.1152s
2	4	adaptive	2.80e-03	35	yes	-	0.7622	0.1398s
3	4	adaptive	2.91e-03	35	yes	-	0.7632	0.0869s
4	4	adaptive	2.41e-03	36	yes	-	0.7582	0.1159s
5	4	adaptive	2.80e-03	37	yes	-	0.7621	0.0510s
6	4	adaptive	2.92e-03	35	yes	-	0.7633	0.1116s
1	4	fixed	2.30e-03	34	yes	-	0.7700	0.0851s
2	4	fixed	2.78e-03	35	yes	-	0.7771	0.1385s
3	4	fixed	2.89e-03	35	yes	-	0.7786	0.0934s
4	4	fixed	2.39e-03	36	yes	-	0.7713	0.0925s
5	4	fixed	2.78e-03	37	yes	-	0.7770	0.1055s
6	4	fixed	2.90e-03	35	yes	-	0.7787	0.1004s
1	8	adaptive	7.54e-05	183	no	fail	N/A	0.9380s
2	8	adaptive	6.46e-05	185	no	fail	N/A	0.9493s
3	8	adaptive	4.16e-05	185	no	fail	N/A	0.9549s
4	8	adaptive	6.46e-05	185	no	fail	N/A	1.2144s
5	8	adaptive	2.92e-05	80	yes	-	0.8564	0.4181s
6	8	adaptive	5.78e-05	185	no	fail	N/A	0.8973s
1	8	fixed	6.60e-05	183	no	fail	N/A	1.2565s
2	8	fixed	1.06e-04	185	no	fail	N/A	1.0339s
3	8	fixed	6.30e-05	185	no	fail	N/A	0.9036s
4	8	fixed	8.42e-05	185	no	fail	N/A	0.9418s
5	8	fixed	6.42e-05	187	no	fail	N/A	1.0778s
6	8	fixed	6.67e-05	185	no	fail	N/A	0.9212s
1	12	adaptive	5.15e-01	304	no	fail	N/A	1.5074s
2	12	adaptive	5.42e-01	286	no	fail	N/A	1.7638s
3	12	adaptive	5.46e-01	265	no	fail	N/A	1.3108s
4	12	adaptive	5.48e-01	269	no	fail	N/A	1.4128s
5	12	adaptive	5.53e-01	286	no	fail	N/A	1.4262s
6	12	adaptive	5.38e-01	292	no	fail	N/A	1.4903s
1	12	fixed	7.83e-01	371	no	fail	N/A	1.6236s
2	12	fixed	7.89e-01	365	no	fail	N/A	1.7248s
3	12	fixed	7.68e-01	335	no	fail	N/A	1.6602s
4	12	fixed	7.42e-01	367	no	fail	N/A	1.9148s
5	12	fixed	7.51e-01	336	no	fail	N/A	1.6957s
6	12	fixed	7.64e-01	395	no	fail	N/A	1.9757s
Avg Success			-	38.77	13/36	-	0.7750	0.1272s

Table 14: $n = 1000$: Chained (MN) - Finite Differences (With exact gradient)

pt	k	mode	grad	iter/max	succ	flag	rate	time
1	4	adaptive	2.32e-03	34	yes	-	0.7573	0.1152s
2	4	adaptive	2.80e-03	35	yes	-	0.7622	0.1398s
3	4	adaptive	2.91e-03	35	yes	-	0.7632	0.0869s
4	4	adaptive	2.41e-03	36	yes	-	0.7582	0.1159s
5	4	adaptive	2.80e-03	37	yes	-	0.7621	0.0510s
6	4	adaptive	2.92e-03	35	yes	-	0.7633	0.1116s
1	4	fixed	2.30e-03	34	yes	-	0.7700	0.0851s
2	4	fixed	2.78e-03	35	yes	-	0.7771	0.1385s
3	4	fixed	2.89e-03	35	yes	-	0.7786	0.0934s
4	4	fixed	2.39e-03	36	yes	-	0.7713	0.0925s
5	4	fixed	2.78e-03	37	yes	-	0.7770	0.1055s
6	4	fixed	2.90e-03	35	yes	-	0.7787	0.1004s
1	8	adaptive	7.54e-05	183	no	fail	N/A	0.9380s
2	8	adaptive	6.46e-05	185	no	fail	N/A	0.9493s
3	8	adaptive	4.16e-05	185	no	fail	N/A	0.9549s
4	8	adaptive	6.46e-05	185	no	fail	N/A	1.2144s
5	8	adaptive	2.92e-05	80	yes	-	0.8564	0.4181s
6	8	adaptive	5.78e-05	185	no	fail	N/A	0.8973s
1	8	fixed	6.60e-05	183	no	fail	N/A	1.2565s
2	8	fixed	1.06e-04	185	no	fail	N/A	1.0339s
3	8	fixed	6.30e-05	185	no	fail	N/A	0.9036s
4	8	fixed	8.42e-05	185	no	fail	N/A	0.9418s
5	8	fixed	6.42e-05	187	no	fail	N/A	1.0778s
6	8	fixed	6.67e-05	185	no	fail	N/A	0.9212s
1	12	adaptive	5.15e-01	304	no	fail	N/A	1.5074s
2	12	adaptive	5.42e-01	286	no	fail	N/A	1.7638s
3	12	adaptive	5.46e-01	265	no	fail	N/A	1.3108s
4	12	adaptive	5.48e-01	269	no	fail	N/A	1.4128s
5	12	adaptive	5.53e-01	286	no	fail	N/A	1.4262s
6	12	adaptive	5.38e-01	292	no	fail	N/A	1.4903s
1	12	fixed	7.83e-01	371	no	fail	N/A	1.6236s
2	12	fixed	7.89e-01	365	no	fail	N/A	1.7248s
3	12	fixed	7.68e-01	335	no	fail	N/A	1.6602s
4	12	fixed	7.42e-01	367	no	fail	N/A	1.9148s
5	12	fixed	7.51e-01	336	no	fail	N/A	1.6957s
6	12	fixed	7.64e-01	395	no	fail	N/A	1.9757s
Avg Success			-	38.77	13/36	-	0.7750	0.1272s

Table 15: $n = 1000$: Chained (MN) - Finite Differences (Approx All)

pt ID	grad	iter/max	succ	flag	rate	time
1	2.01e-06	162	no	fail	N/A	0.3664s
2	3.42e-06	164	no	fail	N/A	0.2823s
3	5.32e-10	16	yes	-	1.6297	0.0250s
4	7.14e-10	17	yes	-	1.5903	0.0207s
5	1.64e-09	17	yes	-	1.3007	0.0196s
6	1.67e-10	20	yes	-	2.4301	0.0219s
Avg	-	17.50	4/6	-	1.7377	0.0218s

Table 16: $n = 1000$: Chained (TN) - Exact Derivatives

pt	k	mode	grad	iter/max	succ	flag	rate	time
1	4	adaptive	2.89e-03	48	yes	-	0.0016	0.0218s
2	4	adaptive	2.85e-03	50	yes	-	0.0025	0.0110s
3	4	adaptive	2.88e-03	49	yes	-	0.0025	0.0158s
4	4	adaptive	2.17e-03	50	yes	-	0.0024	0.0094s
5	4	adaptive	2.13e-03	56	yes	-	0.0020	0.0093s
6	4	adaptive	2.78e-03	50	yes	-	0.0020	0.0141s
1	4	fixed	2.88e-03	48	yes	-	0.0016	0.0195s
2	4	fixed	2.83e-03	50	yes	-	0.0026	0.0158s
3	4	fixed	2.87e-03	49	yes	-	0.0026	0.0123s
4	4	fixed	2.17e-03	50	yes	-	0.0026	0.0147s
5	4	fixed	2.13e-03	56	yes	-	0.0020	0.0199s
6	4	fixed	2.77e-03	52	yes	-	0.0020	0.0108s
1	8	adaptive	6.07e-05	205	no	fail	N/A	0.1651s
2	8	adaptive	6.21e-05	207	no	fail	N/A	0.2302s
3	8	adaptive	5.82e-05	206	no	fail	N/A	0.2085s
4	8	adaptive	5.42e-05	206	no	fail	N/A	0.2432s
5	8	adaptive	5.61e-05	212	no	fail	N/A	0.2219s
6	8	adaptive	4.84e-05	207	no	fail	N/A	0.2149s
1	8	fixed	6.48e-05	205	no	fail	N/A	0.2714s
2	8	fixed	8.57e-05	207	no	fail	N/A	0.3828s
3	8	fixed	1.16e-04	206	no	fail	N/A	0.2518s
4	8	fixed	7.61e-05	206	no	fail	N/A	0.1921s
5	8	fixed	1.08e-04	212	no	fail	N/A	0.2208s
6	8	fixed	9.32e-05	207	no	fail	N/A	0.1861s
1	12	adaptive	5.37e-01	210	no	fail	N/A	0.8738s
2	12	adaptive	6.45e-01	207	no	fail	N/A	0.7639s
3	12	adaptive	6.24e-01	212	no	fail	N/A	0.8083s
4	12	adaptive	7.51e-01	205	no	fail	N/A	0.8167s
5	12	adaptive	6.32e-01	206	no	fail	N/A	0.7914s
6	12	adaptive	6.13e-01	207	no	fail	N/A	0.7786s
1	12	fixed	6.91e-01	211	no	fail	N/A	0.7346s
2	12	fixed	8.45e-01	208	no	fail	N/A	0.6689s
3	12	fixed	1.06e+00	206	no	fail	N/A	0.8426s
4	12	fixed	8.63e-01	207	no	fail	N/A	0.7769s
5	12	fixed	8.54e-01	207	no	fail	N/A	0.7600s
6	12	fixed	8.43e-01	205	no	fail	N/A	0.7540s
Avg Success			-	50.67	12/36	-	0.0022	0.0145s

Table 17: $n = 1000$: Chained (TN) - Finite Differences (With exact gradient)

pt	k	mode	grad	iter/max	succ	flag	rate	time
1	4	adaptive	2.89e-03	48	yes	-	0.0016	0.0218s
2	4	adaptive	2.85e-03	50	yes	-	0.0025	0.0110s
3	4	adaptive	2.88e-03	49	yes	-	0.0025	0.0158s
4	4	adaptive	2.17e-03	50	yes	-	0.0024	0.0094s
5	4	adaptive	2.13e-03	56	yes	-	0.0020	0.0093s
6	4	adaptive	2.78e-03	50	yes	-	0.0020	0.0141s
1	4	fixed	2.88e-03	48	yes	-	0.0016	0.0195s
2	4	fixed	2.83e-03	50	yes	-	0.0026	0.0158s
3	4	fixed	2.87e-03	49	yes	-	0.0026	0.0123s
4	4	fixed	2.17e-03	50	yes	-	0.0026	0.0147s
5	4	fixed	2.13e-03	56	yes	-	0.0020	0.0199s
6	4	fixed	2.77e-03	52	yes	-	0.0020	0.0108s
1	8	adaptive	6.07e-05	205	no	fail	N/A	0.1651s
2	8	adaptive	6.21e-05	207	no	fail	N/A	0.2302s
3	8	adaptive	5.82e-05	206	no	fail	N/A	0.2085s
4	8	adaptive	5.42e-05	206	no	fail	N/A	0.2432s
5	8	adaptive	5.61e-05	212	no	fail	N/A	0.2219s
6	8	adaptive	4.84e-05	207	no	fail	N/A	0.2149s
1	8	fixed	6.48e-05	205	no	fail	N/A	0.2714s
2	8	fixed	8.57e-05	207	no	fail	N/A	0.3828s
3	8	fixed	1.16e-04	206	no	fail	N/A	0.2518s
4	8	fixed	7.61e-05	206	no	fail	N/A	0.1921s
5	8	fixed	1.08e-04	212	no	fail	N/A	0.2208s
6	8	fixed	9.32e-05	207	no	fail	N/A	0.1861s
1	12	adaptive	5.37e-01	210	no	fail	N/A	0.8738s
2	12	adaptive	6.45e-01	216	no	fail	N/A	0.7639s
3	12	adaptive	6.24e-01	205	no	fail	N/A	0.8083s
4	12	adaptive	7.51e-01	207	no	fail	N/A	0.8167s
5	12	adaptive	6.32e-01	206	no	fail	N/A	0.7914s
6	12	adaptive	6.13e-01	203	no	fail	N/A	0.7786s
1	12	fixed	6.91e-01	206	no	fail	N/A	0.7346s
2	12	fixed	8.45e-01	200	no	fail	N/A	0.6689s
3	12	fixed	1.06e+00	213	no	fail	N/A	0.8426s
4	12	fixed	8.63e-01	204	no	fail	N/A	0.7769s
5	12	fixed	8.54e-01	212	no	fail	N/A	0.7600s
6	12	fixed	8.43e-01	208	no	fail	N/A	0.7540s
Avg Success			-	50.67	12/36	-	0.0022	0.0145s

Table 18: $n = 1000$: Chained (TN) - Finite Differences (Approx All)

pt ID	grad	iter/max	succ	flag	rate	time
1	1.79e-08	5	yes	-	1.6788	0.0295s
2	8.77e-11	10	yes	-	1.9121	0.0454s
3	2.74e-10	10	yes	-	1.9604	0.0343s
4	8.33e-09	9	yes	-	1.8782	0.0307s
5	1.32e-08	10	yes	-	1.9610	0.0509s
6	2.75e-07	9	yes	-	1.8686	0.0442s
Avg	-	8.83	6/6	-	1.8765	0.0392s

Table 19: $n = 1000$: Broyden (MN) - Exact Derivatives

pt	k	mode	grad	iter/max	succ	flag	rate	time
1	4	adaptive	2.51e-03	4	yes	-	0.5284	0.0118s
2	4	adaptive	2.51e-04	9	yes	-	0.0550	0.0508s
3	4	adaptive	2.51e-04	9	yes	-	0.0551	0.0787s
4	4	adaptive	9.17e-04	8	yes	-	0.1981	0.0687s
5	4	adaptive	6.17e-04	9	yes	-	0.1339	0.0867s
6	4	adaptive	3.71e-05	9	yes	-	0.0105	0.0565s
1	4	fixed	4.51e-05	5	yes	-	0.0344	0.0380s
2	4	fixed	2.94e-04	9	yes	-	0.0403	0.0400s
3	4	fixed	2.35e-04	9	yes	-	0.0337	0.0370s
4	4	fixed	1.02e-03	8	yes	-	0.1420	0.0184s
5	4	fixed	5.23e-04	9	yes	-	0.0677	0.0316s
6	4	fixed	4.73e-05	9	yes	-	0.0103	0.0361s
1	8	adaptive	4.60e-08	5	yes	-	0.7481	0.0270s
2	8	adaptive	2.55e-09	10	yes	-	0.2599	0.0691s
3	8	adaptive	4.77e-09	10	yes	-	0.3671	0.1299s
4	8	adaptive	5.02e-07	10	yes	-	1.3148	0.1458s
5	8	adaptive	4.26e-08	10	yes	-	0.8052	0.0704s
6	8	adaptive	3.96e-07	9	yes	-	1.2443	0.0663s
1	8	fixed	5.51e-08	5	yes	-	0.6906	0.0290s
2	8	fixed	3.07e-09	10	yes	-	0.2123	0.0793s
3	8	fixed	5.65e-09	10	yes	-	0.3128	0.0824s
4	8	fixed	3.59e-08	9	yes	-	0.6632	0.0593s
5	8	fixed	4.83e-08	10	yes	-	0.7377	0.0714s
6	8	fixed	4.22e-07	9	yes	-	1.1699	0.0595s
1	12	adaptive	6.49e-08	5	yes	-	1.4842	0.0328s
2	12	adaptive	9.06e-10	10	yes	-	1.1965	0.0663s
3	12	adaptive	2.01e-10	10	yes	-	1.2261	0.0754s
4	12	adaptive	1.45e-10	11	yes	-	0.6941	0.0851s
5	12	adaptive	3.91e-08	10	yes	-	1.5761	0.0760s
6	12	adaptive	2.25e-07	9	yes	-	1.7779	0.0865s
1	12	fixed	2.74e-08	5	yes	-	1.8011	0.0576s
2	12	fixed	1.57e-09	10	yes	-	1.5332	0.0742s
3	12	fixed	7.59e-09	10	yes	-	1.6411	0.0925s
4	12	fixed	3.94e-08	9	yes	-	1.6543	0.0640s
5	12	fixed	4.86e-08	10	yes	-	1.8249	0.0713s
6	12	fixed	2.41e-11	10	yes	-	0.0333	0.0906s
Avg Success			-	8.69	36/36	-	0.7299	0.0643s

Table 20: $n = 1000$: Broyden (MN) - Finite Differences (With exact gradient)

pt	k	mode	grad	iter/max	succ	flag	rate	time
1	4	adaptive	2.51e-03	4	yes	-	0.5284	0.0118s
2	4	adaptive	2.51e-04	9	yes	-	0.0550	0.0508s
3	4	adaptive	2.51e-04	9	yes	-	0.0551	0.0787s
4	4	adaptive	9.17e-04	8	yes	-	0.1981	0.0687s
5	4	adaptive	6.17e-04	9	yes	-	0.1339	0.0867s
6	4	adaptive	3.71e-05	9	yes	-	0.0105	0.0565s
1	4	fixed	4.51e-05	5	yes	-	0.0344	0.0380s
2	4	fixed	2.94e-04	9	yes	-	0.0403	0.0400s
3	4	fixed	2.35e-04	9	yes	-	0.0337	0.0370s
4	4	fixed	1.02e-03	8	yes	-	0.1420	0.0184s
5	4	fixed	5.23e-04	9	yes	-	0.0677	0.0316s
6	4	fixed	4.73e-05	9	yes	-	0.0103	0.0361s
1	8	adaptive	4.60e-08	5	yes	-	0.7481	0.0270s
2	8	adaptive	2.55e-09	10	yes	-	0.2599	0.0691s
3	8	adaptive	4.77e-09	10	yes	-	0.3671	0.1299s
4	8	adaptive	5.02e-07	10	yes	-	1.3148	0.1458s
5	8	adaptive	4.26e-08	10	yes	-	0.8052	0.0704s
6	8	adaptive	3.96e-07	9	yes	-	1.2443	0.0663s
1	8	fixed	5.51e-08	5	yes	-	0.6906	0.0290s
2	8	fixed	3.07e-09	10	yes	-	0.2123	0.0793s
3	8	fixed	5.65e-09	10	yes	-	0.3128	0.0824s
4	8	fixed	3.59e-08	9	yes	-	0.6632	0.0593s
5	8	fixed	4.83e-08	10	yes	-	0.7377	0.0714s
6	8	fixed	4.22e-07	9	yes	-	1.1699	0.0595s
1	12	adaptive	6.49e-08	5	yes	-	1.4842	0.0328s
2	12	adaptive	9.06e-10	10	yes	-	1.1965	0.0663s
3	12	adaptive	2.01e-10	10	yes	-	1.2261	0.0754s
4	12	adaptive	1.45e-10	11	yes	-	0.6941	0.0851s
5	12	adaptive	3.91e-08	10	yes	-	1.5761	0.0760s
6	12	adaptive	2.25e-07	9	yes	-	1.7779	0.0865s
1	12	fixed	2.74e-08	5	yes	-	1.8011	0.0576s
2	12	fixed	1.57e-09	10	yes	-	1.5332	0.0742s
3	12	fixed	7.59e-09	10	yes	-	1.6411	0.0925s
4	12	fixed	3.94e-08	9	yes	-	1.6543	0.0640s
5	12	fixed	4.86e-08	10	yes	-	1.8249	0.0713s
6	12	fixed	2.41e-11	10	yes	-	2.0333	0.0906s
Avg Success			-	8.69	36/36	-	0.7855	0.0643s

Table 21: $n = 1000$: Broyden (MN) - Finite Differences (Approx All)

pt ID	grad	iter/max	succ	flag	rate	time
1	1.61e-07	11	yes	-	2.2647	0.0056s
2	5.95e-09	14	yes	-	1.3477	0.0043s
3	6.51e-09	14	yes	-	1.9372	0.0100s
4	2.24e-09	14	yes	-	1.1198	0.0054s
5	2.12e-10	13	yes	-	0.4947	0.0048s
6	1.15e-09	15	yes	-	0.2649	0.0052s
Avg	-	13.50	6/6	-	1.2382	0.0059s

Table 22: $n = 1000$: Broyden (TN) - Exact Derivatives

pt	k	mode	grad	iter/max	succ	flag	rate	time
1	4	adaptive	1.55e-03	9	yes	-	0.3400	0.0055s
2	4	adaptive	2.64e-03	12	yes	-	0.0366	0.0079s
3	4	adaptive	1.68e-02	160	no	fail	N/A	0.3963s
4	4	adaptive	2.40e-03	12	yes	-	0.0289	0.0117s
5	4	adaptive	2.62e-03	10	yes	-	0.0413	0.0120s
6	4	adaptive	2.89e-03	12	yes	-	0.0260	0.0128s
1	4	fixed	1.57e-03	9	yes	-	0.3083	0.0043s
2	4	fixed	1.04e-02	161	no	fail	N/A	0.1902s
3	4	fixed	3.26e-02	159	no	fail	N/A	0.1493s
4	4	fixed	1.69e-02	160	no	fail	N/A	0.1455s
5	4	fixed	1.08e-02	161	no	fail	N/A	0.1586s
6	4	fixed	1.09e-02	161	no	fail	N/A	0.1682s
1	8	adaptive	1.62e-07	11	yes	-	0.7667	0.0105s
2	8	adaptive	3.44e-06	13	yes	-	1.2421	0.0161s
3	8	adaptive	5.05e-06	13	yes	-	0.9091	0.0218s
4	8	adaptive	2.91e-06	13	yes	-	1.2051	0.0174s
5	8	adaptive	3.74e-07	12	yes	-	0.4846	0.0084s
6	8	adaptive	1.48e-06	14	yes	-	0.9493	0.0120s
1	8	fixed	1.62e-07	11	yes	-	0.6625	0.0082s
2	8	fixed	3.44e-06	13	yes	-	1.1234	0.0091s
3	8	fixed	5.05e-06	13	yes	-	0.8058	0.0099s
4	8	fixed	2.91e-06	13	yes	-	1.0886	0.0100s
5	8	fixed	3.74e-07	12	yes	-	0.3904	0.0111s
6	8	fixed	1.48e-06	14	yes	-	0.8341	0.0141s
1	12	adaptive	1.66e-07	11	yes	-	2.2479	0.0104s
2	12	adaptive	5.97e-09	14	yes	-	1.3510	0.0163s
3	12	adaptive	6.52e-09	14	yes	-	1.9371	0.0163s
4	12	adaptive	5.64e-08	13	yes	-	1.6698	0.0119s
5	12	adaptive	2.11e-10	13	yes	-	0.4948	0.0107s
6	12	adaptive	1.17e-09	15	yes	-	0.2653	0.0223s
1	12	fixed	1.62e-07	11	yes	-	2.2702	0.0095s
2	12	fixed	5.96e-09	14	yes	-	1.3506	0.0147s
3	12	fixed	6.55e-09	14	yes	-	1.9299	0.0124s
4	12	fixed	2.25e-09	14	yes	-	1.1212	0.0135s
5	12	fixed	2.13e-10	13	yes	-	0.4954	0.0131s
6	12	fixed	1.16e-09	15	yes	-	0.2658	0.0232s
Avg Success			-	12.57	30/36	-	0.8881	0.0126s

Table 23: $n = 1000$: Broyden (TN) - Finite Differences (With exact gradient)

pt	k	mode	grad	iter/max	succ	flag	rate	time
1	4	adaptive	1.55e-03	9	yes	-	0.3400	0.0055s
2	4	adaptive	2.64e-03	12	yes	-	0.0366	0.0079s
3	4	adaptive	1.68e-02	160	no	fail	N/A	0.3963s
4	4	adaptive	2.40e-03	12	yes	-	0.0289	0.0117s
5	4	adaptive	2.62e-03	10	yes	-	0.0413	0.0120s
6	4	adaptive	2.89e-03	12	yes	-	0.0260	0.0128s
1	4	fixed	1.57e-03	9	yes	-	0.3083	0.0043s
2	4	fixed	1.04e-02	161	no	fail	N/A	0.1902s
3	4	fixed	3.26e-02	159	no	fail	N/A	0.1493s
4	4	fixed	1.69e-02	160	no	fail	N/A	0.1455s
5	4	fixed	1.08e-02	161	no	fail	N/A	0.1586s
6	4	fixed	1.09e-02	161	no	fail	N/A	0.1682s
1	8	adaptive	1.62e-07	11	yes	-	0.7667	0.0105s
2	8	adaptive	3.44e-06	13	yes	-	1.2421	0.0161s
3	8	adaptive	5.05e-06	13	yes	-	0.9091	0.0218s
4	8	adaptive	2.91e-06	13	yes	-	1.2051	0.0174s
5	8	adaptive	3.74e-07	12	yes	-	0.4846	0.0084s
6	8	adaptive	1.48e-06	14	yes	-	0.9493	0.0120s
1	8	fixed	1.62e-07	11	yes	-	0.6625	0.0082s
2	8	fixed	3.44e-06	13	yes	-	1.1234	0.0091s
3	8	fixed	5.05e-06	13	yes	-	0.8058	0.0099s
4	8	fixed	2.91e-06	13	yes	-	1.0886	0.0100s
5	8	fixed	3.74e-07	12	yes	-	0.3904	0.0111s
6	8	fixed	1.48e-06	14	yes	-	0.8341	0.0141s
1	12	adaptive	1.66e-07	11	yes	-	2.2479	0.0104s
2	12	adaptive	5.97e-09	14	yes	-	1.3510	0.0163s
3	12	adaptive	6.52e-09	14	yes	-	1.9371	0.0163s
4	12	adaptive	5.64e-08	13	yes	-	1.6698	0.0119s
5	12	adaptive	2.11e-10	13	yes	-	0.4948	0.0107s
6	12	adaptive	1.17e-09	15	yes	-	0.2653	0.0223s
1	12	fixed	1.62e-07	11	yes	-	2.2702	0.0095s
2	12	fixed	5.96e-09	14	yes	-	1.3506	0.0147s
3	12	fixed	6.55e-09	14	yes	-	1.9299	0.0124s
4	12	fixed	2.25e-09	14	yes	-	1.1212	0.0135s
5	12	fixed	2.13e-10	13	yes	-	0.4954	0.0131s
6	12	fixed	1.16e-09	15	yes	-	0.2658	0.0232s
Avg Success			-	12.57	30/36	-	0.8881	0.0126s

Table 24: $n = 1000$: Broyden (TN) - Finite Differences (Approx All)

Analysis for Dimension $n = 10000$

pt ID	grad	iter/max	succ	flag	rate	time
1	8.64e-07	5	yes	-	2.5771	0.3024s
2	6.04e-11	10	yes	-	1.9293	0.5978s
3	1.02e-07	13	yes	-	1.7301	0.7973s
4	1.53e-09	9	yes	-	1.8590	0.5104s
5	8.68e-12	16	yes	-	1.9294	0.8806s
6	2.85e-11	15	yes	-	1.8824	0.8593s
Avg	-	11.33	6/6	-	1.9845	0.6580s

Table 25: $n = 10000$: Chained (MN) - Exact Derivatives

pt	k	mode	grad	iter/max	succ	flag	rate	time
1	4	adaptive	8.05e-03	30	yes	-	0.7417	1.5143s
2	4	adaptive	8.32e-03	31	yes	-	0.7429	1.6394s
3	4	adaptive	9.57e-03	31	yes	-	0.7488	1.5377s
4	4	adaptive	7.43e-03	32	yes	-	0.7389	1.5639s
5	4	adaptive	8.44e-03	32	yes	-	0.7434	1.3956s
6	4	adaptive	7.56e-03	32	yes	-	0.7395	1.6026s
1	4	fixed	8.00e-03	30	yes	-	0.7599	1.6915s
2	4	fixed	8.27e-03	31	yes	-	0.7621	1.5768s
3	4	fixed	9.51e-03	31	yes	-	0.7724	1.5475s
4	4	fixed	7.38e-03	32	yes	-	0.7548	1.9060s
5	4	fixed	8.39e-03	32	yes	-	0.7631	1.6274s
6	4	fixed	7.51e-03	32	yes	-	0.7559	1.4575s
1	8	adaptive	1.04e-03	180	no	fail	N/A	9.2064s
2	8	adaptive	1.63e-04	181	no	fail	N/A	8.5350s
3	8	adaptive	2.18e-04	181	no	fail	N/A	8.4935s
4	8	adaptive	2.53e-04	181	no	fail	N/A	8.4313s
5	8	adaptive	2.10e-04	182	no	fail	N/A	8.5558s
6	8	adaptive	2.09e-04	181	no	fail	N/A	8.3711s
1	8	fixed	9.46e-05	46	yes	-	0.9831	2.3149s
2	8	fixed	2.94e-04	181	no	fail	N/A	6.5999s
3	8	fixed	3.62e-04	181	no	fail	N/A	8.6143s
4	8	fixed	2.66e-04	181	no	fail	N/A	8.4060s
5	8	fixed	2.79e-04	182	no	fail	N/A	9.0238s
6	8	fixed	2.74e-04	181	no	fail	N/A	8.5545s
1	12	adaptive	9.49e-01	213	no	fail	N/A	9.0526s
2	12	adaptive	1.76e+00	260	no	fail	N/A	11.0180s
3	12	adaptive	1.76e+00	254	no	fail	N/A	11.0429s
4	12	adaptive	1.74e+00	262	no	fail	N/A	11.5253s
5	12	adaptive	1.75e+00	266	no	fail	N/A	12.7677s
6	12	adaptive	1.75e+00	260	no	fail	N/A	7.6317s
1	12	fixed	2.48e+00	937	no	fail	N/A	39.9147s
2	12	fixed	2.44e+00	329	no	fail	N/A	14.0999s
3	12	fixed	2.42e+00	317	no	fail	N/A	13.7438s
4	12	fixed	2.44e+00	294	no	fail	N/A	12.6236s
5	12	fixed	2.45e+00	320	no	fail	N/A	13.9272s
6	12	fixed	2.46e+00	301	no	fail	N/A	12.7535s
Avg Success			-	32.46	13/36	-	0.7697	1.6442s

Table 26: $n = 10000$: Chained (MN) - Finite Differences (With exact gradient)

pt	k	mode	grad	iter/max	succ	flag	rate	time
1	4	adaptive	8.05e-03	30	yes	-	0.7417	1.5143s
2	4	adaptive	8.32e-03	31	yes	-	0.7429	1.6394s
3	4	adaptive	9.57e-03	31	yes	-	0.7488	1.5377s
4	4	adaptive	7.43e-03	32	yes	-	0.7389	1.5639s
5	4	adaptive	8.44e-03	32	yes	-	0.7434	1.3956s
6	4	adaptive	7.56e-03	32	yes	-	0.7395	1.6026s
1	4	fixed	8.00e-03	30	yes	-	0.7599	1.6915s
2	4	fixed	8.27e-03	31	yes	-	0.7621	1.5768s
3	4	fixed	9.51e-03	31	yes	-	0.7724	1.5475s
4	4	fixed	7.38e-03	32	yes	-	0.7548	1.9060s
5	4	fixed	8.39e-03	32	yes	-	0.7631	1.6274s
6	4	fixed	7.51e-03	32	yes	-	0.7559	1.4575s
1	8	adaptive	1.04e-03	180	no	fail	N/A	9.2064s
2	8	adaptive	1.63e-04	181	no	fail	N/A	8.5350s
3	8	adaptive	2.18e-04	181	no	fail	N/A	8.4935s
4	8	adaptive	2.53e-04	181	no	fail	N/A	8.4313s
5	8	adaptive	2.10e-04	182	no	fail	N/A	8.5558s
6	8	adaptive	2.09e-04	181	no	fail	N/A	8.3711s
1	8	fixed	9.46e-05	46	yes	-	0.9831	2.3149s
2	8	fixed	2.94e-04	181	no	fail	N/A	6.5999s
3	8	fixed	3.62e-04	181	no	fail	N/A	8.6143s
4	8	fixed	2.66e-04	181	no	fail	N/A	8.4060s
5	8	fixed	2.79e-04	182	no	fail	N/A	9.0238s
6	8	fixed	2.74e-04	181	no	fail	N/A	8.5545s
1	12	adaptive	9.49e-01	213	no	fail	N/A	9.0526s
2	12	adaptive	1.76e+00	260	no	fail	N/A	11.0180s
3	12	adaptive	1.76e+00	254	no	fail	N/A	11.0429s
4	12	adaptive	1.74e+00	262	no	fail	N/A	11.5253s
5	12	adaptive	1.75e+00	266	no	fail	N/A	12.7677s
6	12	adaptive	1.75e+00	260	no	fail	N/A	7.6317s
1	12	fixed	2.48e+00	937	no	fail	N/A	39.9147s
2	12	fixed	2.44e+00	329	no	fail	N/A	14.0999s
3	12	fixed	2.42e+00	317	no	fail	N/A	13.7438s
4	12	fixed	2.44e+00	294	no	fail	N/A	12.6236s
5	12	fixed	2.45e+00	320	no	fail	N/A	13.9272s
6	12	fixed	2.46e+00	301	no	fail	N/A	12.7535s
Avg Success			-	32.46	13/36	-	0.7697	1.6442s

Table 27: $n = 10000$: Chained (MN) - Finite Differences (Approx All)

pt ID	grad	iter/max	succ	flag	rate	time
1	9.19e-07	17	yes	-	0.6121	0.2570s
2	2.19e-10	17	yes	-	0.0479	0.3039s
3	1.06e-09	21	yes	-	0.2005	0.3274s
4	4.46e-10	17	yes	-	1.7263	0.3424s
5	1.17e-08	16	yes	-	0.1748	0.2597s
6	5.01e-10	16	yes	-	0.1837	0.2069s
Avg	-	17.33	6/6	-	0.4909	0.2829s

Table 28: $n = 10000$: Chained (TN) - Exact Derivatives

pt	k	mode	grad	iter/max	succ	flag	rate	time
1	4	adaptive	9.18e-03	42	yes	-	0.0018	0.1998s
2	4	adaptive	8.61e-03	46	yes	-	0.0019	0.3356s
3	4	adaptive	9.51e-03	47	yes	-	0.0019	0.2326s
4	4	adaptive	9.38e-03	43	yes	-	0.0025	0.2214s
5	4	adaptive	9.87e-03	43	yes	-	0.0025	0.1899s
6	4	adaptive	9.18e-03	43	yes	-	0.0025	0.1939s
1	4	fixed	9.15e-03	42	yes	-	0.0019	0.2161s
2	4	fixed	8.57e-03	46	yes	-	0.0019	0.2593s
3	4	fixed	9.48e-03	47	yes	-	0.0017	0.2483s
4	4	fixed	9.34e-03	43	yes	-	0.0026	0.2253s
5	4	fixed	9.84e-03	43	yes	-	0.0026	0.2373s
6	4	fixed	9.14e-03	43	yes	-	0.0026	0.2155s
1	8	adaptive	9.25e-05	67	yes	-	0.0514	0.3492s
2	8	adaptive	1.32e-04	206	no	fail	N/A	1.7831s
3	8	adaptive	1.91e-04	207	no	fail	N/A	1.8819s
4	8	adaptive	1.82e-04	203	no	fail	N/A	1.7484s
5	8	adaptive	2.62e-04	203	no	fail	N/A	1.8797s
6	8	adaptive	2.24e-04	203	no	fail	N/A	1.8817s
1	8	fixed	9.10e-05	68	yes	-	1.8092	0.3807s
2	8	fixed	3.16e-04	205	no	fail	N/A	1.8338s
3	8	fixed	1.39e-04	207	no	fail	N/A	1.7703s
4	8	fixed	2.42e-04	203	no	fail	N/A	1.6907s
5	8	fixed	2.46e-04	204	no	fail	N/A	1.7258s
6	8	fixed	2.45e-04	203	no	fail	N/A	1.7295s
1	12	adaptive	6.76e-01	205	no	fail	N/A	6.7777s
2	12	adaptive	1.95e+00	202	no	fail	N/A	6.3302s
3	12	adaptive	1.96e+00	203	no	fail	N/A	6.3077s
4	12	adaptive	1.94e+00	207	no	fail	N/A	6.8519s
5	12	adaptive	2.46e+00	205	no	fail	N/A	6.1701s
6	12	adaptive	2.43e+00	206	no	fail	N/A	2.5114s
1	12	fixed	8.37e-01	206	no	fail	N/A	7.0030s
2	12	fixed	2.73e+00	203	no	fail	N/A	6.1940s
3	12	fixed	2.65e+00	204	no	fail	N/A	6.1860s
4	12	fixed	3.13e+00	206	no	fail	N/A	6.0727s
5	12	fixed	3.40e+00	207	no	fail	N/A	5.8998s
6	12	fixed	3.32e+00	203	no	fail	N/A	5.8473s
Avg Success			-	47.36	14/36	-	0.1348	0.2503s

Table 29: $n = 10000$: Chained (TN) - Finite Differences (With exact gradient)

pt	k	mode	grad	iter/max	succ	flag	rate	time
1	4	adaptive	9.18e-03	42	yes	-	0.0018	0.1998s
2	4	adaptive	8.61e-03	46	yes	-	0.0019	0.3356s
3	4	adaptive	9.51e-03	47	yes	-	0.0019	0.2326s
4	4	adaptive	9.38e-03	43	yes	-	0.0025	0.2214s
5	4	adaptive	9.87e-03	43	yes	-	0.0025	0.1899s
6	4	adaptive	9.18e-03	43	yes	-	0.0025	0.1939s
1	4	fixed	9.15e-03	42	yes	-	0.0019	0.2161s
2	4	fixed	8.57e-03	46	yes	-	0.0019	0.2593s
3	4	fixed	9.48e-03	47	yes	-	0.0017	0.2483s
4	4	fixed	9.34e-03	43	yes	-	0.0026	0.2253s
5	4	fixed	9.84e-03	43	yes	-	0.0026	0.2373s
6	4	fixed	9.14e-03	43	yes	-	0.0026	0.2155s
1	8	adaptive	9.25e-05	67	yes	-	0.0514	0.3492s
2	8	adaptive	1.32e-04	206	no	fail	N/A	1.7831s
3	8	adaptive	1.91e-04	207	no	fail	N/A	1.8819s
4	8	adaptive	1.82e-04	203	no	fail	N/A	1.7484s
5	8	adaptive	2.62e-04	203	no	fail	N/A	1.8797s
6	8	adaptive	2.24e-04	203	no	fail	N/A	1.8817s
1	8	fixed	9.10e-05	68	yes	-	1.8092	0.3807s
2	8	fixed	3.16e-04	205	no	fail	N/A	1.8338s
3	8	fixed	1.39e-04	207	no	fail	N/A	1.7703s
4	8	fixed	2.42e-04	203	no	fail	N/A	1.6907s
5	8	fixed	2.46e-04	203	no	fail	N/A	1.7258s
6	8	fixed	2.45e-04	203	no	fail	N/A	1.7295s
1	12	adaptive	6.76e-01	204	no	fail	N/A	6.7777s
2	12	adaptive	1.95e+00	206	no	fail	N/A	6.3302s
3	12	adaptive	1.96e+00	207	no	fail	N/A	6.3077s
4	12	adaptive	1.94e+00	203	no	fail	N/A	6.8519s
5	12	adaptive	2.46e+00	205	no	fail	N/A	6.1701s
6	12	adaptive	2.43e+00	203	no	fail	N/A	2.5114s
1	12	fixed	8.37e-01	203	no	fail	N/A	7.0030s
2	12	fixed	2.73e+00	206	no	fail	N/A	6.1940s
3	12	fixed	2.65e+00	207	no	fail	N/A	6.1860s
4	12	fixed	3.13e+00	205	no	fail	N/A	6.0727s
5	12	fixed	3.40e+00	204	no	fail	N/A	5.8998s
6	12	fixed	3.32e+00	203	no	fail	N/A	5.8473s
Avg Success			-	47.36	14/36	-	0.1348	0.2503s

Table 30: $n = 10000$: Chained (TN) - Finite Differences (Approx All)

pt ID	grad	iter/max	succ	flag	rate	time
1	1.79e-08	5	yes	-	1.8745	0.3030s
2	6.92e-08	8	yes	-	1.8434	0.5433s
3	2.71e-08	8	yes	-	1.7983	0.5087s
4	7.10e-09	8	yes	-	1.8819	0.5191s
5	9.12e-09	8	yes	-	1.7831	0.4651s
6	1.81e-07	8	yes	-	1.7929	0.5405s
Avg	-	7.50	6/6	-	1.8290	0.4799s

Table 31: $n = 10000$: Broyden (MN) - Exact Derivatives

pt	k	mode	grad	iter/max	succ	flag	rate	time
1	4	adaptive	7.40e-03	4	yes	-	0.5137	0.2245s
2	4	adaptive	2.87e-03	7	yes	-	0.1759	0.4188s
3	4	adaptive	1.86e-03	7	yes	-	0.1135	0.5704s
4	4	adaptive	1.63e-03	7	yes	-	0.1123	0.5583s
5	4	adaptive	1.07e-03	7	yes	-	0.0640	0.5170s
6	4	adaptive	3.58e-03	7	yes	-	0.1888	0.4771s
1	4	fixed	8.99e-03	4	yes	-	0.4558	0.2237s
2	4	fixed	3.15e-03	7	yes	-	0.1245	0.5037s
3	4	fixed	2.06e-03	7	yes	-	0.0810	0.4410s
4	4	fixed	1.85e-03	7	yes	-	0.0806	0.4038s
5	4	fixed	1.20e-03	7	yes	-	0.0479	0.4065s
6	4	fixed	3.85e-03	7	yes	-	0.1331	0.4532s
1	8	adaptive	5.95e-08	5	yes	-	0.5331	0.3515s
2	8	adaptive	1.43e-07	8	yes	-	0.7976	0.5257s
3	8	adaptive	7.26e-08	8	yes	-	0.6725	0.6326s
4	8	adaptive	3.84e-08	8	yes	-	0.5465	0.5257s
5	8	adaptive	3.27e-08	8	yes	-	0.5223	0.5271s
6	8	adaptive	2.90e-07	8	yes	-	0.9448	0.5298s
1	8	fixed	7.17e-08	5	yes	-	0.4857	0.3027s
2	8	fixed	1.60e-07	8	yes	-	0.7315	0.5704s
3	8	fixed	8.22e-08	8	yes	-	0.6070	0.5318s
4	8	fixed	4.48e-08	8	yes	-	0.4862	0.5141s
5	8	fixed	3.75e-08	8	yes	-	0.4592	0.5866s
6	8	fixed	3.13e-07	8	yes	-	0.8717	0.4983s
1	12	adaptive	6.50e-08	5	yes	-	1.2832	0.1298s
2	12	adaptive	3.85e-10	12	yes	-	0.6966	0.3151s
3	12	adaptive	5.36e-07	9	yes	-	2.1360	0.2451s
4	12	adaptive	4.50e-08	12	yes	-	1.6857	0.3136s
5	12	adaptive	5.87e-11	10	yes	-	0.5675	0.2698s
6	12	adaptive	6.12e-07	11	yes	-	1.8221	0.2956s
1	12	fixed	3.27e-08	5	yes	-	1.5097	0.3424s
2	12	fixed	1.62e-07	8	yes	-	1.8663	0.5187s
3	12	fixed	7.96e-08	8	yes	-	1.7959	0.5254s
4	12	fixed	4.27e-08	8	yes	-	1.6989	0.5367s
5	12	fixed	3.15e-08	8	yes	-	1.8010	0.5358s
6	12	fixed	2.38e-07	8	yes	-	2.0102	0.5038s
Avg Success			-	7.56	36/36	-	0.7951	0.4396s

Table 32: $n = 10000$: Broyden (MN) - Finite Differences (With exact gradient)

pt	k	mode	grad	iter/max	succ	flag	rate	time
1	4	adaptive	7.40e-03	4	yes	-	0.5137	0.2245s
2	4	adaptive	2.87e-03	7	yes	-	0.1759	0.4188s
3	4	adaptive	1.86e-03	7	yes	-	0.1135	0.5704s
4	4	adaptive	1.63e-03	7	yes	-	0.1123	0.5583s
5	4	adaptive	1.07e-03	7	yes	-	0.0640	0.5170s
6	4	adaptive	3.58e-03	7	yes	-	0.1888	0.4771s
1	4	fixed	8.99e-03	4	yes	-	0.4558	0.2237s
2	4	fixed	3.15e-03	7	yes	-	0.1245	0.5037s
3	4	fixed	2.06e-03	7	yes	-	0.0810	0.4410s
4	4	fixed	1.85e-03	7	yes	-	0.0806	0.4038s
5	4	fixed	1.20e-03	7	yes	-	0.0479	0.4065s
6	4	fixed	3.85e-03	7	yes	-	0.1331	0.4532s
1	8	adaptive	5.95e-08	5	yes	-	0.5331	0.3515s
2	8	adaptive	1.43e-07	8	yes	-	0.7976	0.5257s
3	8	adaptive	7.26e-08	8	yes	-	0.6725	0.6326s
4	8	adaptive	3.84e-08	8	yes	-	0.5465	0.5257s
5	8	adaptive	3.27e-08	8	yes	-	0.5223	0.5271s
6	8	adaptive	2.90e-07	8	yes	-	0.9448	0.5298s
1	8	fixed	7.17e-08	5	yes	-	0.4857	0.3027s
2	8	fixed	1.60e-07	8	yes	-	0.7315	0.5704s
3	8	fixed	8.22e-08	8	yes	-	0.6070	0.5318s
4	8	fixed	4.48e-08	8	yes	-	0.4862	0.5141s
5	8	fixed	3.75e-08	8	yes	-	0.4592	0.5866s
6	8	fixed	3.13e-07	8	yes	-	0.8717	0.4983s
1	12	adaptive	6.50e-08	5	yes	-	1.2832	0.1298s
2	12	adaptive	3.85e-10	12	yes	-	0.6966	0.3151s
3	12	adaptive	5.36e-07	9	yes	-	2.1360	0.2451s
4	12	adaptive	4.50e-08	12	yes	-	1.6857	0.3136s
5	12	adaptive	5.87e-11	10	yes	-	0.5675	0.2698s
6	12	adaptive	6.12e-07	11	yes	-	1.8221	0.2956s
1	12	fixed	3.27e-08	5	yes	-	1.5097	0.3424s
2	12	fixed	1.62e-07	8	yes	-	1.8663	0.5187s
3	12	fixed	7.96e-08	8	yes	-	1.7959	0.5254s
4	12	fixed	4.27e-08	8	yes	-	1.6989	0.5367s
5	12	fixed	3.15e-08	8	yes	-	1.8010	0.5358s
6	12	fixed	2.38e-07	8	yes	-	2.0102	0.5038s
Avg Success			-	7.56	36/36	-	0.7951	0.4396s

Table 33: $n = 10000$: Broyden (MN) - Finite Differences (Approx All)

pt ID	grad	iter/max	succ	flag	rate	time
1	1.65e-08	12	yes	-	1.6506	0.1041s
2	1.22e-07	14	yes	-	1.8529	0.1085s
3	1.37e-09	15	yes	-	0.7074	0.1128s
4	5.12e-09	15	yes	-	1.6655	0.1049s
5	1.58e-07	14	yes	-	1.9648	0.0968s
6	1.59e-07	14	yes	-	1.6691	0.1017s
Avg	-	14.00	6/6	-	1.5851	0.1048s

Table 34: $n = 10000$: Broyden (TN) - Exact Derivatives

pt	k	mode	grad	iter/max	succ	flag	rate	time
1	4	adaptive	6.85e-03	9	yes	-	0.0872	0.0705s
2	4	adaptive	2.12e-02	161	no	fail	N/A	2.2666s
3	4	adaptive	1.37e-03	12	yes	-	0.0330	0.0941s
4	4	adaptive	6.82e-03	12	yes	-	0.0664	0.1171s
5	4	adaptive	2.19e-02	160	no	fail	N/A	1.9432s
6	4	adaptive	2.18e-02	160	no	fail	N/A	2.2217s
1	4	fixed	7.01e-03	9	yes	-	0.0843	0.0696s
2	4	fixed	6.81e-02	161	no	fail	N/A	2.0312s
3	4	fixed	1.26e-02	161	no	fail	N/A	1.9869s
4	4	fixed	6.75e-03	12	yes	-	0.0678	0.0796s
5	4	fixed	2.34e-02	162	no	fail	N/A	2.0148s
6	4	fixed	2.20e-02	160	no	fail	N/A	2.1390s
1	8	adaptive	6.56e-06	11	yes	-	0.5299	0.0876s
2	8	adaptive	3.10e-05	13	yes	-	0.9634	0.0990s
3	8	adaptive	1.84e-06	14	yes	-	0.4422	0.1250s
4	8	adaptive	3.97e-06	14	yes	-	0.5395	0.0988s
5	8	adaptive	3.40e-05	13	yes	-	0.9514	0.0922s
6	8	adaptive	3.29e-05	13	yes	-	0.9501	0.1397s
1	8	fixed	6.56e-06	11	yes	-	0.4444	0.0921s
2	8	fixed	3.10e-05	13	yes	-	0.9001	0.1015s
3	8	fixed	1.84e-06	14	yes	-	0.3515	0.0986s
4	8	fixed	3.98e-06	14	yes	-	0.4569	0.1021s
5	8	fixed	3.40e-05	13	yes	-	0.8980	0.0924s
6	8	fixed	3.29e-05	13	yes	-	0.8927	0.0964s
1	12	adaptive	1.66e-08	12	yes	-	1.6467	0.0424s
2	12	adaptive	1.22e-07	14	yes	-	1.8557	0.0527s
3	12	adaptive	1.38e-09	15	yes	-	0.7080	0.0468s
4	12	adaptive	5.34e-09	15	yes	-	1.6612	0.0519s
5	12	adaptive	1.58e-07	14	yes	-	1.9649	0.0478s
6	12	adaptive	1.60e-07	14	yes	-	1.6669	0.0511s
1	12	fixed	1.65e-08	12	yes	-	1.6465	0.1003s
2	12	fixed	1.22e-07	14	yes	-	1.8539	0.1102s
3	12	fixed	1.38e-09	15	yes	-	0.7070	0.1133s
4	12	fixed	5.15e-09	15	yes	-	1.6446	0.1170s
5	12	fixed	1.58e-07	14	yes	-	1.9647	0.1009s
6	12	fixed	1.60e-07	14	yes	-	1.6694	0.1047s
Avg Success			-	13.03	29/36	-	0.9534	0.0895s

Table 35: $n = 10000$: Broyden (TN) - Finite Differences (With exact gradient)

pt	k	mode	grad	iter/max	succ	flag	rate	time
1	4	adaptive	6.85e-03	9	yes	-	0.0872	0.0705s
2	4	adaptive	2.12e-02	161	no	fail	N/A	2.2666s
3	4	adaptive	1.37e-03	12	yes	-	0.0664	0.0941s
4	4	adaptive	6.82e-03	12	yes	-	0.0330	0.1171s
5	4	adaptive	2.19e-02	160	no	fail	N/A	1.9432s
6	4	adaptive	2.18e-02	160	no	fail	N/A	2.2217s
1	4	fixed	7.01e-03	9	yes	-	0.0843	0.0696s
2	4	fixed	6.81e-02	161	no	fail	N/A	2.0312s
3	4	fixed	1.26e-02	161	no	fail	N/A	1.9869s
4	4	fixed	6.75e-03	12	yes	-	0.0678	0.0796s
5	4	fixed	2.34e-02	162	no	fail	N/A	2.0148s
6	4	fixed	2.20e-02	160	no	fail	N/A	2.1390s
1	8	adaptive	6.56e-06	11	yes	-	0.5299	0.0876s
2	8	adaptive	3.10e-05	13	yes	-	0.9634	0.0990s
3	8	adaptive	1.84e-06	14	yes	-	0.4422	0.1250s
4	8	adaptive	3.97e-06	14	yes	-	0.5395	0.0988s
5	8	adaptive	3.40e-05	13	yes	-	0.9514	0.0922s
6	8	adaptive	3.29e-05	13	yes	-	0.9501	0.1397s
1	8	fixed	6.56e-06	11	yes	-	0.4444	0.0921s
2	8	fixed	3.10e-05	13	yes	-	0.9001	0.1015s
3	8	fixed	1.84e-06	14	yes	-	0.3515	0.0986s
4	8	fixed	3.98e-06	14	yes	-	0.4569	0.1021s
5	8	fixed	3.40e-05	13	yes	-	0.8980	0.0924s
6	8	fixed	3.29e-05	13	yes	-	0.8927	0.0964s
1	12	adaptive	1.66e-08	12	yes	-	1.6467	0.0424s
2	12	adaptive	1.22e-07	14	yes	-	1.8557	0.0527s
3	12	adaptive	1.38e-09	15	yes	-	0.7080	0.0468s
4	12	adaptive	5.34e-09	15	yes	-	1.6612	0.0519s
5	12	adaptive	1.58e-07	14	yes	-	1.9649	0.0478s
6	12	adaptive	1.60e-07	14	yes	-	1.6669	0.0511s
1	12	fixed	1.65e-08	12	yes	-	1.6465	0.1003s
2	12	fixed	1.22e-07	14	yes	-	1.8539	0.1102s
3	12	fixed	1.38e-09	15	yes	-	0.7070	0.1133s
4	12	fixed	5.15e-09	15	yes	-	1.6446	0.1170s
5	12	fixed	1.58e-07	14	yes	-	1.9647	0.1009s
6	12	fixed	1.60e-07	14	yes	-	1.6694	0.1047s
Avg Success			-	13.03	29/36	-	0.9534	0.0895s

Table 36: $n = 10000$: Broyden (TN) - Finite Differences (Approx All)

Analysis for Dimension $n = 100000$

pt ID	grad	iter/max	succ	flag	rate	time
1	8.64e-07	5	yes	-	2.4227	1.2673s
2	2.44e-09	10	yes	-	1.8043	2.4914s
3	1.64e-09	10	yes	-	1.7621	2.5840s
4	4.25e-08	10	yes	-	1.8430	2.4747s
5	3.57e-11	14	yes	-	1.8846	3.5017s
6	8.91e-09	10	yes	-	1.7768	2.4234s
Avg	-	9.83	6/6	-	1.9156	2.4571s

Table 37: $n = 100000$: Chained (MN) - Exact Derivatives

pt	k	mode	grad	iter/max	succ	flag	rate	time
1	4	adaptive	2.79e-02	26	yes	-	0.6778	6.0710s
2	4	adaptive	3.00e-02	27	yes	-	0.6826	5.8684s
3	4	adaptive	2.28e-02	29	yes	-	0.6678	5.6119s
4	4	adaptive	2.92e-02	28	yes	-	0.6809	5.8288s
5	4	adaptive	2.85e-02	28	yes	-	0.6792	7.1520s
6	4	adaptive	2.94e-02	28	yes	-	0.6812	8.2047s
1	4	fixed	2.78e-02	26	yes	-	0.7130	5.2049s
2	4	fixed	2.98e-02	27	yes	-	0.7222	5.7794s
3	4	fixed	2.27e-02	29	yes	-	0.6912	5.9507s
4	4	fixed	2.91e-02	28	yes	-	0.7189	5.6268s
5	4	fixed	2.84e-02	28	yes	-	0.7157	5.5923s
6	4	fixed	2.93e-02	28	yes	-	0.7196	5.5860s
1	8	adaptive	1.33e-03	176	no	fail	N/A	56.7533s
2	8	adaptive	6.04e-04	177	no	fail	N/A	77.1777s
3	8	adaptive	5.79e-04	178	no	fail	N/A	83.9051s
4	8	adaptive	6.81e-04	178	no	fail	N/A	68.3859s
5	8	adaptive	4.54e-04	178	no	fail	N/A	76.0429s
6	8	adaptive	6.41e-04	178	no	fail	N/A	63.3760s
1	8	fixed	2.65e-04	41	yes	-	0.9835	12.2049s
2	8	fixed	9.97e-04	177	no	fail	N/A	57.9261s
3	8	fixed	7.62e-04	178	no	fail	N/A	44.9538s
4	8	fixed	8.79e-04	178	no	fail	N/A	34.8918s
5	8	fixed	7.78e-04	178	no	fail	N/A	53.3126s
6	8	fixed	8.89e-04	178	no	fail	N/A	56.3287s
1	12	adaptive	8.63e-01	184	no	fail	N/A	46.5069s
2	12	adaptive	1.11e+05	465	no	fail	N/A	455.8451s
3	12	adaptive	1.11e+05	468	no	fail	N/A	465.8968s
4	12	adaptive	1.09e+05	487	no	fail	N/A	480.7557s
5	12	adaptive	1.25e+05	490	no	fail	N/A	535.8344s
6	12	adaptive	1.42e+05	472	no	fail	N/A	470.4651s
1	12	fixed	3.65e+00	315	no	fail	N/A	424.3719s
2	12	fixed	7.69e+00	309	no	fail	N/A	81.2756s
3	12	fixed	7.70e+00	307	no	fail	N/A	86.3028s
4	12	fixed	7.67e+00	318	no	fail	N/A	82.7206s
5	12	fixed	7.68e+00	319	no	fail	N/A	86.5019s
6	12	fixed	7.67e+00	307	no	fail	N/A	69.5669s
Avg Success			-	28.69	13/36	-	0.7180	6.5140s

Table 38: $n = 100000$: Chained (MN) - Finite Differences (With exact gradient)

pt	k	mode	grad	iter/max	succ	flag	rate	time
1	4	adaptive	2.79e-02	26	yes	-	0.6778	6.0710s
2	4	adaptive	3.00e-02	27	yes	-	0.6826	5.8684s
3	4	adaptive	2.28e-02	29	yes	-	0.6678	5.6119s
4	4	adaptive	2.92e-02	28	yes	-	0.6809	5.8288s
5	4	adaptive	2.85e-02	28	yes	-	0.6792	7.1520s
6	4	adaptive	2.94e-02	28	yes	-	0.6812	8.2047s
1	4	fixed	2.78e-02	26	yes	-	0.7130	5.2049s
2	4	fixed	2.98e-02	27	yes	-	0.7222	5.7794s
3	4	fixed	2.27e-02	29	yes	-	0.6912	5.9507s
4	4	fixed	2.91e-02	28	yes	-	0.7189	5.6268s
5	4	fixed	2.84e-02	28	yes	-	0.7157	5.5923s
6	4	fixed	2.93e-02	28	yes	-	0.7196	5.5860s
1	8	adaptive	1.33e-03	176	no	fail	N/A	56.7533s
2	8	adaptive	6.04e-04	177	no	fail	N/A	77.1777s
3	8	adaptive	5.79e-04	178	no	fail	N/A	83.9051s
4	8	adaptive	6.81e-04	178	no	fail	N/A	68.3859s
5	8	adaptive	4.54e-04	178	no	fail	N/A	76.0429s
6	8	adaptive	6.41e-04	178	no	fail	N/A	63.3760s
1	8	fixed	2.65e-04	41	yes	-	0.9835	12.2049s
2	8	fixed	9.97e-04	177	no	fail	N/A	57.9261s
3	8	fixed	7.62e-04	178	no	fail	N/A	44.9538s
4	8	fixed	8.79e-04	178	no	fail	N/A	34.8918s
5	8	fixed	7.78e-04	178	no	fail	N/A	53.3126s
6	8	fixed	8.89e-04	178	no	fail	N/A	56.3287s
1	12	adaptive	8.63e-01	184	no	fail	N/A	46.5069s
2	12	adaptive	1.11e+05	180	no	fail	N/A	455.8451s
3	12	adaptive	1.11e+05	181	no	fail	N/A	465.8968s
4	12	adaptive	1.09e+05	178	no	fail	N/A	480.7557s
5	12	adaptive	1.25e+05	189	no	fail	N/A	535.8344s
6	12	adaptive	1.42e+05	177	no	fail	N/A	470.4651s
1	12	fixed	3.65e+00	180	no	fail	N/A	424.3719s
2	12	fixed	7.69e+00	176	no	fail	N/A	81.2756s
3	12	fixed	7.70e+00	177	no	fail	N/A	86.3028s
4	12	fixed	7.67e+00	178	no	fail	N/A	82.7206s
5	12	fixed	7.68e+00	176	no	fail	N/A	86.5019s
6	12	fixed	7.67e+00	180	no	fail	N/A	69.5669s
Avg Success			-	28.69	13/36	-	0.7180	6.5140s

Table 39: $n = 100000$: Chained (MN) - Finite Differences (Approx All)

pt ID	grad	iter/max	succ	flag	rate	time
1	1.74e-06	14	yes	-	0.0920	0.9705s
2	1.99e-06	17	yes	-	0.0714	1.2107s
3	2.17e-06	16	yes	-	0.0664	1.1774s
4	2.36e-06	16	yes	-	0.0663	1.1433s
5	2.17e-06	21	yes	-	0.0297	1.4855s
6	2.79e-06	16	yes	-	0.0663	1.4577s
Avg	-	16.67	6/6	-	0.0654	1.2409s

Table 40: $n = 100000$: Chained (TN) - Exact Derivatives

pt	k	mode	grad	iter/max	succ	flag	rate	time
1	4	adaptive	2.92e-02	36	yes	-	0.0019	1.5742s
2	4	adaptive	2.84e-02	39	yes	-	0.0026	1.2268s
3	4	adaptive	2.93e-02	37	yes	-	0.0026	1.0108s
4	4	adaptive	2.99e-02	37	yes	-	0.0026	1.2452s
5	4	adaptive	2.13e-02	47	yes	-	0.0018	2.0381s
6	4	adaptive	2.12e-02	39	yes	-	0.0025	1.4073s
1	4	fixed	2.90e-02	36	yes	-	0.0018	0.9887s
2	4	fixed	2.84e-02	39	yes	-	0.0026	1.0504s
3	4	fixed	2.92e-02	37	yes	-	0.0026	0.9531s
4	4	fixed	2.98e-02	37	yes	-	0.0026	0.9624s
5	4	fixed	2.13e-02	46	yes	-	0.0019	1.2127s
6	4	fixed	2.11e-02	39	yes	-	0.0026	1.7049s
1	8	adaptive	2.67e-04	63	yes	-	0.8331	2.3442s
2	8	adaptive	3.50e-04	199	no	fail	N/A	14.4547s
3	8	adaptive	6.22e-04	197	no	fail	N/A	8.8337s
4	8	adaptive	6.05e-04	197	no	fail	N/A	17.7723s
5	8	adaptive	5.73e-04	204	no	fail	N/A	13.9628s
6	8	adaptive	6.44e-04	197	no	fail	N/A	10.3478s
1	8	fixed	2.69e-04	63	yes	-	1.9840	2.6464s
2	8	fixed	9.53e-04	199	no	fail	N/A	8.7527s
3	8	fixed	9.91e-04	197	no	fail	N/A	7.1901s
4	8	fixed	8.30e-04	197	no	fail	N/A	11.7109s
5	8	fixed	8.37e-04	204	no	fail	N/A	11.5782s
6	8	fixed	8.35e-04	197	no	fail	N/A	10.8564s
1	12	adaptive	3.53e-01	315	no	fail	N/A	8.7092s
2	12	adaptive	6.15e+00	207	no	fail	N/A	45.1720s
3	12	adaptive	6.19e+00	213	no	fail	N/A	57.2858s
4	12	adaptive	6.18e+00	198	no	fail	N/A	94.2029s
5	12	adaptive	6.16e+00	197	no	fail	N/A	74.3629s
6	12	adaptive	6.19e+00	199	no	fail	N/A	59.9901s
1	12	fixed	3.80e-01	197	no	fail	N/A	8.4137s
2	12	fixed	1.00e+01	204	no	fail	N/A	60.3770s
3	12	fixed	9.70e+00	197	no	fail	N/A	55.6510s
4	12	fixed	1.05e+01	201	no	fail	N/A	49.8957s
5	12	fixed	8.52e+00	199	no	fail	N/A	61.2502s
6	12	fixed	1.06e+01	206	no	fail	N/A	47.5099s
Avg Success			-	42.50	14/36	-	0.2032	1.4547s

Table 41: $n = 100000$: Chained (TN) - Finite Differences (With exact gradient)

pt	k	mode	grad	iter/max	succ	flag	rate	time
1	4	adaptive	2.92e-02	36	yes	-	0.0019	1.5742s
2	4	adaptive	2.84e-02	39	yes	-	0.0026	1.2268s
3	4	adaptive	2.93e-02	37	yes	-	0.0026	1.0108s
4	4	adaptive	2.99e-02	37	yes	-	0.0026	1.2452s
5	4	adaptive	2.13e-02	47	yes	-	0.0018	2.0381s
6	4	adaptive	2.12e-02	39	yes	-	0.0025	1.4073s
1	4	fixed	2.90e-02	36	yes	-	0.0018	0.9887s
2	4	fixed	2.84e-02	39	yes	-	0.0026	1.0504s
3	4	fixed	2.92e-02	37	yes	-	0.0026	0.9531s
4	4	fixed	2.98e-02	37	yes	-	0.0026	0.9624s
5	4	fixed	2.13e-02	46	yes	-	0.0019	1.2127s
6	4	fixed	2.11e-02	39	yes	-	0.0026	1.7049s
1	8	adaptive	2.67e-04	63	yes	-	0.8331	2.3442s
2	8	adaptive	3.50e-04	199	no	fail	N/A	14.4547s
3	8	adaptive	6.22e-04	197	no	fail	N/A	8.8337s
4	8	adaptive	6.05e-04	197	no	fail	N/A	17.7723s
5	8	adaptive	5.73e-04	204	no	fail	N/A	13.9628s
6	8	adaptive	6.44e-04	197	no	fail	N/A	10.3478s
1	8	fixed	2.69e-04	63	yes	-	1.1140	2.6464s
2	8	fixed	9.53e-04	199	no	fail	N/A	8.7527s
3	8	fixed	9.91e-04	197	no	fail	N/A	7.1901s
4	8	fixed	8.30e-04	197	no	fail	N/A	11.7109s
5	8	fixed	8.37e-04	204	no	fail	N/A	11.5782s
6	8	fixed	8.35e-04	197	no	fail	N/A	10.8564s
1	12	adaptive	3.53e-01	315	no	fail	N/A	8.7092s
2	12	adaptive	6.15e+00	201	no	fail	N/A	45.1720s
3	12	adaptive	6.19e+00	199	no	fail	N/A	57.2858s
4	12	adaptive	6.18e+00	197	no	fail	N/A	94.2029s
5	12	adaptive	6.16e+00	198	no	fail	N/A	74.3629s
6	12	adaptive	6.19e+00	204	no	fail	N/A	59.9901s
1	12	fixed	3.80e-01	197	no	fail	N/A	8.4137s
2	12	fixed	1.00e+01	205	no	fail	N/A	60.3770s
3	12	fixed	9.70e+00	196	no	fail	N/A	55.6510s
4	12	fixed	1.05e+01	198	no	fail	N/A	49.8957s
5	12	fixed	8.52e+00	200	no	fail	N/A	61.2502s
6	12	fixed	1.06e+01	199	no	fail	N/A	47.5099s
Avg Success			-	42.50	14/36	-	0.1411	1.4547s

Table 42: $n = 100000$: Chained (TN) - Finite Differences (Approx All)

pt ID	grad	iter/max	succ	flag	rate	time
1	1.79e-08	5	yes	-	1.9705	1.8415s
2	7.45e-09	8	yes	-	1.7176	6.4731s
3	6.27e-09	8	yes	-	1.7355	3.9022s
4	2.58e-08	8	yes	-	1.7526	2.8915s
5	1.57e-08	8	yes	-	1.6942	2.9724s
6	1.67e-09	8	yes	-	1.7251	2.7233s
Avg	-	7.50	6/6	-	1.7659	3.4673s

Table 43: $n = 100000$: Broyden (MN) - Exact Derivatives

pt	k	mode	grad	iter/max	succ	flag	rate	time
1	4	adaptive	2.32e-02	4	yes	-	0.5121	1.3655s
2	4	adaptive	1.70e-03	7	yes	-	0.0358	3.1875s
3	4	adaptive	1.67e-03	7	yes	-	0.0356	2.9282s
4	4	adaptive	2.88e-03	7	yes	-	0.0556	3.2464s
5	4	adaptive	1.99e-03	7	yes	-	0.0403	3.7865s
6	4	adaptive	1.14e-03	7	yes	-	0.0279	2.5538s
1	4	fixed	2.82e-02	4	yes	-	0.4545	2.6512s
2	4	fixed	1.96e-03	7	yes	-	0.0298	3.6989s
3	4	fixed	1.93e-03	7	yes	-	0.0297	2.6504s
4	4	fixed	3.25e-03	7	yes	-	0.0426	2.4736s
5	4	fixed	2.28e-03	7	yes	-	0.0328	2.5408s
6	4	fixed	1.34e-03	7	yes	-	0.0249	2.4915s
1	8	adaptive	1.33e-07	5	yes	-	0.4687	3.3122s
2	8	adaptive	3.65e-08	8	yes	-	0.3265	4.2570s
3	8	adaptive	3.30e-08	8	yes	-	0.3063	3.7233s
4	8	adaptive	8.42e-08	8	yes	-	0.4728	4.5226s
5	8	adaptive	5.16e-08	8	yes	-	0.3768	4.5803s
6	8	adaptive	2.08e-04	7	yes	-	1.6986	2.9600s
1	8	fixed	1.62e-07	5	yes	-	0.4260	2.0068s
2	8	fixed	4.25e-08	8	yes	-	0.2695	4.5011s
3	8	fixed	3.85e-08	8	yes	-	0.2507	5.0998s
4	8	fixed	9.64e-08	8	yes	-	0.4111	4.8327s
5	8	fixed	5.91e-08	8	yes	-	0.3174	4.1185s
6	8	fixed	2.09e-04	7	yes	-	1.6832	4.1188s
1	12	adaptive	6.59e-08	5	yes	-	1.2259	1.6907s
2	12	adaptive	5.29e-07	8	yes	-	1.8414	3.7604s
3	12	adaptive	6.73e-08	9	yes	-	2.0647	4.0047s
4	12	adaptive	2.79e-08	8	yes	-	1.2976	3.6201s
5	12	adaptive	1.22e-06	11	yes	-	1.9183	4.4598s
6	12	adaptive	4.63e-07	13	yes	-	2.0170	5.4192s
1	12	fixed	6.55e-08	5	yes	-	1.3960	1.7636s
2	12	fixed	4.96e-08	8	yes	-	1.5909	2.7949s
3	12	fixed	3.38e-08	8	yes	-	1.6406	3.1427s
4	12	fixed	1.06e-07	8	yes	-	1.7075	3.1329s
5	12	fixed	6.27e-08	8	yes	-	1.7008	3.0170s
6	12	fixed	1.55e-08	8	yes	-	1.5841	3.1910s
Avg Success			-	7.36	36/36	-	0.7865	3.3779s

Table 44: $n = 100000$: Broyden (MN) - Finite Differences (With exact gradient)

pt	k	mode	grad	iter/max	succ	flag	rate	time
1	4	adaptive	2.32e-02	4	yes	-	0.5121	1.3655s
2	4	adaptive	1.70e-03	7	yes	-	0.0358	3.1875s
3	4	adaptive	1.67e-03	7	yes	-	0.0356	2.9282s
4	4	adaptive	2.88e-03	7	yes	-	0.0556	3.2464s
5	4	adaptive	1.99e-03	7	yes	-	0.0403	3.7865s
6	4	adaptive	1.14e-03	7	yes	-	0.0279	2.5538s
1	4	fixed	2.82e-02	4	yes	-	0.4545	2.6512s
2	4	fixed	1.96e-03	7	yes	-	0.0298	3.6989s
3	4	fixed	1.93e-03	7	yes	-	0.0297	2.6504s
4	4	fixed	3.25e-03	7	yes	-	0.0426	2.4736s
5	4	fixed	2.28e-03	7	yes	-	0.0328	2.5408s
6	4	fixed	1.34e-03	7	yes	-	0.0249	2.4915s
1	8	adaptive	1.33e-07	5	yes	-	0.4687	3.3122s
2	8	adaptive	3.65e-08	8	yes	-	0.3265	4.2570s
3	8	adaptive	3.30e-08	8	yes	-	0.3063	3.7233s
4	8	adaptive	8.42e-08	8	yes	-	0.4728	4.5226s
5	8	adaptive	5.16e-08	8	yes	-	0.3768	4.5803s
6	8	adaptive	2.08e-04	7	yes	-	1.6986	2.9600s
1	8	fixed	1.62e-07	5	yes	-	0.4260	2.0068s
2	8	fixed	4.25e-08	8	yes	-	0.2695	4.5011s
3	8	fixed	3.85e-08	8	yes	-	0.2507	5.0998s
4	8	fixed	9.64e-08	8	yes	-	0.4111	4.8327s
5	8	fixed	5.91e-08	8	yes	-	0.3174	4.1185s
6	8	fixed	2.09e-04	7	yes	-	1.6832	4.1188s
1	12	adaptive	6.59e-08	5	yes	-	1.2259	1.6907s
2	12	adaptive	5.29e-07	8	yes	-	1.8414	3.7604s
3	12	adaptive	6.73e-08	9	yes	-	2.0647	4.0047s
4	12	adaptive	2.79e-08	8	yes	-	1.2976	3.6201s
5	12	adaptive	1.22e-06	11	yes	-	1.9183	4.4598s
6	12	adaptive	4.63e-07	13	yes	-	2.0170	5.4192s
1	12	fixed	6.55e-08	5	yes	-	1.3960	1.7636s
2	12	fixed	4.96e-08	8	yes	-	1.5909	2.7949s
3	12	fixed	3.38e-08	8	yes	-	1.6406	3.1427s
4	12	fixed	1.06e-07	8	yes	-	1.7075	3.1329s
5	12	fixed	6.27e-08	8	yes	-	1.7008	3.0170s
6	12	fixed	1.55e-08	8	yes	-	1.5841	3.1910s
Avg Success			-	7.36	36/36	-	0.7865	3.3779s

Table 45: $n = 100000$: Broyden (MN) - Finite Differences (Approx All)

pt ID	grad	iter/max	succ	flag	rate	time
1	5.73e-09	15	yes	-	1.0967	1.3185s
2	3.35e-08	15	yes	-	1.4069	1.0446s
3	2.36e-07	15	yes	-	1.8424	0.6743s
4	8.91e-07	16	yes	-	1.2787	0.6410s
5	1.35e-06	15	yes	-	2.0231	0.6494s
6	3.35e-08	15	yes	-	1.5145	0.7403s
Avg	-	15.17	6/6	-	1.5271	0.8447s

Table 46: $n = 100000$: Broyden (TN) - Exact Derivatives

pt	k	mode	grad	iter/max	succ	flag	rate	time
1	4	adaptive	2.01e-02	11	yes	-	3.1823	0.6182s
2	4	adaptive	1.64e-02	12	yes	-	0.0023	0.6796s
3	4	adaptive	1.85e-02	12	yes	-	0.0017	0.7497s
4	4	adaptive	2.16e-02	12	yes	-	0.0217	0.6618s
5	4	adaptive	2.21e-02	12	yes	-	0.0156	0.8289s
6	4	adaptive	1.89e-02	12	yes	-	0.0110	0.9659s
1	4	fixed	1.65e-02	11	yes	-	1.0724	1.0572s
2	4	fixed	1.65e-02	12	yes	-	0.0186	0.6903s
3	4	fixed	1.87e-02	12	yes	-	0.0069	0.6222s
4	4	fixed	2.16e-02	12	yes	-	0.0203	0.5277s
5	4	fixed	2.22e-02	12	yes	-	0.0132	0.5488s
6	4	fixed	1.90e-02	12	yes	-	0.0062	0.6750s
1	8	adaptive	2.76e-04	13	yes	-	2.4231	1.0437s
2	8	adaptive	1.25e-05	14	yes	-	0.5952	1.0109s
3	8	adaptive	5.58e-05	14	yes	-	0.9348	0.8732s
4	8	adaptive	9.85e-05	15	yes	-	2.6381	1.0567s
5	8	adaptive	1.35e-04	14	yes	-	0.8377	0.6481s
6	8	adaptive	1.49e-05	14	yes	-	0.5884	1.0134s
1	8	fixed	2.76e-04	13	yes	-	2.3585	0.8543s
2	8	fixed	1.25e-05	14	yes	-	0.4982	0.9816s
3	8	fixed	5.58e-05	14	yes	-	0.8415	0.7684s
4	8	fixed	9.85e-05	15	yes	-	2.4462	0.9969s
5	8	fixed	1.35e-04	14	yes	-	0.7908	0.9607s
6	8	fixed	1.49e-05	14	yes	-	0.4957	1.0808s
1	12	adaptive	5.53e-09	15	yes	-	1.0900	0.8579s
2	12	adaptive	3.34e-08	15	yes	-	1.3942	1.0870s
3	12	adaptive	2.59e-07	15	yes	-	1.8383	0.8148s
4	12	adaptive	8.91e-07	16	yes	-	1.2775	1.3160s
5	12	adaptive	5.68e-07	16	yes	-	1.5401	0.8964s
6	12	adaptive	7.68e-08	17	yes	-	2.0863	1.2089s
1	12	fixed	5.67e-09	15	yes	-	1.0887	0.7308s
2	12	fixed	3.35e-08	15	yes	-	1.3824	0.7299s
3	12	fixed	2.36e-07	15	yes	-	1.8416	0.8771s
4	12	fixed	8.92e-07	16	yes	-	1.2774	1.0171s
5	12	fixed	1.35e-06	15	yes	-	2.0224	0.7676s
6	12	fixed	3.36e-08	15	yes	-	1.4797	0.6837s
Avg Success			-	13.75	36/36	-	1.0594	0.8584s

Table 47: $n = 100000$: Broyden (TN) - Finite Differences (With exact gradient)

pt	k	mode	grad	iter/max	succ	flag	rate	time
1	4	adaptive	2.01e-02	11	yes	-	3.1823	0.6182s
2	4	adaptive	1.64e-02	12	yes	-	0.0023	0.6796s
3	4	adaptive	1.85e-02	12	yes	-	0.0017	0.7497s
4	4	adaptive	2.16e-02	12	yes	-	0.0217	0.6618s
5	4	adaptive	2.21e-02	12	yes	-	0.0156	0.8289s
6	4	adaptive	1.89e-02	12	yes	-	0.0110	0.9659s
1	4	fixed	1.65e-02	11	yes	-	1.0724	1.0572s
2	4	fixed	1.65e-02	12	yes	-	0.0186	0.6903s
3	4	fixed	1.87e-02	12	yes	-	0.0069	0.6222s
4	4	fixed	2.16e-02	12	yes	-	0.0203	0.5277s
5	4	fixed	2.22e-02	12	yes	-	0.0132	0.5488s
6	4	fixed	1.90e-02	12	yes	-	0.0062	0.6750s
1	8	adaptive	2.76e-04	13	yes	-	2.4231	1.0437s
2	8	adaptive	1.25e-05	14	yes	-	0.5952	1.0109s
3	8	adaptive	5.58e-05	14	yes	-	0.9348	0.8732s
4	8	adaptive	9.85e-05	15	yes	-	2.6381	1.0567s
5	8	adaptive	1.35e-04	14	yes	-	0.8377	0.6481s
6	8	adaptive	1.49e-05	14	yes	-	0.5884	1.0134s
1	8	fixed	2.76e-04	13	yes	-	2.3585	0.8543s
2	8	fixed	1.25e-05	14	yes	-	0.4982	0.9816s
3	8	fixed	5.58e-05	14	yes	-	0.8415	0.7684s
4	8	fixed	9.85e-05	15	yes	-	2.4462	0.9969s
5	8	fixed	1.35e-04	14	yes	-	0.7908	0.9607s
6	8	fixed	1.49e-05	14	yes	-	0.4957	1.0808s
1	12	adaptive	5.53e-09	15	yes	-	1.0900	0.8579s
2	12	adaptive	3.34e-08	15	yes	-	1.3942	1.0870s
3	12	adaptive	2.59e-07	15	yes	-	1.8383	0.8148s
4	12	adaptive	8.91e-07	16	yes	-	1.2775	1.3160s
5	12	adaptive	5.68e-07	16	yes	-	1.5401	0.8964s
6	12	adaptive	7.68e-08	17	yes	-	2.0863	1.2089s
1	12	fixed	5.67e-09	15	yes	-	1.0887	0.7308s
2	12	fixed	3.35e-08	15	yes	-	1.3824	0.7299s
3	12	fixed	2.36e-07	15	yes	-	1.8416	0.8771s
4	12	fixed	8.92e-07	16	yes	-	1.2774	1.0171s
5	12	fixed	1.35e-06	15	yes	-	2.0224	0.7676s
6	12	fixed	3.36e-08	15	yes	-	1.4797	0.6837s
Avg Success			-	13.75	36/36	-	1.0594	0.8584s

Table 48: $n = 100000$: Broyden (TN) - Finite Differences (Approx All)